

1. Problem Statement

Group projects are commonly used in universities to develop teamwork and collaboration skills among students. However, evaluating individual performance in group assignments is often difficult for teachers. In many cases, a small number of students complete most of the project work, while some members contribute very little. Despite the unequal participation, all students usually receive the same marks. This creates dissatisfaction among active students and reduces accountability within the team.

Although tools like GitHub can track code commits, they cannot fully measure other important activities such as communication, attendance, teamwork, and task participation. Manual peer evaluations are also not always reliable because they may be biased or incomplete.

From the students' perspective, participation in group projects is often hindered by a lack of transparency in grading, no clear breakdown of individual contributions, and uncertainty regarding how their efforts translate into final grades. Moreover, teachers lack a centralized system to monitor group dynamics in real-time, identify low contributors early, and generate fair, data-driven evaluation reports.

To overcome these limitations, there is a pressing need for a centralized, secure, and automated group project management solution. The proposed **GroupSync** system aims to address these challenges by providing a web-based, role-driven platform that supports secure authentication, course and group management, GitHub activity integration, peer review submission, automatic contribution scoring, and transparent report generation. By integrating attendance tracking, working hours, and analytics, the system seeks to enhance fairness, accountability, and efficiency throughout the entire group project lifecycle.

2. Inception

2.1 Introduction

Inception is the initial and foundational phase of the requirements engineering process. Its primary purpose is to establish a clear understanding of the problem domain, define the scope of the software system, and identify the needs, expectations, and constraints of all relevant stakeholders. During this phase, the development team works closely with stakeholders to gather initial requirements, identify potential risks, and ensure that the project has a clear direction before proceeding to detailed analysis and design.

For the proposed **GroupSync – Intelligent Group Project Management with Contribution Analytics** system, the inception phase was conducted to thoroughly understand the challenges associated with evaluating individual contributions in university group projects. The team analyzed existing manual and semi-digital evaluation methods, identified stakeholder concerns regarding fairness and transparency, and defined the overall feasibility of developing a centralized, role-driven platform.

The inception activities for GroupSync were organized around the following four aspects:

1. Identifying Stakeholders
2. Recognizing Multiple Viewpoints
3. Working Towards Collaboration
4. Asking the First Questions

2.2 Identifying Stakeholders

A stakeholder refers to any individual, group, or organization that is directly or indirectly affected by the development, implementation, or operation of a system. Stakeholders include end-users who interact with the system, as well as other parties whose workflows, decisions, or responsibilities may be influenced by the system's outcomes.

To identify the stakeholders of the GroupSync System, the development team analyzed the academic group project environment within a university setting. The identification process focused on understanding who uses the system, who benefits from its outputs, who manages its operations, and whose work is affected by its implementation.

Based on this analysis, the following stakeholders were identified for the GroupSync System:

1. Students (Team Members)

Students are the primary end-users of the system. They participate in group projects, collaborate with team members, submit peer reviews, connect their GitHub repositories, and track their individual contribution scores. Their primary interest is a fair, transparent, and easy-to-use system that recognizes their efforts accurately.

2. Teachers (Professors / Course Instructors)

Teachers are responsible for designing group project assignments, evaluating student performance, and assigning final grades. They use the system to create courses, set evaluation weightage, monitor group progress, generate reports, and identify low-contributing students. Their primary concern is having reliable, data-driven insights to support fair grading.

3. System Administrator (Admin)

The system administrator manages user accounts, assigns roles, monitors platform-wide activities, and ensures system security and stability. This stakeholder plays a critical role in maintaining data integrity, handling support requests, and configuring system-wide parameters.

4. University Administration

University administration oversees academic policies, grading standards, and institutional governance. Although they may not directly use the system, they are indirectly affected by its

outcomes, as the credibility and fairness of project evaluations reflect on the institution's academic integrity and policy enforcement.

5. System Developers

The development team is responsible for system design, implementation, testing, deployment, and future enhancements. Their work is influenced by requirements clarity, scalability demands, security considerations, and performance expectations.

6. System Operators

System operators are responsible for day-to-day technical operations, including monitoring system performance, managing backups, ensuring high availability during peak submission periods, and handling routine maintenance tasks.

2.3 Viewpoints of Stakeholders

Different stakeholders have distinct expectations, priorities, and concerns regarding the GroupSync System. Understanding these viewpoints is essential to capture a complete, realistic, and conflict-free set of system requirements.

2.3.1 System Administrator (Admin) Viewpoints

- Centralized control over user accounts, roles, permissions, and platform-wide configurations.
- Ability to create, modify, and delete student and teacher accounts as needed.
- Strong authentication mechanisms and role-based access control to prevent unauthorized access.
- Real-time monitoring of system activities, active user sessions, and platform health.
- Access to detailed system logs and audit trails for security auditing and accountability.
- High system availability, especially during peer review deadlines and project submission periods.
- Capability to manage backup schedules and data recovery procedures.

2.3.2 Teachers (Professors) Viewpoints

- Easy and intuitive creation and management of courses, group project assignments, and deadlines.
- Ability to define and adjust evaluation weightage for different contribution metrics (e.g., GitHub commits, attendance, peer reviews, working hours).
- Clear visualization of each student's individual contribution score (0-100) with a detailed breakdown.

- Automated flagging of students whose contribution score falls below a predefined threshold.
- Generation of downloadable reports (PDF and Excel formats) for final grading and record-keeping.
- Access to visual analytics dashboards showing contribution distribution, group comparisons, and low performers.
- Assurance that peer reviews are submitted anonymously and are free from bias.
- Ability to leave feedback or warnings for individual students or entire groups.

2.3.3 Students (Team Members) Viewpoints

- A simple, clean, and user-friendly interface for managing group activities and tracking contributions.
- Secure login and clear visibility of personal contribution breakdown (commits, attendance, peer review ratings, working hours).
- Ability to create a new group or join an existing group using a unique invitation code.
- Ability to connect their GitHub repository and automatically track commit activity.
- Assurance that submitted peer reviews remain anonymous and will not affect team relationships negatively.
- Confidence that their hard work will be recognized and fairly evaluated.
- Receive timely notifications for pending peer reviews, low contribution scores, or teacher feedback.
- Access to a personal dashboard showing contribution trends over time.

2.3.4 University Administration Viewpoints

- Credibility, fairness, and integrity of the overall project evaluation process across all courses.
- A structured, rule-based, and data-driven assessment of student collaboration and teamwork skills.
- Availability of audit records and historical reports for accountability and dispute resolution.
- Minimization of legal, reputational, and academic risks associated with unfair grading complaints.
- Reliable and consistent evaluation outcomes that support institutional decision-making.

2.3.5 System Developers Viewpoints

- Clear, unambiguous, and well-documented system requirements.
- Scalable and modular system architecture (e.g., Laravel/Node.js backend, MySQL database, React.js frontend).
- Efficient performance during high concurrent usage, particularly near submission deadlines.
- Secure implementation following industry best practices (e.g., password hashing, input validation, API security).
- Well-documented REST APIs for GitHub integration and internal operations.
- Support for comprehensive testing, debugging, and future feature enhancements.

2.3.6 System Operators Viewpoints

- Easy maintainability through modular code structure, clear documentation, and standardized deployment procedures.
- Availability of monitoring tools, error logs, and health checks for quick issue resolution.
- Reliable backup and recovery mechanisms to prevent loss of contribution data.
- Stable system operation throughout the academic semester, especially during critical periods.
- Safe deployment of updates and patches with minimal downtime.

2.4 Working Towards Collaboration

Because different stakeholders have diverse and sometimes conflicting expectations, a collaborative requirements consolidation process was conducted. The objective was to merge stakeholder needs into a coherent, practical, and mutually acceptable set of initial system requirements.

The following steps were followed to complete this task:

1. Identification of common and conflicting requirements across all stakeholder groups.
2. Categorization of requirements based on functionality, security, usability, and governance.
3. Assignment of priority points to each requirement through stakeholder discussions and voting.
4. Final decision-making based on priority, technical feasibility, and project scope.

2.4.1 Common Requirements

The following requirements were identified as common across most stakeholder groups:

1. A user-friendly, intuitive, and efficient system interface.
2. Secure and reliable system operation with minimal downtime.
3. Role-based access control to ensure appropriate permissions.
4. Internet-based accessibility from approved devices and locations.
5. Accurate, transparent, and automated generation of contribution scores.

2.4.2 Conflicting Requirements

Some requirements differed across stakeholders due to varying responsibilities and priorities. These conflicts were identified and resolved through negotiation and prioritization:

Conflicting Area	Stakeholder Concern
System complexity vs. advanced analytics	Teachers requested detailed analytics (charts, trends, comparisons), while developers preferred a simpler implementation due to time constraints.
Ease of access vs. strict permissions	Students wanted quick and easy access, while Admins and Teachers demanded strict role-based restrictions to protect data integrity.
Feature completeness vs. budget/time	All stakeholders desired a full feature set, but budget and timeline limitations required prioritization of core features.
Peer review flexibility vs. bias prevention	Students wanted flexible peer review submission, while Teachers expressed concern about potential biased or dishonest reviews.

Resolution Approach:

These conflicts were resolved through structured discussions, prioritization matrices, and consensus-building. Critical requirements related to security, core functionality (score calculation, GitHub integration), and fairness (anonymous peer review, audit logs) were given the highest priority. Advanced analytics and visualization features were planned for later sprints or future enhancements.

2.4.3 Initial Final Requirements

Based on the collaborative analysis, prioritization process, and feasibility assessment, the following **20 initial final requirements** were agreed upon for the GroupSync System:

1. The system shall be accessible via the Internet during the entire academic semester.

2. The system shall allow valid users (Students, Teachers, Admins) to securely log in and log out using authenticated credentials.
3. The system shall restrict access to system functionalities based on predefined user roles (Student, Teacher, Admin).
4. The system shall allow Teachers to create, configure, and manage courses and group project assignments.
5. The system shall allow Students to create new project groups and generate a unique invitation code for team members.
6. The system shall allow Students to join an existing group by entering a valid invitation code.
7. The system shall allow Students to connect their GitHub repository to their group for automated activity tracking.
8. The system shall ensure that each student can belong to only one group per project assignment.
9. The system shall automatically fetch commit history, repository activity, and push frequency using the GitHub REST API.
10. The system shall allow Students to submit anonymous peer reviews for each of their team members based on communication, reliability, and task participation.
11. The system shall ensure that peer review ratings and comments remain anonymous to other students but visible to Teachers.
12. The system shall allow Teachers to define and adjust weightage percentages for contribution metrics (GitHub commits, attendance records, peer reviews, logged working hours).
13. The system shall automatically calculate an individual contribution score (on a scale of 0 to 100) for each student based on the defined weightage and collected data.
14. The system shall provide Students with a visual dashboard showing their personal contribution score and a detailed breakdown of how it was calculated.
15. The system shall automatically flag students whose contribution score falls below a teacher-defined threshold and notify the Teacher.
16. The system shall provide Teachers with visual analytics dashboards, including contribution distribution charts, group comparisons, and low performer lists.
17. The system shall allow Teachers to generate and download contribution reports in PDF and Excel formats for final grading.

18. The system shall allow Administrators to view, add, edit, and delete user accounts (Students and Teachers) and monitor platform activities.
 19. The system shall maintain detailed audit logs of user activities, administrative actions, and system events to ensure accountability and support forensic analysis.
 20. The system shall support automated backup and recovery mechanisms to prevent data loss in case of system failure.
-

2.5 Asking the First Questions

The initial phase of stakeholder interaction involved asking a structured set of context-free questions to clarify project objectives, identify stakeholder expectations, and define system boundaries. These questions were organized into three categories:

Category 1: Stakeholder and Goal Identification

- Who are the primary users of the GroupSync System?
- What problems does the system aim to solve?
- What are the expected benefits for students, teachers, and administrators?
- Are there any existing systems or manual processes currently being used?

Category 2: Scope and Feasibility

- What are the must-have features versus nice-to-have features?
- What are the technical constraints (e.g., hosting, API rate limits, browser compatibility)?
- Is the project feasible within the given timeline (4 weeks) and team size (6 members)?
- What third-party integrations (e.g., GitHub API) are required?

Category 3: Requirement Clarity and Communication

- Are the requirements clearly understood by all stakeholders?
- Are there any ambiguous or conflicting requirements that need further discussion?
- How will changes to requirements be managed during development?
- How will the development team communicate progress back to stakeholders?

The answers to these questions helped the development team:

- Confirm that the core problem (unfair grading in group projects) was valid and widely recognized.

- Prioritize features such as GitHub integration, contribution scoring, peer review submission, and report generation.
- Identify potential risks, including GitHub API rate limits, biased peer reviews, and hosting limitations.
- Establish a shared understanding of system scope, preventing scope creep during later development phases.

By completing the inception phase with these structured questions and stakeholder discussions, the GroupSync project team established a solid foundation for requirements elicitation, analysis, and subsequent modeling activities.

3. Elicitation

3.1 Introduction

Requirements elicitation is a critical and systematic phase in the software development life cycle. Its primary purpose is to discover, understand, define, and document the needs and expectations of stakeholders regarding the proposed software system. Unlike the inception phase, which focuses on initial problem understanding and high-level scoping, the elicitation phase involves deeper investigation, structured collaboration, and formal documentation of functional and non-functional requirements.

During the elicitation phase, several challenges are commonly encountered, including:

- Unclear or evolving project scope
- Conflicting stakeholder expectations
- Difficulty articulating needs by non-technical stakeholders
- Gaps in understanding between stakeholders and developers
- Requirement volatility due to changing academic or technical environments

To overcome these challenges in the **GroupSync – Intelligent Group Project Management with Contribution Analytics** project, the elicitation process was carried out in a structured, collaborative, and iterative manner. The development team emphasized close interaction with students, teachers, and administrative representatives to ensure that both functional requirements (what the system must do) and non-functional requirements (how well the system must perform) were accurately captured, validated, and prioritized.

By following an organized elicitation approach, the project team aimed to:

- Reduce ambiguity and misinterpretation of requirements
- Manage requirement changes systematically

- Build a clear, validated foundation for system design and implementation
- Ensure that the final system addresses real academic project management needs

The elicitation phase for GroupSync was organized around the following four core activities:

1. Collaborative Requirements Gathering
2. Quality Function Deployment (QFD)
3. Usage Scenarios
4. Elicitation Work Products

3.2 Eliciting Requirements

Unlike the inception phase, which primarily relies on a question-and-answer approach to establish initial scope, the elicitation phase for GroupSync combined multiple complementary techniques to ensure comprehensive requirement discovery. These techniques were applied iteratively, allowing the team to refine and elaborate requirements as understanding deepened.

The following activities were performed during the elicitation phase:

Activity	Purpose	Methods Used
Collaborative Requirements Gathering	To collect raw requirements through direct stakeholder interaction	Focus group discussions, interviews, surveys
Quality Function Deployment (QFD)	To categorize requirements into normal, expected, and exciting levels	Prioritization matrices, stakeholder voting
Usage Scenarios	To understand real-world system interactions and edge cases	Scenario writing, walkthroughs, exception analysis
Elicitation Work Products	To formally document and validate elicited requirements	Requirement specification tables, scope statements, environment descriptions

Each of these activities is described in detail in the following subsections.

3.3 Collaborative Requirements Gathering

Collaborative requirements gathering was conducted to clearly understand the needs, expectations, and pain points of all stakeholders involved in the GroupSync System. The development team recognized that effective requirements cannot be developed in isolation;

instead, they emerge through structured dialogue and shared understanding between developers and stakeholders.

Participants Involved:

Stakeholder Group	Number of Participants	Role in Elicitation
Students	12 (from different departments and semesters)	Provided feedback on existing group project challenges, desired features, and usability expectations
Teachers	4 (from CSE and other departments)	Shared experiences with unfair grading, manual evaluation difficulties, and reporting needs
System Administrator (simulated)	2 (team members acting in admin role)	Identified security, user management, and audit requirements
University Administration Representative	1 (academic advisor)	Provided input on institutional policies, data privacy, and grading compliance

Elicitation Methods Used:

1. **Semi-Structured Interviews:** Individual interviews were conducted with two teachers and four students to explore their experiences with group project evaluation in depth. Each interview lasted approximately 30-45 minutes.
2. **Focus Group Discussion:** A 90-minute focus group session was held with 8 students and 2 teachers to discuss common challenges, brainstorm feature ideas, and identify

conflicting expectations. The discussion was moderated by the team leader, and notes were recorded by the documentation team.

3. **Online Survey:** A structured questionnaire was distributed to 45 students across multiple semesters to collect quantitative data on:

- Frequency of unfair grading experiences
- Willingness to submit anonymous peer reviews
- Preferred features for a contribution tracking system
- Technical familiarity (GitHub, version control, etc.)

Key Findings from Collaborative Gathering:

Identified Issue	Stakeholder Source	Implication for GroupSync
"Some team members do almost nothing, yet get the same grade as me."	Students (multiple)	Contribution score must differentiate individual efforts
"I have no way to track who did what after the project is submitted."	Teachers	Real-time dashboards and activity logs are essential
"I would review my teammates honestly if it was anonymous."	Students	Anonymous peer review mechanism is critical
"Manually collecting attendance and GitHub links is time-consuming."	Teachers	Automated GitHub integration and attendance tracking required
"I don't know how my final grade was calculated from group work."	Students	Transparent score breakdown must be visible to each student
"The system must comply with university grading policies."	Administration	Configurable weightage and audit trails are mandatory

Based on the feedback and discussions, the project team analyzed the collected information and finalized a comprehensive, prioritized list of system requirements for GroupSync. These requirements were then categorized using Quality Function Deployment (QFD), as described in the following section.

3.4 Quality Function Deployment (QFD)

Quality Function Deployment (QFD) is a systematic technique used to translate stakeholder needs, expectations, and desires into structured, actionable system requirements. It helps development teams understand which features are mandatory (normal), which are expected but not explicitly stated (expected), and which are innovative and delightful (exciting). By categorizing requirements, the team can balance functionality, usability, and innovation within project constraints.

For the GroupSync System, requirements were categorized into three distinct levels:

Category	Definition	Priority
Normal Requirements	Basic, explicitly stated features that the system must have to be acceptable. Absence causes dissatisfaction.	High (Mandatory)
Expected Requirements	Implicit features that users assume will be present. Not always stated but sorely missed if absent.	High (Implicit)
Exciting Requirements	Innovative, unexpected features that delight users. Their absence does not cause dissatisfaction.	Medium (If feasible)

3.4.1 Normal Requirements

Normal requirements represent the core, non-negotiable functionalities of the GroupSync System. These requirements were explicitly requested by stakeholders during interviews and focus group sessions. Without these features, the system would be considered incomplete or unacceptable.

Authentication and Access Control:

1. The system shall allow Students to register using their university email address.
2. The system shall allow Teachers and Admins to register using institutional email credentials.
3. The system shall provide secure login and logout functionality for all user roles.
4. The system shall allow users to change their passwords after successful authentication.
5. The system shall enforce role-based access control (Student, Teacher, Admin) throughout all interfaces.

User Management (Admin):

6. The system shall allow Admin to view, add, edit, and delete student accounts.
7. The system shall allow Admin to view, add, edit, and delete teacher accounts.

8. The system shall allow Admin to assign and modify user roles as needed.

Course and Group Management:

9. The system shall allow Teachers to create new courses with course code, title, and semester information.
10. The system shall allow Teachers to enroll students manually or via self-enrollment using a course code.
11. The system shall allow Students to create a new project group and receive a unique invitation code.
12. The system shall allow Students to join an existing group by entering a valid invitation code.
13. The system shall prevent a student from belonging to more than one group per project assignment.

GitHub Integration:

14. The system shall allow Students to connect their personal GitHub account to the platform.
15. The system shall allow Students to link a specific repository to their project group.
16. The system shall automatically fetch commit history, repository activity, and push frequency using the GitHub REST API.
17. The system shall prevent the same GitHub repository from being linked to multiple groups.

Contribution Tracking and Scoring:

18. The system shall allow Teachers to define weightage percentages for contribution metrics (commits, attendance, peer reviews, working hours).
19. The system shall validate that total weightage equals 100% before saving.
20. The system shall automatically calculate an individual contribution score (0-100) for each student based on the defined weightage and collected data.
21. The system shall provide Students with a visual dashboard showing their personal contribution score and detailed breakdown.

Peer Review System:

22. The system shall allow Students to submit anonymous peer reviews for each of their team members.

23. The system shall require ratings for communication, reliability, and task participation.
24. The system shall allow optional text comments to accompany ratings.
25. The system shall prevent a student from submitting more than one peer review per team member per project.

Reporting and Analytics:

26. The system shall allow Teachers to generate downloadable contribution reports in PDF and Excel formats.
27. The system shall provide Teachers with visual analytics dashboards (charts and graphs) showing contribution distribution.
28. The system shall automatically flag students whose contribution score falls below a teacher-defined threshold.

System Administration:

29. The system shall maintain audit logs of user activities, administrative actions, and system events.
30. The system shall support automated backup and recovery mechanisms.
31. The system shall be accessible through a responsive, web-based interface.
32. The system shall operate on a centralized server infrastructure.

3.4.2 Expected Requirements

Expected requirements are features that stakeholders implicitly assume will be present in a well-designed system. These requirements are often not stated explicitly during interviews because users consider them standard or obvious. However, their absence would cause significant dissatisfaction and make the system feel incomplete or unprofessional.

1. **Data Persistence and Integrity:** The system shall maintain a centralized, secure, and consistent database for storing all user profiles, courses, groups, contribution data, peer reviews, and system logs.
2. **Automatic Role-Based UI Adaptation:** The system shall automatically adjust menus, dashboards, and available actions based on the logged-in user's role (Student, Teacher, Admin) without requiring manual configuration.
3. **Secure Credential Storage:** The system shall encrypt all user passwords using a strong hashing algorithm (e.g., bcrypt) and shall never store plain-text passwords.
4. **Duplicate Prevention:** The system shall ensure that a student can cast only one peer review per team member and can belong to only one group per project.

5. **Data Separation for Anonymity:** The system shall separate identifying information (student name, email) from peer review ratings and comments to ensure true anonymity from other students.
6. **Deadline Enforcement:** The system shall automatically enforce peer review deadlines by disabling submission functionality after the teacher-defined cutoff date.
7. **Anti-Tampering Measures:** The system shall prevent modification or deletion of submitted peer reviews and contribution data after the deadline has passed.
8. **Automated Score Calculation:** The system shall automatically recalculate contribution scores whenever new data is available (e.g., new commits, submitted peer reviews).
9. **Threshold-Based Alerts:** The system shall automatically notify Teachers when a student's contribution score drops below the defined threshold without requiring manual checking.
10. **Concurrent User Support:** The system shall maintain data consistency and stable performance when multiple users (e.g., 50+ students) access the system simultaneously during peak submission periods.
11. **Input Validation and Sanitization:** The system shall validate all user inputs (forms, API calls) and sanitize data to prevent SQL injection, cross-site scripting (XSS), and other common security vulnerabilities.
12. **Consistent User Interface:** The system shall provide a consistent look, feel, and navigation structure across all user roles and device sizes (desktop, tablet, mobile).
13. **Session Management:** The system shall automatically terminate inactive user sessions after a predefined timeout period (e.g., 30 minutes) and require re-authentication for sensitive operations.
14. **Error Handling and User Feedback:** The system shall display clear, user-friendly error messages (e.g., "Invalid email or password") without exposing technical details or stack traces.
15. **Data Export Capability:** The system shall allow Teachers to export raw contribution data (CSV format) for external analysis or record-keeping.

3.4.3 Exciting Requirements

Exciting requirements are innovative features that go beyond stakeholder expectations. These features are not requested during initial discussions because users may not realize they are possible. When present, exciting requirements generate delight and positive surprise. However, their absence does not cause dissatisfaction, as users did not expect them in the first place.

For the GroupSync System, the following exciting requirements were identified as potential differentiators that could significantly enhance user experience and system value:

1. **Real-Time Contribution Score Gauge:** Display a dynamic, color-coded gauge (e.g., 0-100 with green/yellow/red zones) on the Student dashboard that updates in real-time as new commits are pushed or peer reviews are submitted.
2. **Contribution Trend Over Time:** Provide Students with a line chart showing how their contribution score has evolved over the duration of the project (e.g., Week 1, Week 2, Week 3), helping them visualize improvement or decline.
3. **Automated Peer Review Reminders:** Send automated email or in-app notifications to Students who have not yet submitted their peer reviews, with configurable reminders (e.g., 3 days before deadline, 1 day before deadline).
4. **Teacher Preview Mode for Reports:** Allow Teachers to preview how a report will look (PDF/Excel) before finalizing and sharing with students or administration, including the ability to add comments or adjustments.
5. **Group Activity Heatmap:** Display a visual heatmap (calendar-style) showing daily commit activity, peer review submissions, and logged hours for each group, helping Teachers quickly identify active and inactive periods.
6. **Comparison View for Teachers:** Allow Teachers to compare two or more groups side-by-side in the analytics dashboard, showing average contribution scores, commit frequency, and peer review completion rates.
7. **GitHub Commit Quality Analysis:** Go beyond commit count by analyzing commit message quality, consistency, and distribution across team members (e.g., did one student make 100 commits in one day vs. 10 commits over 10 days).
8. **Student Leaderboard (Anonymous):** Display an anonymized leaderboard showing contribution score percentiles, allowing Students to see how they rank compared to peers without revealing specific identities.
9. **Automated Low-Contributor Warning to Student:** When a student's contribution score drops below the threshold, automatically send a private, constructive notification to the student (e.g., "Your contribution score has dropped below 40%. Consider increasing activity or communicating with your team.").
10. **Exportable Audit Logs for Administration:** Allow Administrators to export audit logs in structured formats (JSON, CSV) for external compliance review or long-term archival.

Prioritization Note: Due to the 4-week development timeline and team size of six members, exciting requirements were prioritized for implementation only after all normal and expected requirements were completed. Selected high-value exciting features (such as the real-time score

gauge and automated reminders) were planned for the final sprint if time permitted. Remaining exciting requirements were documented for future enhancement phases.

3.5 Usage Scenarios

Usage scenarios (also known as use case scenarios or narrative descriptions) are detailed, step-by-step descriptions of how specific users interact with a system to accomplish a goal. These scenarios cover both normal (expected) and exceptional (error, boundary, or malicious) situations. Developing usage scenarios helps the development team:

- Validate requirements against real-world situations
- Identify missing requirements or edge cases
- Understand system behavior under error conditions
- Design appropriate error messages and recovery mechanisms

The following usage scenarios were developed for the GroupSync System based on stakeholder interviews, focus group discussions, and risk analysis.

3.5.1 Scenario 1: Invalid Login Attempt

Description: A user attempts to access the system using incorrect credentials.

Actors: Student, Teacher, or Admin

Normal Flow:

1. User navigates to the GroupSync login page.
2. User enters an incorrect username/email and/or password.
3. User submits the login form.
4. System validates the credentials against the database.
5. System rejects the authentication request.
6. System displays an error message: "Invalid email or password. Please try again."
7. User remains on the login page.

Exceptional Flow (Multiple Failures):

1. User attempts to log in with incorrect credentials five times consecutively.
2. System detects repeated failed attempts from the same IP address or user account.
3. System temporarily blocks the user account for 15 minutes.

4. System displays a message: "Too many failed attempts. Your account has been temporarily locked. Please try again after 15 minutes or contact support."
5. System records the failed attempts and lockout event in the audit log for security auditing.

Post-Condition: User is not granted access to the system. All failed attempts are logged.

3.5.2 Scenario 2: Unauthorized Access Attempt

Description: A logged-in user attempts to access a function or page that is outside their assigned role's permissions.

Actors: Student, Teacher, or Admin

Normal Flow:

1. A Student is logged into the GroupSync System.
2. The Student manually types the URL for the Teacher's Analytics Dashboard (e.g., /teacher/analytics) into the browser address bar.
3. Student presses Enter to navigate.
4. System receives the request and extracts the user's role from the active session.
5. System verifies that the requested resource requires the "Teacher" role.
6. System detects a role mismatch (current role is "Student").
7. System denies access to the requested page.
8. System displays a warning message: "Access Denied. You do not have permission to view this page."
9. System logs the unauthorized access attempt in the audit log, including user ID, timestamp, requested URL, and source IP address.
10. System redirects the Student back to their appropriate dashboard.

Post-Condition: Unauthorized access is prevented and recorded. User remains within their authorized boundaries.

3.5.3 Scenario 3: Late Peer Review Submission

Description: A student attempts to submit a peer review after the teacher-defined deadline has passed.

Actors: Student, System

Normal Flow:

1. Teacher creates a project assignment and sets the peer review deadline to May 25, 2026, at 11:59 PM.
2. Current system date and time is May 26, 2026, at 10:00 AM (deadline has passed).
3. Student logs into GroupSync and navigates to the Peer Review section.
4. Student selects a team member and fills out the rating and comment fields.
5. Student clicks the "Submit Review" button.
6. System checks the current server time against the assignment's peer review deadline.
7. System determines that the current time is after the deadline.
8. System rejects the submission.
9. System displays a message: "Peer review deadline has passed (May 25, 2026, 11:59 PM). Submissions are no longer accepted."
10. System does not save the review.
11. System logs the attempted late submission in the audit log.

Alternative Flow (Grace Period if configured):

- If Teacher has configured a grace period (e.g., 24 hours), System may allow late submission but mark it as "Late" in the report.

Post-Condition: Peer review is not submitted. Student is informed of deadline expiration.

3.5.4 Scenario 4: Duplicate GitHub Repository Connection

Description: A student attempts to connect a GitHub repository that is already linked to another group in the same course and project assignment.

Actors: Student, System

Normal Flow:

1. Student A from Group Alpha connects repository "project-x" to their group.
2. System successfully links the repository and records its unique repository ID.
3. Student B from Group Beta later attempts to connect the same repository "project-x" to their group.

4. Student B authorizes GitHub access and selects the repository.
5. System checks the repository's unique ID against existing connections for the same course and assignment.
6. System detects that the repository is already linked to Group Alpha.
7. System rejects the connection request.
8. System displays an error message: "This repository is already connected to another group in this course. Each repository can only be linked to one group."
9. System logs the duplicate connection attempt in the audit log.

Post-Condition: Repository remains linked only to the original group. Second connection is rejected.

3.5.5 Scenario 5: System Failure During Active Submission

Description: A temporary system failure (e.g., database connection loss, server timeout) occurs while multiple students are actively submitting peer reviews or pushing commits.

Actors: Students, System, System Operator

Normal Flow:

1. Students are actively submitting peer reviews during peak hours (e.g., 1 hour before deadline).
2. A temporary database connection failure occurs due to high load.
3. System detects the failure (e.g., database query timeout).
4. System suspends ongoing write operations (submissions, updates) to prevent data corruption.
5. System displays a user-friendly message to affected students: "System is experiencing high load. Your submission has been saved temporarily. Please try again in a few minutes."
6. Background recovery mechanisms are activated automatically.
7. Database connection is restored after 2 minutes.
8. System verifies data integrity and consistency.
9. System restores normal operation.
10. Students can resume and complete their submissions.

Post-Condition: No submission data is lost, duplicated, or corrupted. System returns to full operation. Audit log records the failure and recovery events.

3.5.6 Scenario 6: Invalid Group Configuration

Description: A Teacher attempts to create or update a project assignment configuration with incomplete or invalid parameters.

Actors: Teacher, System

Normal Flow:

1. Teacher creates a new project assignment titled "Software Engineering Group Project."
2. Teacher sets weightage as follows: GitHub Commits = 40%, Attendance = 30%, Peer Reviews = 20% (Total = 90%).
3. Teacher clicks "Save Configuration."
4. System validates that total weightage equals 100%.
5. System detects that total weightage is 90% (less than 100%).
6. System rejects the configuration.
7. System displays specific error messages:
 - "Total weightage must equal 100%. Current total: 90%."
 - "Please adjust the values to reach 100%."
8. System does not save the invalid configuration.
9. Teacher adjusts weightage: GitHub Commits = 40%, Attendance = 30%, Peer Reviews = 20%, Working Hours = 10% (Total = 100%).
10. Teacher clicks "Save Configuration" again.
11. System validates and accepts the configuration.
12. System displays a success message: "Configuration saved successfully."

Post-Condition: Only valid, complete configurations are saved. Clear error messages guide correction.

3.5.7 Scenario 7: Malicious Activity Detection

Description: The system detects unusual or potentially malicious behavior, such as rapid repeated API calls, brute-force login attempts, or unusual data access patterns.

Actors: System, Attacker (external), System Administrator

Normal Flow:

1. System continuously monitors request patterns, login attempts, and API usage.
2. The system detects 100+ API calls to the GitHub integration endpoint from a single user within 60 seconds (well above normal usage).
3. System flags the activity as "suspicious" and classifies it as a potential denial-of-service (DoS) or abuse attempt.
4. System automatically restricts access for the involved user account for 30 minutes.
5. System logs detailed activity records, including user ID, timestamps, endpoints accessed, request frequency, and source IP address.
6. System sends an automated alert notification to the System Administrator via email or dashboard.
7. Admin reviews the logs and determines appropriate action (e.g., permanent ban, password reset, contact user).
8. System continues to monitor for similar patterns from other users.

Post-Condition: Potential malicious activity is contained. Audit trail is available for investigation. Admin is notified.

3.6 Elicitation Work Product

The output of the requirements elicitation process varies depending on the size, complexity, and nature of the system being developed. For the GroupSync System, the elicitation work products were carefully prepared to ensure clarity, completeness, traceability, and alignment with stakeholder expectations.

The following elicitation work products were delivered for the GroupSync project:

Work Product	Description	Purpose
Structured Requirements Statement	A clear, organized document listing all functional and non-functional requirements derived from stakeholder discussions. Includes normal,	Serves as the foundation for design, implementation, and testing.

	expected, and exciting categories.	
Project Scope Statement	A well-defined description of system boundaries, including features that are included (e.g., GitHub integration, peer reviews, contribution scoring) and explicitly excluded (e.g., real-time chat, plagiarism detection).	Prevents scope creep and manages stakeholder expectations.
Stakeholder Register	A comprehensive list of all individuals and groups who participated in elicitation, including Students (12 participants), Teachers (4 participants), Administration (1 participant), and simulated Admin roles.	Ensures traceability and identifies points of contact for future validation.
Detailed Usage Scenarios	A set of 7 documented scenarios covering normal, error, and malicious conditions. Each scenario includes actors, flow, exceptions, and post-conditions.	Validates requirements against real-world situations and identifies edge cases.
Technical Environment Description	A specification of the system's technical environment, including web-based frontend (HTML5, TailwindCSS, React.js), backend (PHP/Laravel, Node.js), database (MySQL), GitHub REST API, deployment platform (InfinityFree), and version control (Git/GitHub).	Guides architectural decisions and technology selection.

Summary of Elicitation Outcomes:

By the conclusion of the elicitation phase, the GroupSync project team had achieved the following:

- Identified and documented **32 normal requirements, 15 expected requirements, and 10 exciting requirements**.
- Validated all requirements with at least two stakeholder groups.

- Resolved initial conflicts (e.g., feature completeness vs. timeline) through prioritization.
- Established a clear technical direction based on stakeholder needs and team capabilities.
- Created a validated foundation for scenario-based modeling, data modeling, class-based modeling, and behavioral modeling in subsequent phases.

The elicitation work products served as the primary input for the remaining sections of this Software Requirements Specification (SRS) document, ensuring that the entire development process remains grounded in authentic stakeholder needs and validated requirements.

4. Scenario-Based Modeling

4.1 Use Case Scenario

Table 1: Use Cases Table

Level - 0	Level - 1	Level-2	Actors
GroupSync	Authentication	Sign Up	Student
		Sign In	Student, Teacher, Admin
		Sign Out	Student, Teacher, Admin
		Change Password	Student, Teacher, Admin
	User Management	Create User	Admin
		Update User Role	Admin
		Delete User	Admin
	Course & Group Management	Create Course	Teacher
		Enroll Students	Teacher
		Create Group	Student

		Join Group	Student
		Connect GitHub Repo	Student
	Contribution Tracking	Set Weightage	Teacher
		Fetch GitHub Activity	System
		View Contribution Score	Student, Teacher
	Peer Review Process	Submit Peer Review	Student
		View Peer Review Status	Student
	Report & Analytics	Generate Report	Teacher
		View Analytics Dashboard	Teacher
		Flag Low Contributors	System, Teacher
	Monitoring & System Admin	View Audit Logs	Admin
		Manage System Settings	Admin

4.2 Use Case Description

4.2.1 Authentication

1. Use Case: Sign Up

- **Primary Actor:** Student
- **Goal:** To create a new account.
- **Preconditions:** System is online. Student not registered.
- **Trigger:** Student wants to register.
- **Scenario:**

1. Student visits Sign Up page.
 2. Student provides registration information (university email).
 3. System validates information.
 4. System creates a new student account.
 5. Confirmation message is shown.
- **Exceptions:** Invalid information. Email already exists.

2. Use Case: Sign In

- **Primary Actors:** Student, Teacher, Admin
- **Goal:** To access the system.
- **Precondition:** User must be registered.
- **Trigger:** User wants to log in.
- **Scenario:**
 1. User enters username and password.
 2. System validates credentials.
 3. System identifies user role.
 4. User is redirected to respective dashboard.
- **Exceptions:** Invalid credentials. Account blocked.

3. Use Case: Sign Out

- **Primary Actors:** Student, Teacher, Admin
- **Goal:** To exit the system.
- **Precondition:** User must be logged in.
- **Trigger:** User wants to log out.
- **Scenario:**
 1. User clicks Sign Out.
 2. System terminates the session.
 3. User is redirected to login page.

4. Use Case: Change Password

- **Primary Actors:** Student, Teacher, Admin
- **Goal:** To update account password.
- **Precondition:** User must be logged in.
- **Trigger:** User wants to change password.
- **Scenario:**
 1. User opens Change Password option.
 2. User enters old and new password.
 3. System validates the new password.
 4. Password is updated successfully.
- **Exceptions:** Weak password. Incorrect current password.

4.2.2 User Management

1. Use Case: Create User

- **Primary Actor:** Admin
- **Goal:** To create a new system user.
- **Precondition:** Admin must be logged in.
- **Trigger:** Admin needs to add a user.
- **Scenario:**
 1. Admin opens User Management.
 2. Admin selects Create User.
 3. Admin enters user details and role (Student/Teacher).
 4. System validates information.
 5. User account is created.
- **Exceptions:** User already exists. Invalid data.

2. Use Case: Update User Role

- **Primary Actor:** Admin
- **Goal:** To modify user role.
- **Precondition:** Admin logged in.

- **Trigger:** Admin wants to change user role.
- **Scenario:**
 1. Admin selects an existing user.
 2. Admin updates the user role.
 3. System saves changes.
 4. Confirmation shown.
- **Exceptions:** User not found.

3. Use Case: Delete User

- **Primary Actor:** Admin
- **Goal:** To remove a user.
- **Precondition:** Admin logged in.
- **Trigger:** Admin wants to delete a user.
- **Scenario:**
 1. Admin selects the user.
 2. Admin confirms deletion.
 3. System removes the user account.
 4. Confirmation shown.
- **Exception:** Network failure.

4.2.3 Course & Group Management

1. Use Case: Create Course

- **Primary Actor:** Teacher
- **Goal:** To create a new course.
- **Precondition:** Teacher logged in.
- **Trigger:** Teacher wants to create a course.
- **Scenario:**
 1. Teacher selects Create Course.
 2. Teacher enters course code, title, semester.

3. System saves course information.

- **Exception:** Network failure.

2. Use Case: Create Group

- **Primary Actor:** Student
- **Goal:** To create a new project group.
- **Precondition:** Student logged in, enrolled in course.
- **Trigger:** Student wants to create a group.
- **Scenario:**
 1. Student selects Create Group.
 2. System generates a unique invitation code.
 3. Group is created.
- **Exception:** Network failure.

3. Use Case: Join Group

- **Primary Actor:** Student
- **Goal:** To join an existing group.
- **Precondition:** Student logged in.
- **Trigger:** Student has an invitation code.
- **Scenario:**
 1. Student selects Join Group.
 2. Student enters invitation code.
 3. System validates code.
 4. Student is added to the group.
- **Exception:** Invalid code. Group full.

4. Use Case: Connect GitHub Repository

- **Primary Actor:** Student
- **Goal:** To link a GitHub repo for activity tracking.
- **Precondition:** Student in a group.

- **Trigger:** Student wants to connect repo.
- **Scenario:**
 1. Student selects Connect GitHub.
 2. Student authorizes GitHub access.
 3. Student selects the repository.
 4. System links the repo to the group.
- **Exception:** Network failure. Invalid repo.

4.2.4 Contribution Tracking

1. Use Case: Set Weightage

- **Primary Actor:** Teacher
- **Goal:** To define evaluation criteria.
- **Precondition:** Teacher logged in, project assignment created.
- **Trigger:** Teacher wants to configure scoring.
- **Scenario:**
 1. Teacher opens Assignment Settings.
 2. Teacher sets percentage for commits, attendance, peer reviews, hours.
 3. System validates total equals 100%.
 4. Weightage is saved.
- **Exception:** Invalid total.

2. Use Case: View Contribution Score

- **Primary Actors:** Student, Teacher
- **Goal:** To view individual contribution score.
- **Precondition:** User logged in.
- **Trigger:** User selects View Score.
- **Scenario:**
 1. User selects a project.
 2. System displays score (0-100) and breakdown.

- **Exception:** Network failure.

4.2.5 Peer Review Process

1. Use Case: Submit Peer Review

- **Primary Actor:** Student
- **Goal:** To review team members anonymously.
- **Precondition:** Student logged in, peer review period active.
- **Trigger:** Student wants to submit review.
- **Scenario:**
 1. Student opens Peer Review section.
 2. System displays team members.
 3. Student provides ratings and comments.
 4. Student submits review.
 5. System confirms submission.
- **Exceptions:** Network failure. Missing ratings.

2. Use Case: View Peer Review Status

- **Primary Actor:** Student
- **Goal:** To check if reviews are submitted.
- **Precondition:** Student logged in.
- **Trigger:** Student wants to check status.
- **Scenario:**
 1. Student opens Peer Review Status.
 2. System shows which reviews are pending/completed.
- **Exception:** Network failure.

4.2.6 Report & Analytics Management

1. Use Case: Generate Report

- **Primary Actor:** Teacher
- **Goal:** To download contribution report.

- **Precondition:** Teacher logged in, project ended.
- **Trigger:** Teacher requests report.
- **Scenario:**
 1. Teacher selects Generate Report.
 2. System compiles contribution data.
 3. System generates PDF/Excel file.
 4. Report is downloaded.
- **Exception:** Network failure.

2. Use Case: View Analytics Dashboard

- **Primary Actor:** Teacher
- **Goal:** To see visual charts of contributions.
- **Precondition:** Teacher logged in.
- **Trigger:** Teacher opens Analytics.
- **Scenario:**
 1. Teacher selects a course/project.
 2. System displays contribution distribution, low performers, group comparisons.
- **Exception:** Network failure.

3. Use Case: Flag Low Contributors

- **Primary Actor:** System / Teacher
- **Goal:** To identify students below threshold.
- **Precondition:** Teacher defined threshold.
- **Trigger:** Contribution score drops below threshold.
- **Scenario:**
 1. System periodically checks scores.
 2. System flags students below threshold.
 3. Teacher is notified.
- **Exception:** N/A.

4.2.7 Monitoring & System Admin

1. Use Case: View Audit Logs

- **Primary Actor:** Admin
- **Goal:** To monitor system activities.
- **Precondition:** Admin logged in.
- **Trigger:** Admin requests logs.
- **Scenario:**
 1. Admin opens Audit Logs.
 2. System displays recent activities.
- **Exception:** Network failure.

4.3 Use Case Diagram

Level 0 for GroupSync System

- This use case summarizes how users (Student, Teacher, Admin) interact with GroupSync.

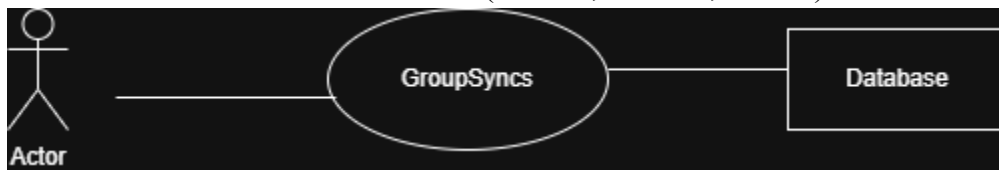


Figure 1: Level 0 for GroupSync

Level 1 for GroupSync System

Highlights administrative interactions: Manage Users, Manage System Settings, View Audit Logs.

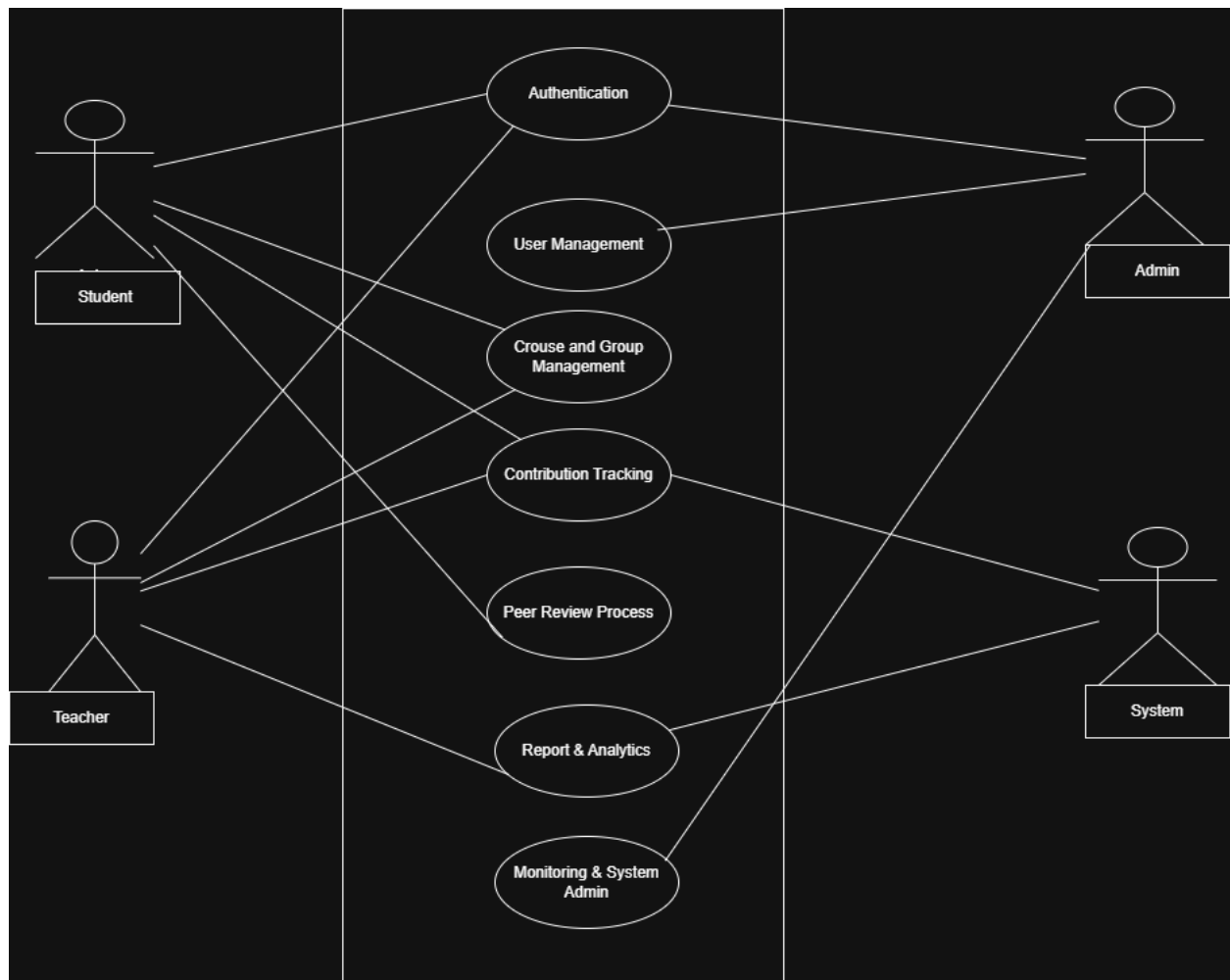


Figure 2: Level 1 for GroupSync

Level 2.1 (Authentication) for GroupSync System

- Defines secure authentication: Sign Up, Sign In, Sign Out, Change Password.

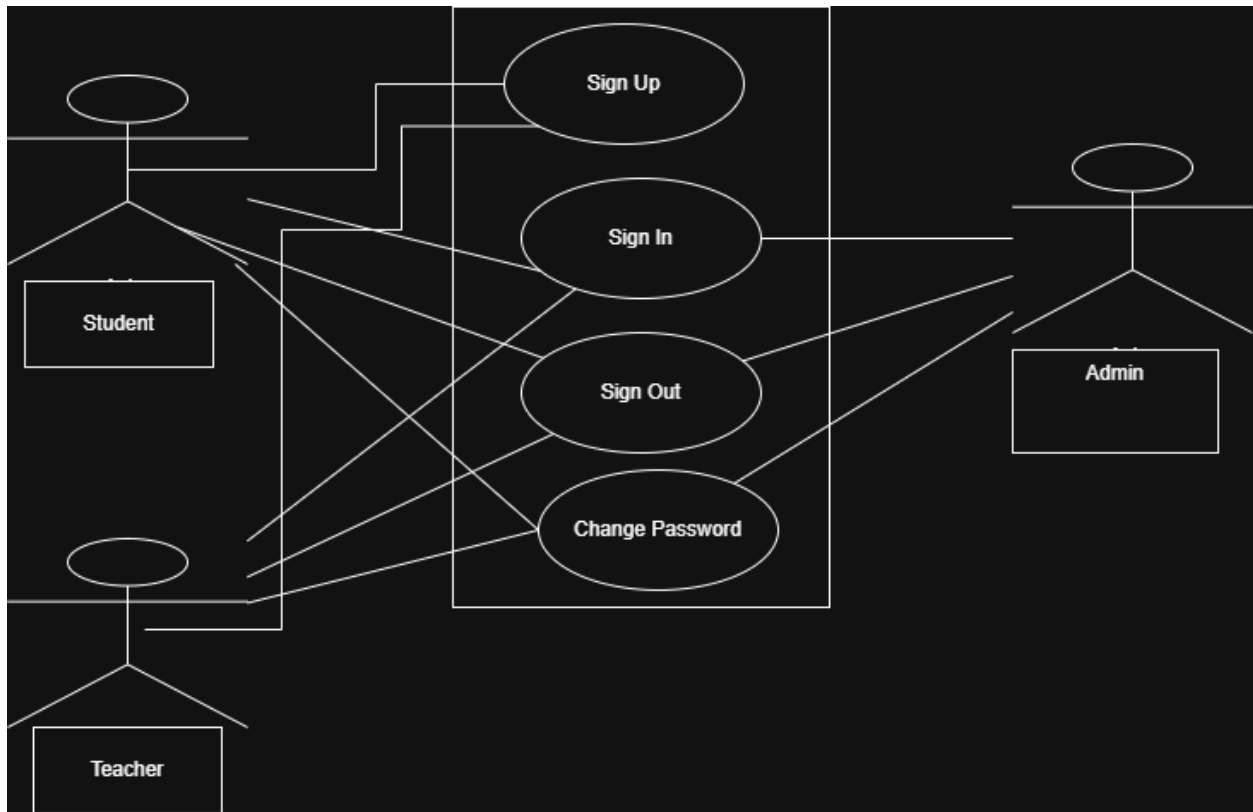


Figure 3: Level 2.1 (Authentication) for GroupSyncs

Level 2.2 (User Management) for GroupSync System

- Represents how Admin creates, updates, and deletes users

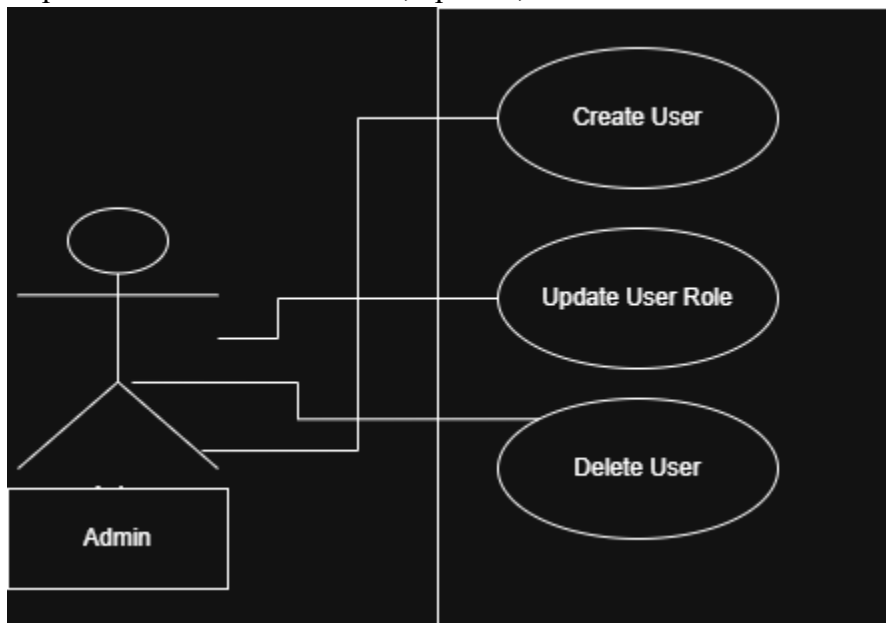


Figure 4: Level 2.2 (User Management) for GroupSync

Level 2.3 (Course & Group Management) for GroupSync System

- Describes how Teachers create courses and Students create/join groups.

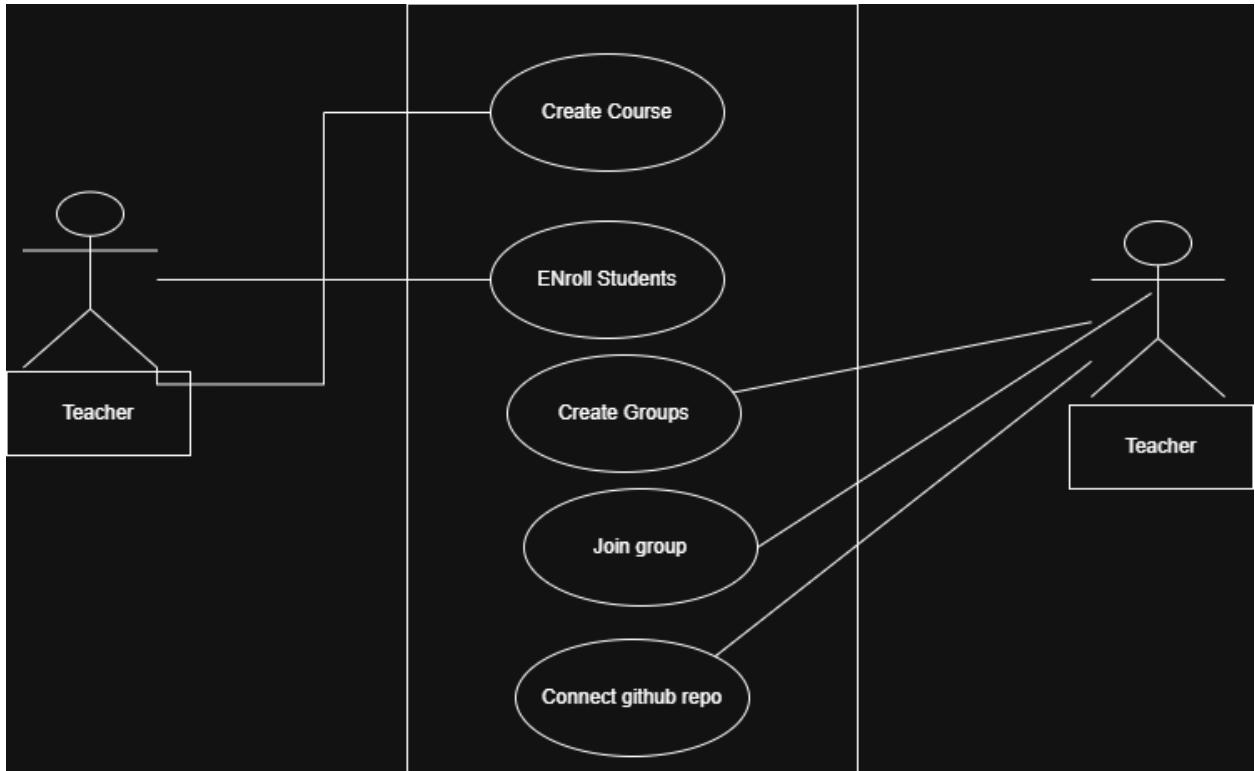


Figure 5: Level 2.3 Course & Group Management for GroupSync

Level 2.4 (Contribution Tracking) for GroupSync System

- Shows Teachers setting weightage and Students viewing their scores.

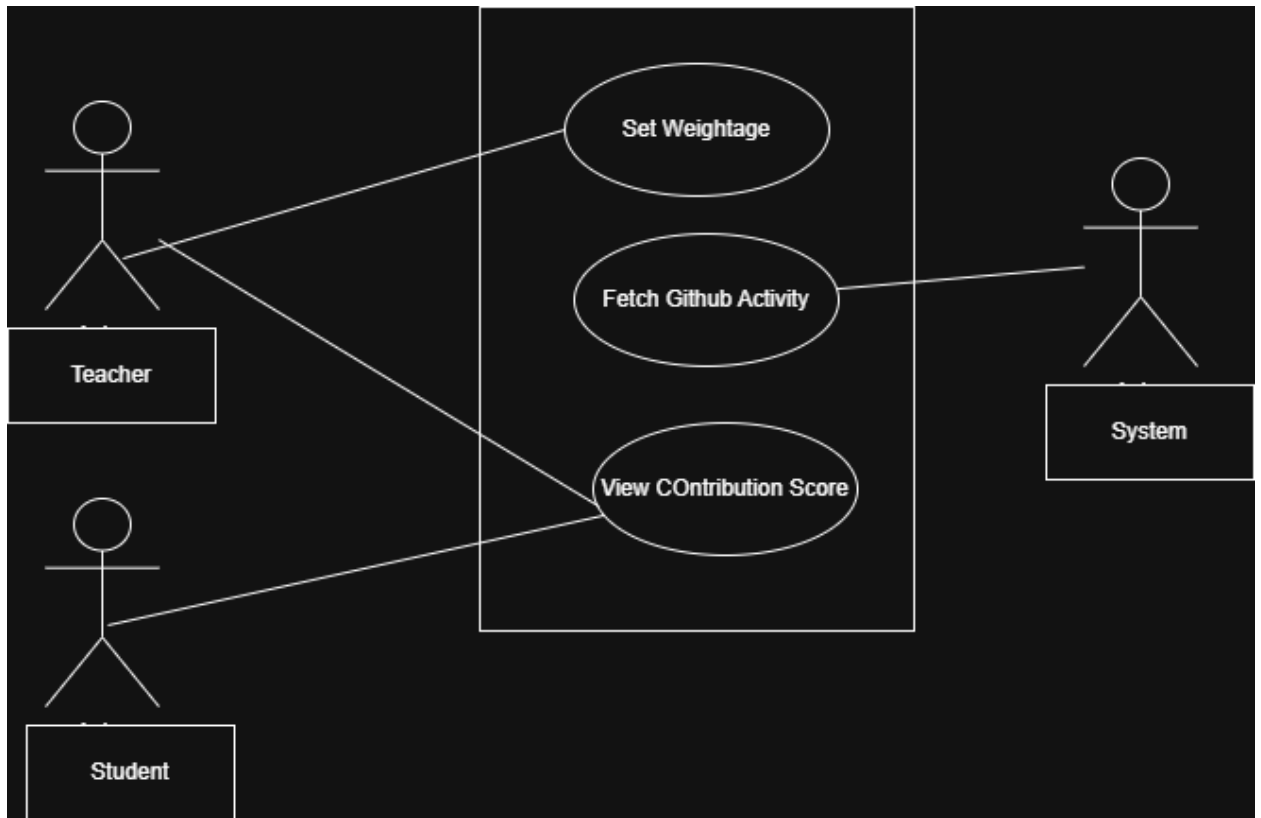


Figure 6: Level 2.3 **Contribution Tracking** for GroupSync

Level 2.5 (Peer Review Process) for GroupSync System

- Illustrates how Students submit anonymous peer reviews.

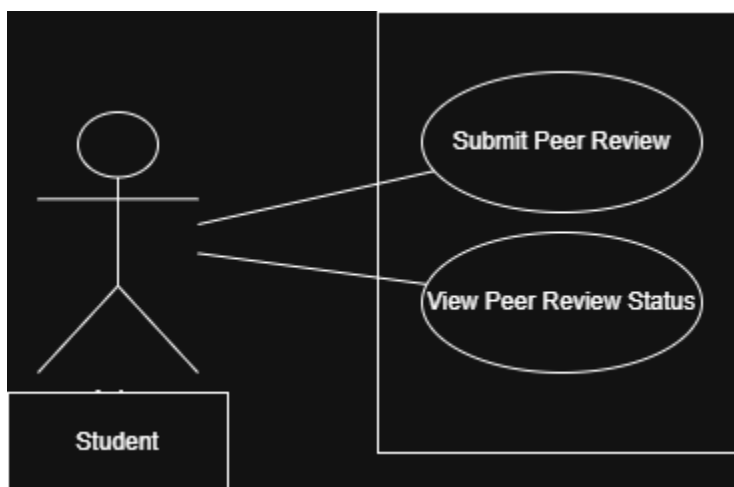


Figure 7: Level 2.3 **Peer Review Process** for GroupSync

Level 2.6 (Report & Analytics) for GroupSync System

- Shows Teachers generating reports and viewing analytics dashboards.

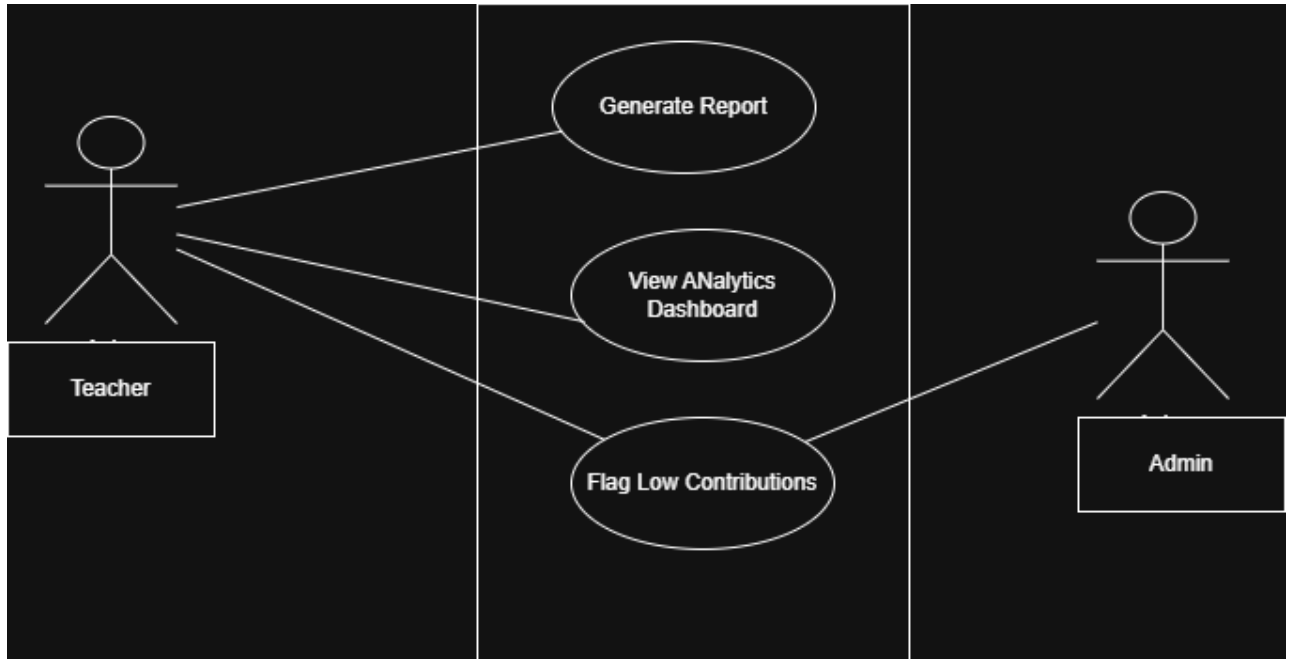


Figure 8: Level 2.3 **Report & Analytics** for GroupSync

Figure 9: Level 2.7 (Monitoring & Admin) for GroupSync System

- Covers Admin monitoring logs and managing system settings.

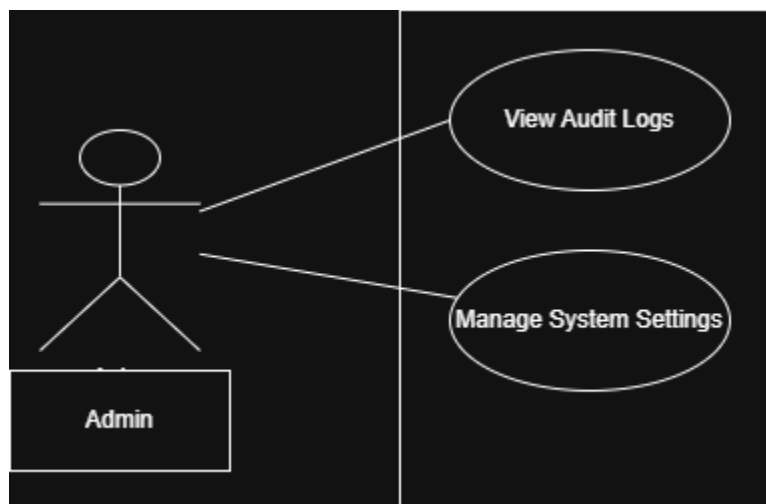


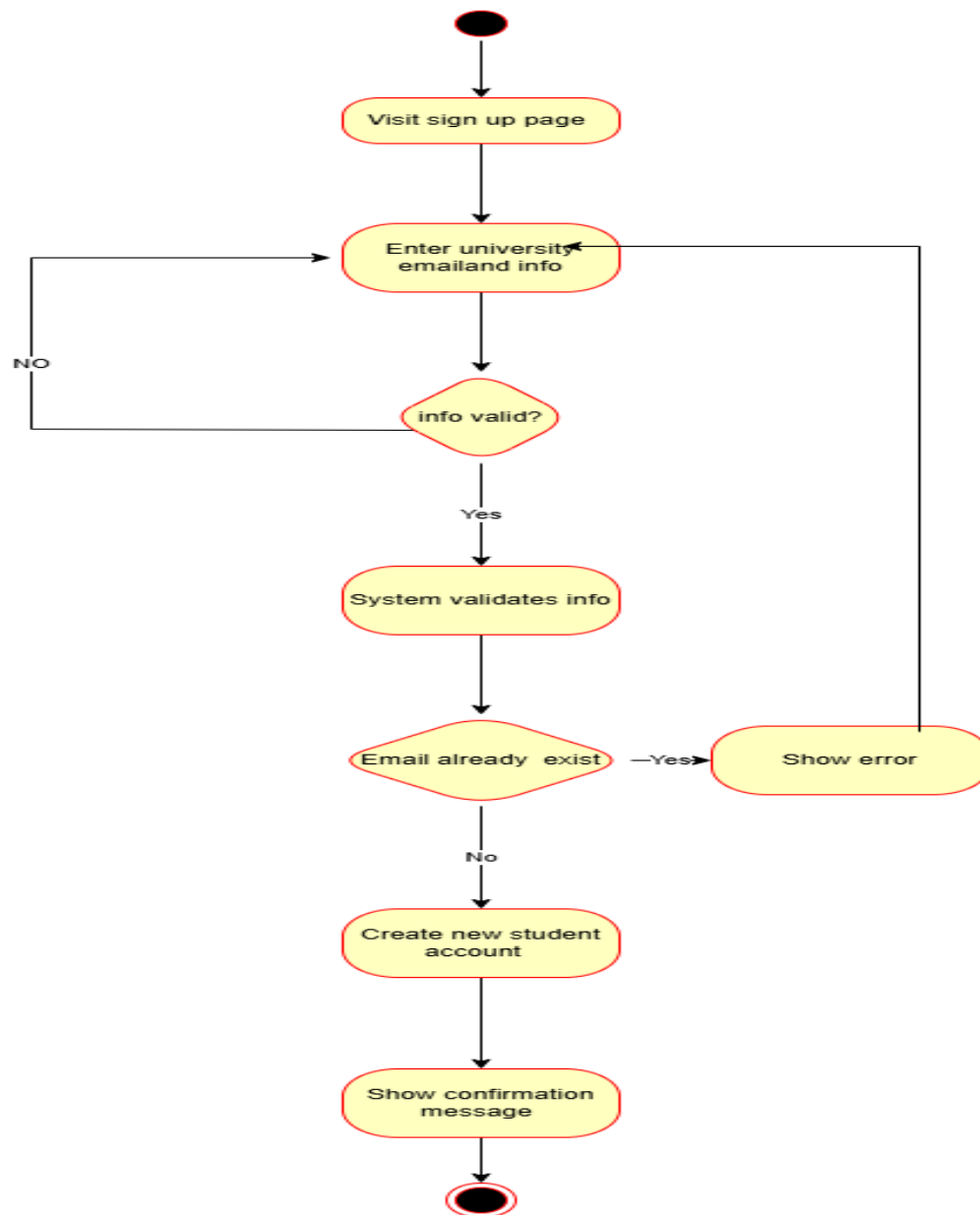
Figure 8: Level 2.3 **Monitoring & Admin** for GroupSync

4.4 Activity Diagram & Swimlane Diagram

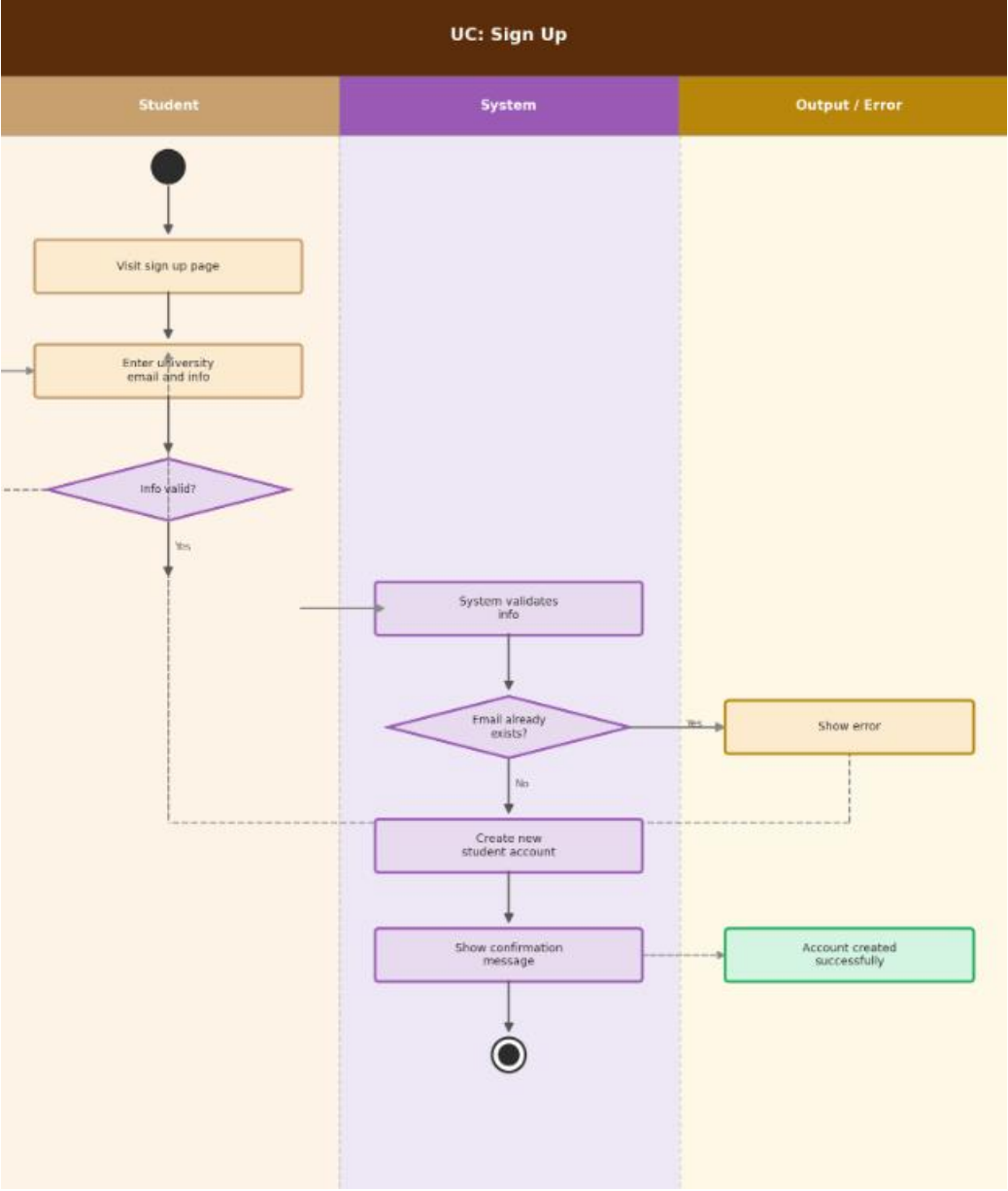
The activity diagram in the GroupSync System presents the step-by-step workflow of the group project management process, including authentication, course and group management, GitHub integration, peer review submission, contribution scoring, and report generation. It helps clearly visualize actions and decision points, making the overall process easier to understand.

The swimlane diagram divides the workflow among different actors such as Student, Teacher, System Administrator, and the System. Each lane shows role-specific responsibilities, improving clarity, accountability, and coordination throughout the project lifecycle.

Activity Diagram:



Swimlane Diagram



Activity Diagram:

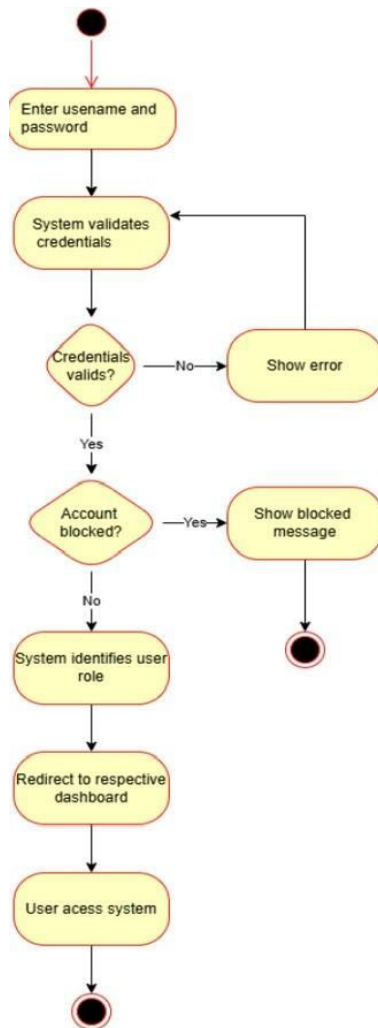


Figure: Sign In - Activity Diagram

This activity diagram depicts the authentication process for GroupSync users including Students, Teachers, and Admins. The user enters username and password. The system verifies credentials against the database. If credentials are valid, the system identifies the user role and redirects to the respective dashboard (Student dashboard, Teacher panel, or Admin panel). If credentials are invalid, an error message is displayed. After multiple consecutive failed attempts, the system temporarily blocks the user account.

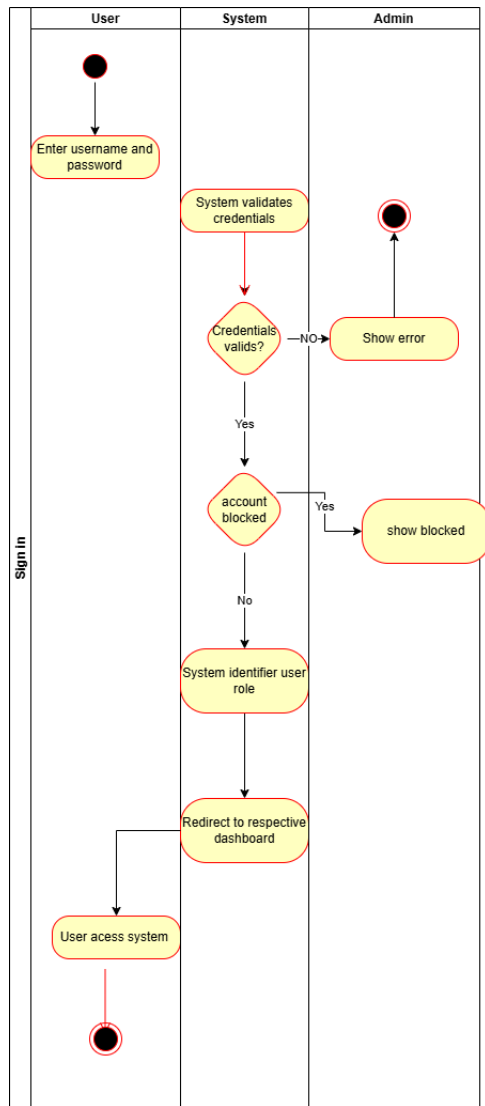


Figure: Sign In - Swimlane Diagram

The swimlane diagram shows coordination between users and the system during login. The User lane shows credential entry and submission. The System lane handles credential validation, role identification, dashboard redirection, and logging of failed attempts for security auditing.

Activity Diagram:



Figure: Sign Out - Activity Diagram

This activity diagram demonstrates the user logout workflow. The logged-in user clicks the Sign Out button. The system terminates the active session, invalidates authentication tokens, and redirects the user to the login page. This ensures secure termination of active sessions.

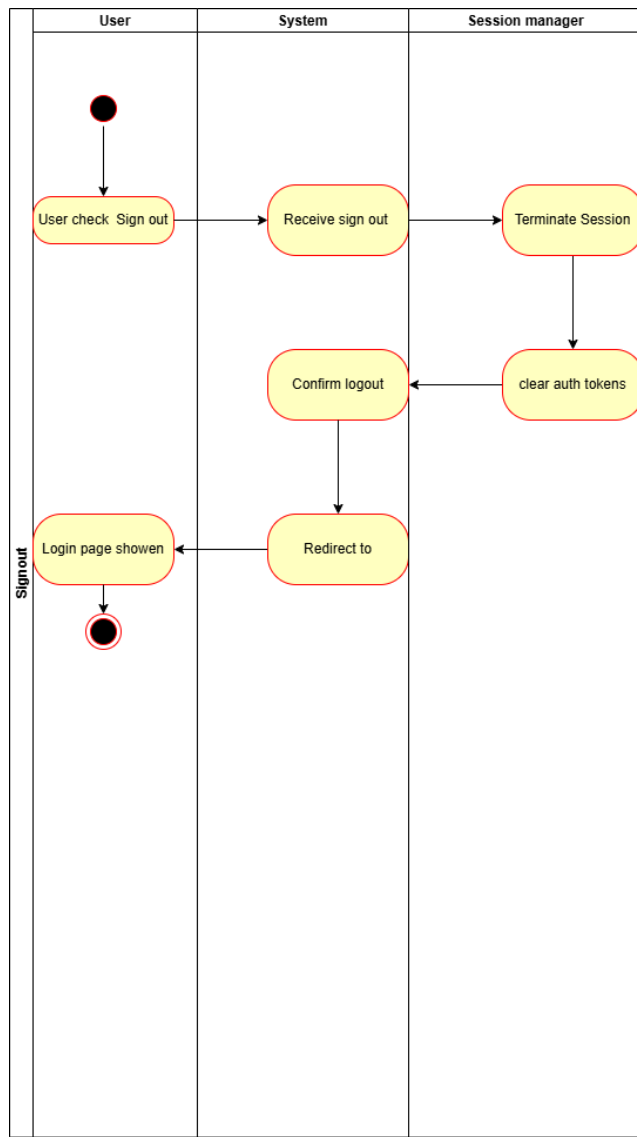


Figure: Sign Out - Swimlane Diagram

The swimlane diagram highlights system and user actions during logout. The User lane shows the logout request. The System lane handles session invalidation and redirection, ensuring no residual session data remains.

Activity Diagram:

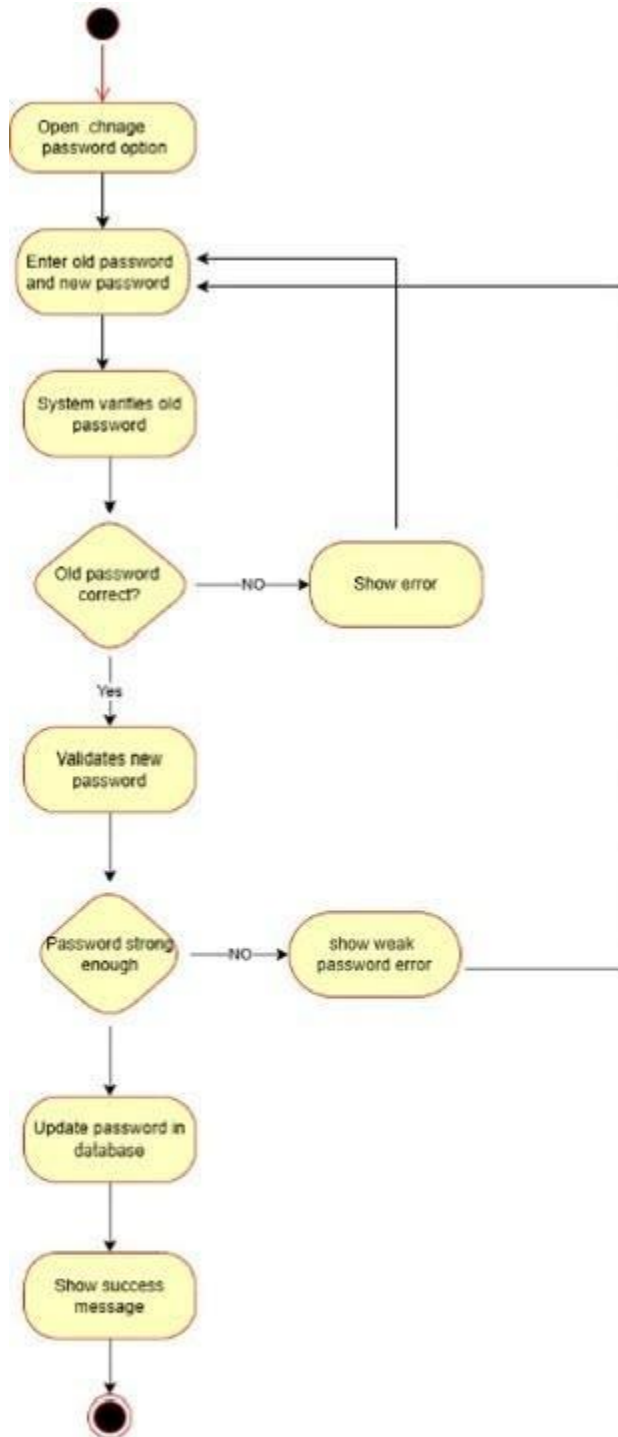


Figure: Change Password - Activity Diagram
This activity diagram outlines the password update procedure. The logged-in user enters old password and new password.

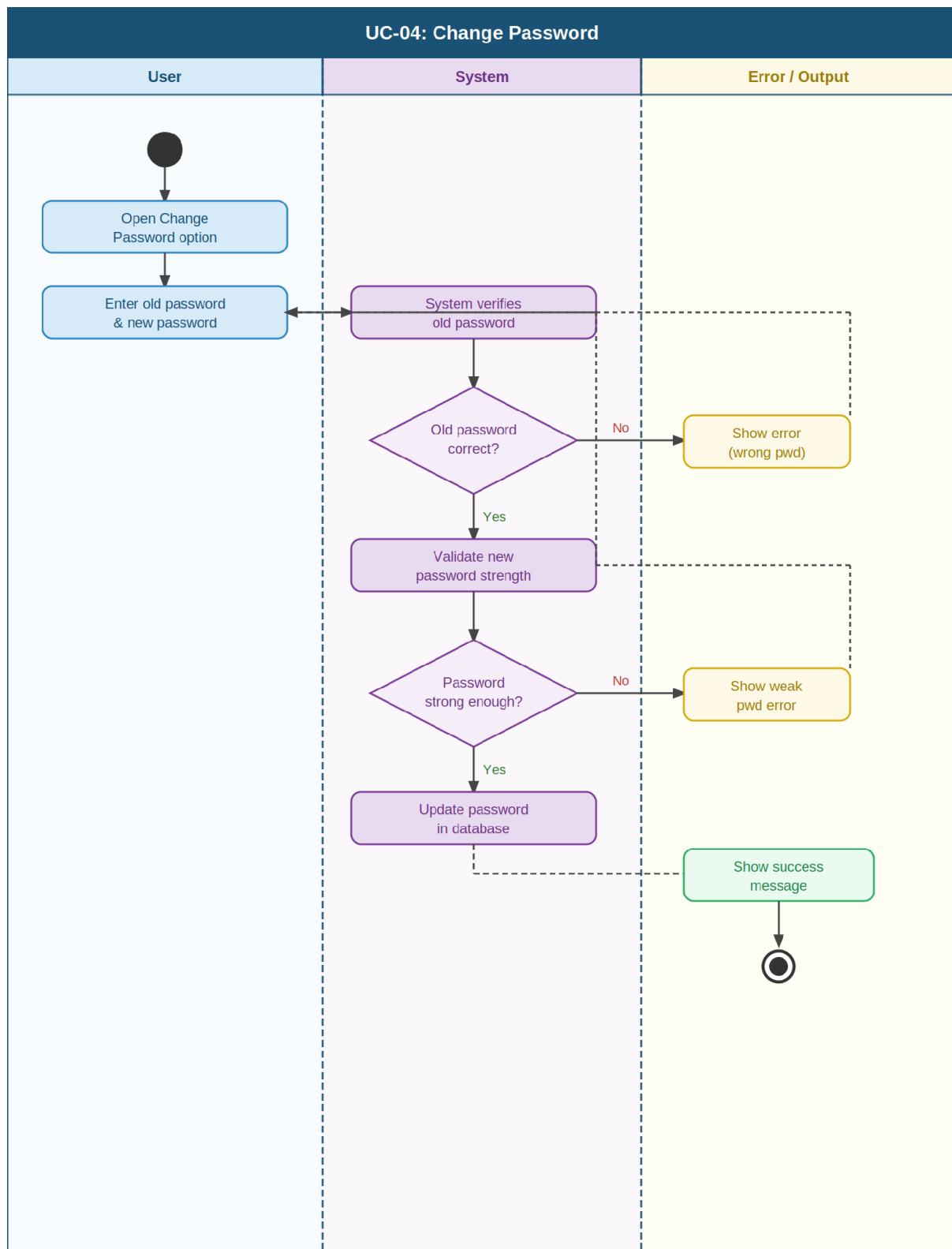


Figure: Change Password - Swimlane Diagram

The swimlane diagram illustrates interaction between users and the system during password change.

Activity Diagram:

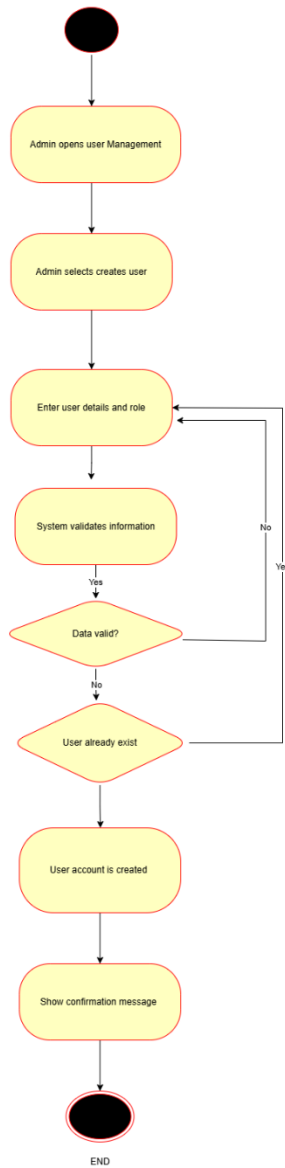
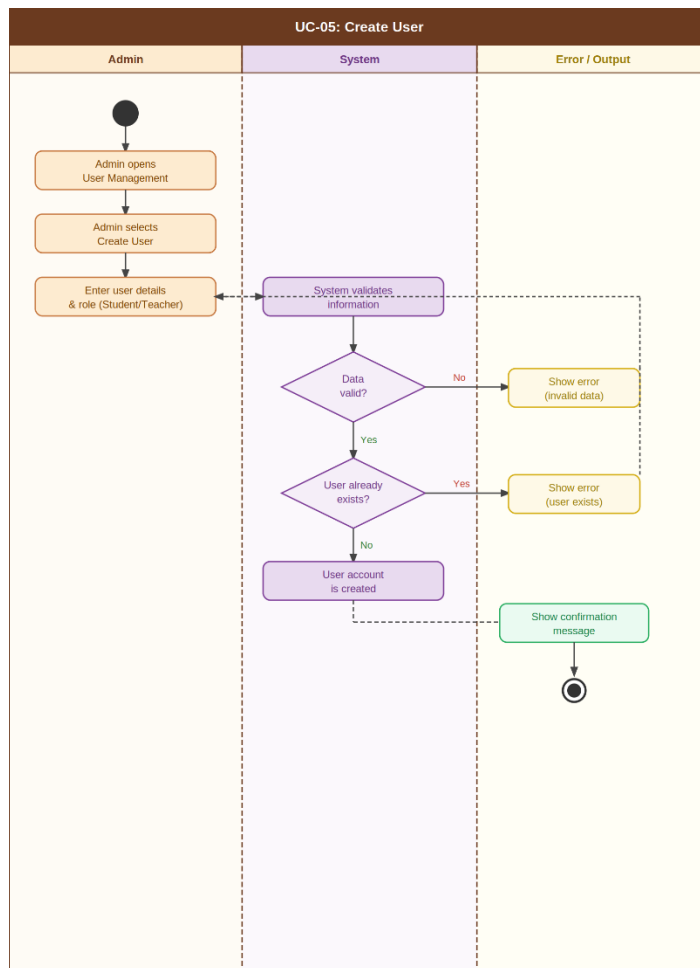


Figure: Create User - Activity Diagram

This activity diagram represents the process of user account creation by the Admin. The Admin accesses User Management, selects Create User, and enters user details including name, email, and role (Student or Teacher).



Swimlane Diagram:

Figure: Create User - Swimlane Diagram

The swimlane diagram highlights Admin authority and system processing. The Admin lane shows information input and submission.

Activity diagram:

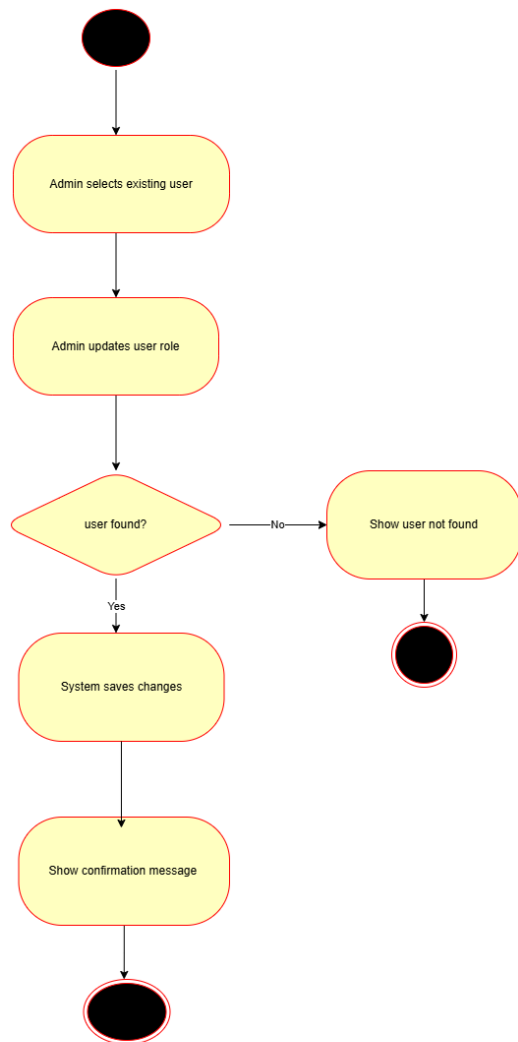


Figure: Update User Role - Activity Diagram

This activity diagram shows how user roles are modified by the Admin. The Admin selects an existing user and chooses a new role (Student to Teacher or vice versa).

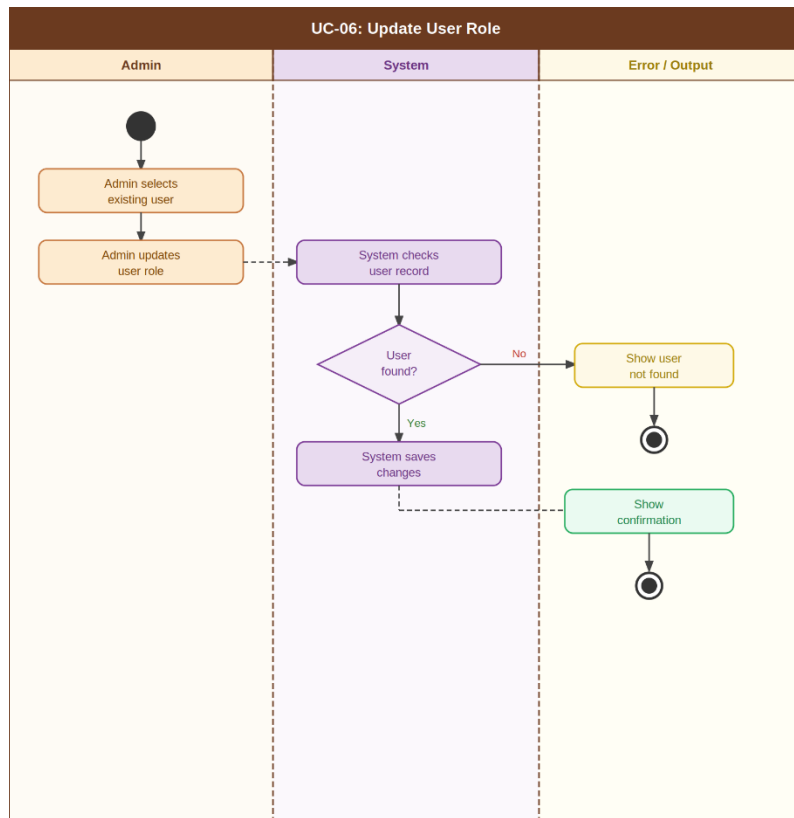


Figure: Update User Role - Swimlane Diagram

The swimlane diagram explains Admin-system interaction during role updates. The Admin lane selects user and new role.

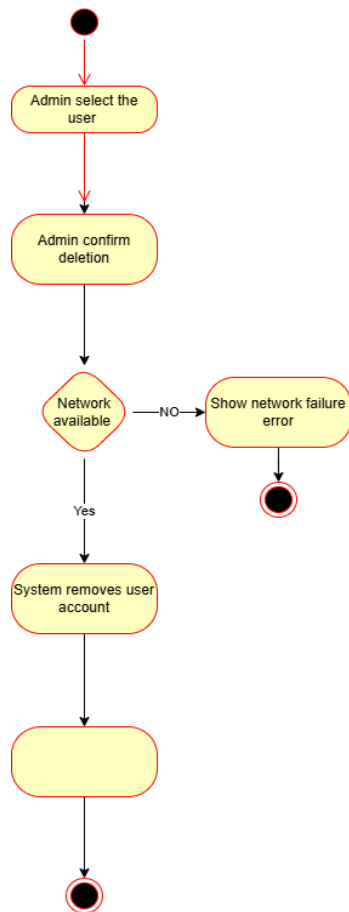


Figure: Delete User - Activity Diagram

This activity diagram illustrates the user deletion process performed by the Admin. The Admin selects a user from the list and confirms the deletion request.

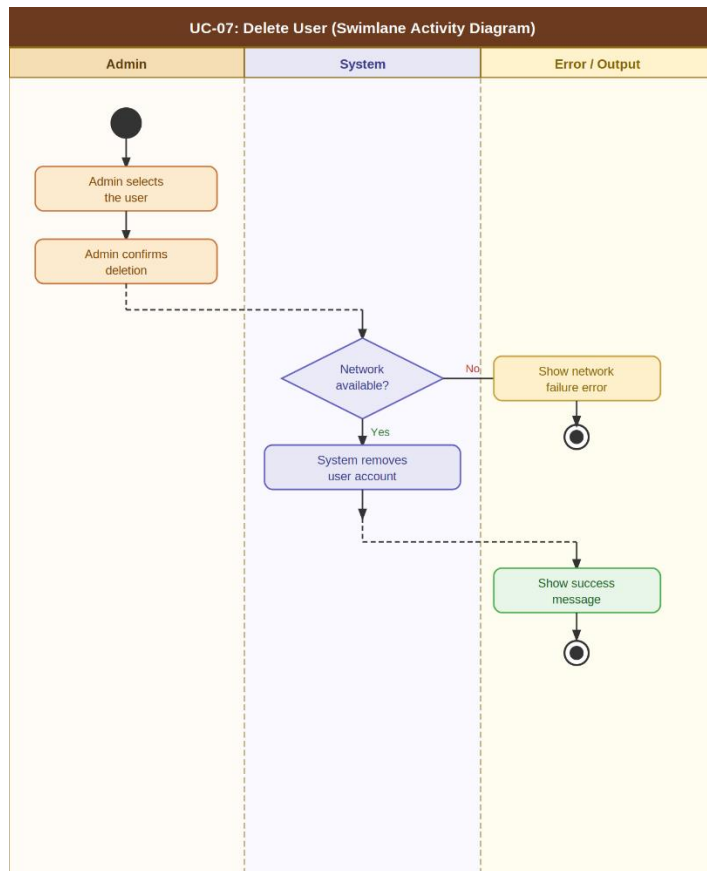


Figure: Delete User - Swimlane Diagram

The swimlane diagram highlights Admin authorization steps. The Admin lane initiates deletion and confirms.

Activity Diagram:

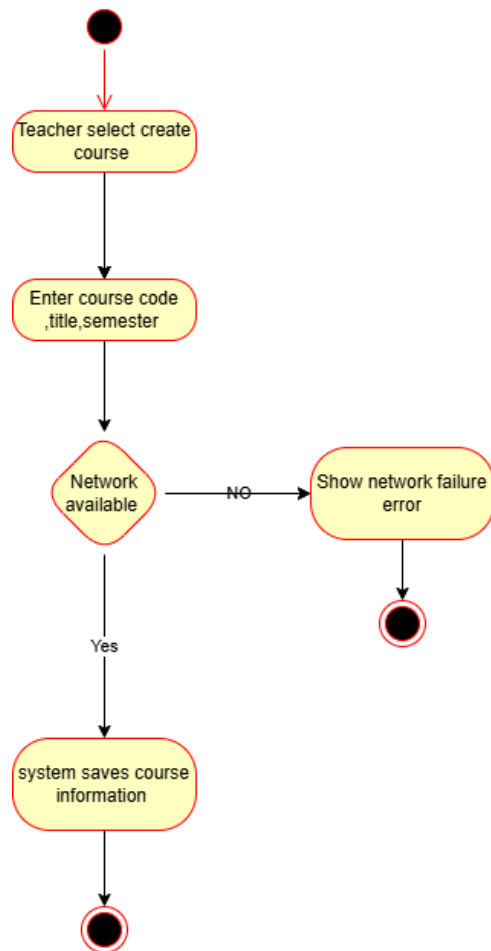


Figure: Create Course - Activity Diagram

This activity diagram describes course creation by the Teacher. The Teacher selects Create Course and enters course code, course title, semester, and academic year. The system validates that the course code is unique.

Swimlane Diagram:

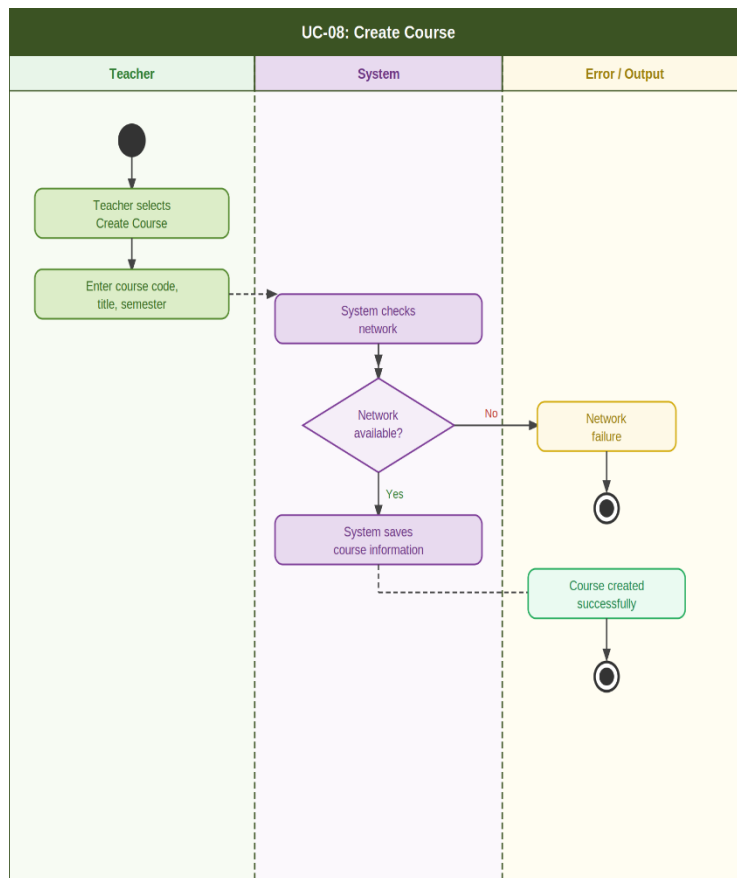


Figure: Create Course - Swimlane Diagram

The swimlane diagram shows Teacher-system coordination during course creation. The Teacher lane inputs course details.

The swimlane diagram shows Teacher-system coordination during course creation. The Teacher lane inputs course details.

Activity Diagram:

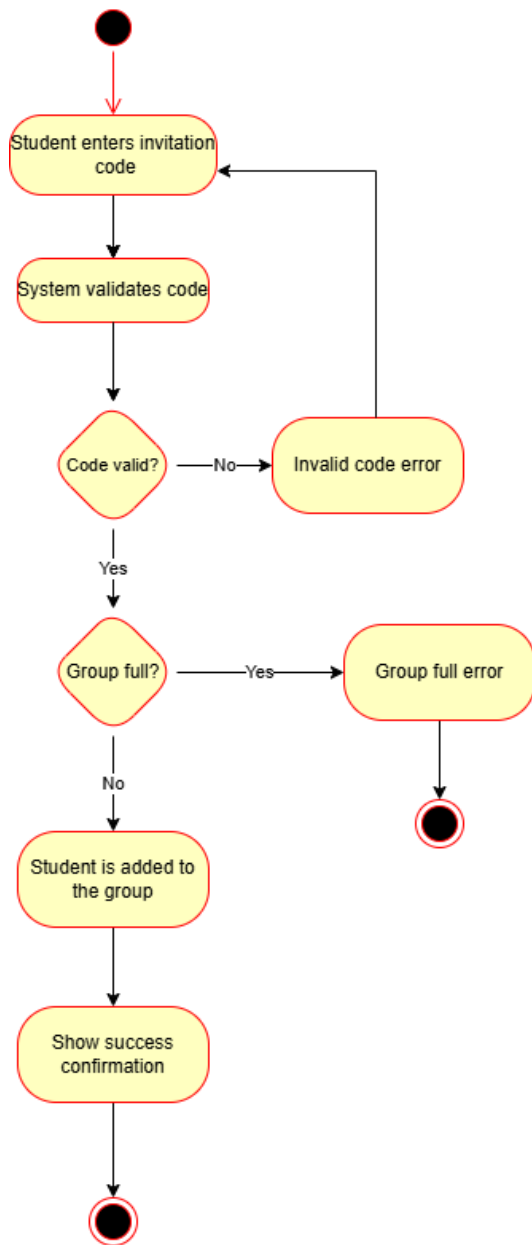


Figure: Join Group - Activity Diagram

This activity diagram illustrates how a Student joins an existing group using an invitation code. The Student selects Join Group and enters the invitation code received from a group member.

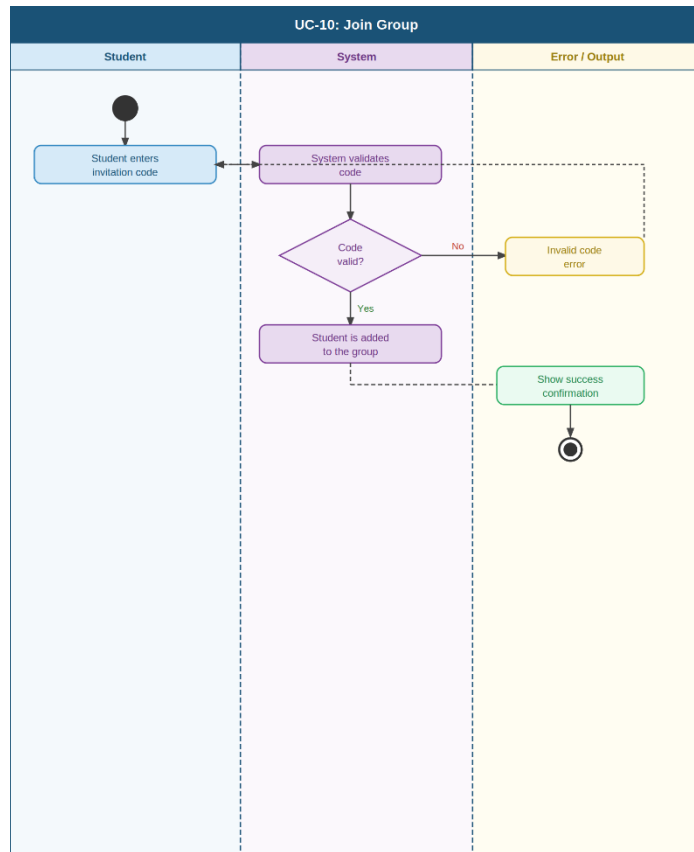


Figure: Join Group - Swimlane Diagram

The swimlane diagram shows Student and System roles during group joining. The Student lane provides the invitation code.

Activity Diagram:

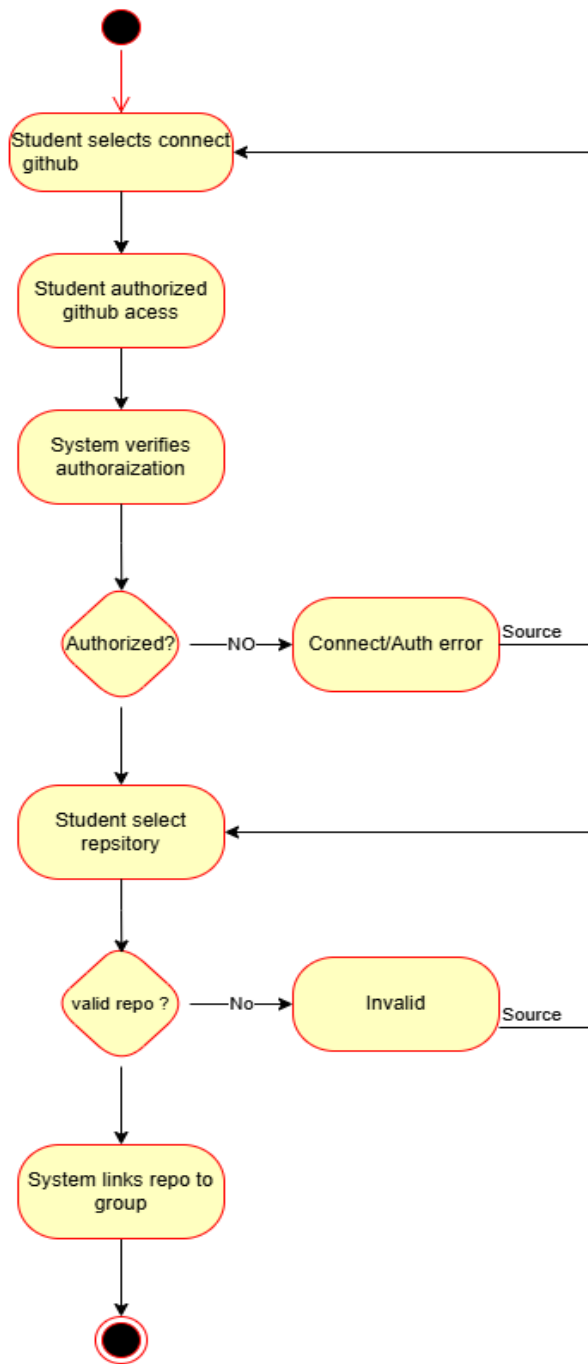
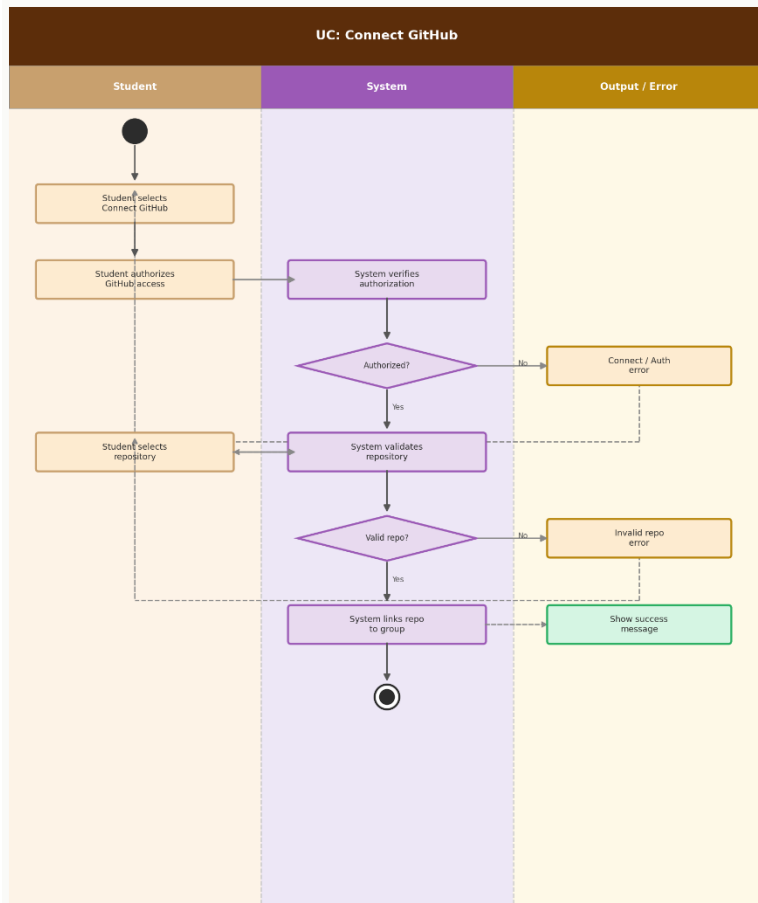


Figure: Connect GitHub Repository - Activity Diagram

This activity diagram represents the GitHub repository connection process. The Student selects Connect GitHub and authorizes GroupSync to access GitHub account via OAuth.



Swimlane Diagram:

Figure: Connect GitHub Repository - Swimlane Diagram

The swimlane diagram highlights interaction between Student, System, and GitHub API. The Student lane initiates connection and authorizes access.

Activity Diagram:

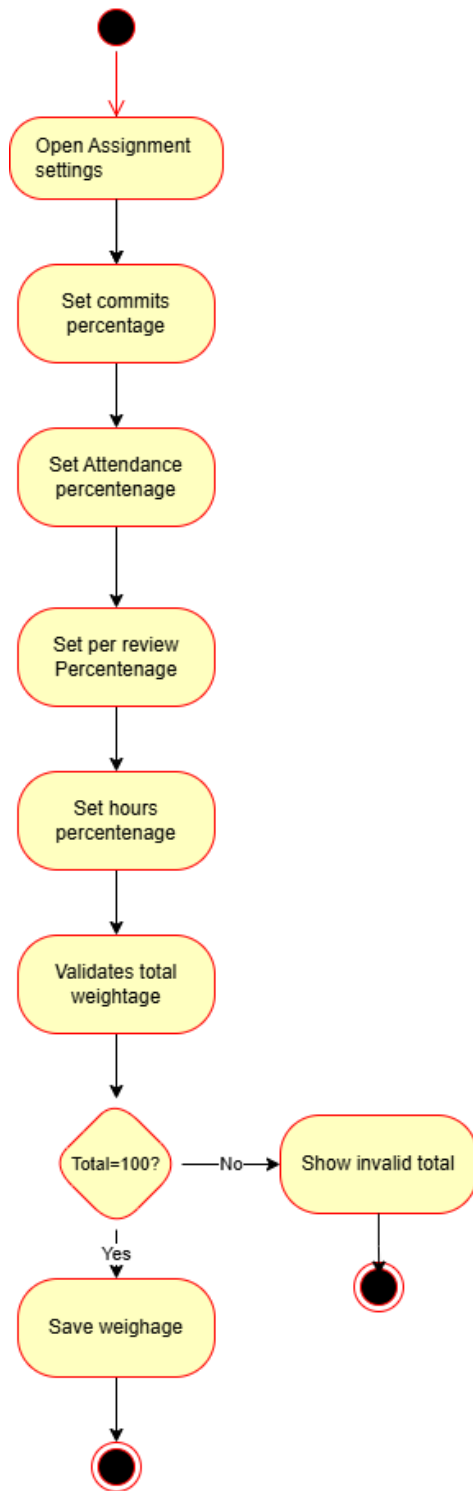


figure: Set Weightage - Activity Diagram

This activity diagram shows how Teachers define evaluation weightage for contribution scoring. The Teacher opens Assignment Settings and enters percentage values for four metrics: GitHub commits, attendance, peer reviews, and working hours.

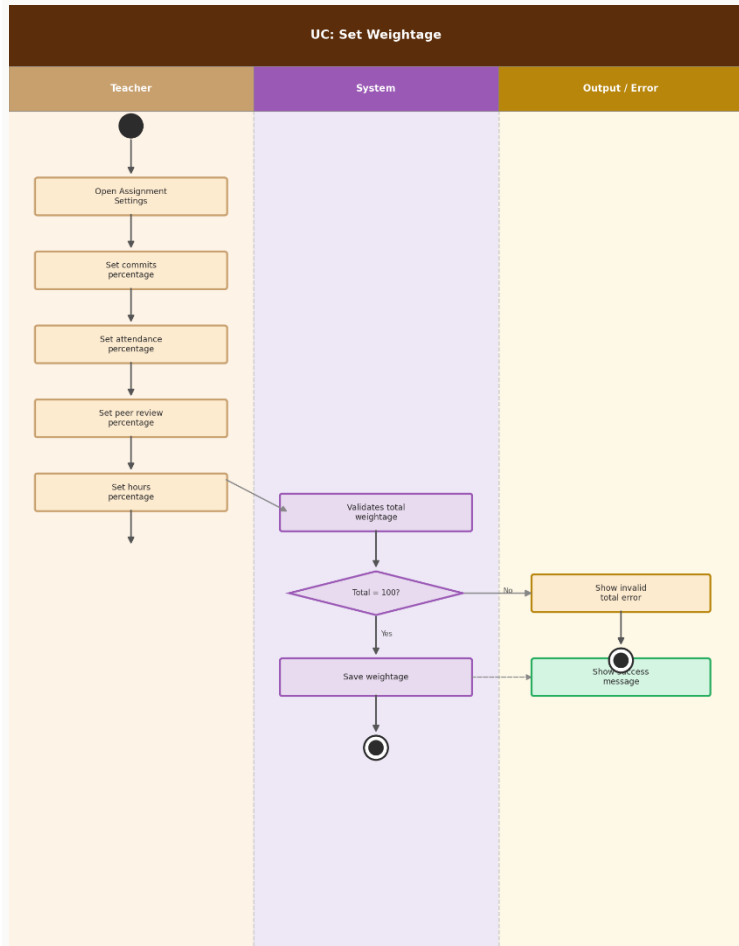


Figure: Set Weightage - Swimlane Diagram

The swimlane diagram shows Teacher input and system validation. The Teacher lane provides weightage values.

Activity Diagram:

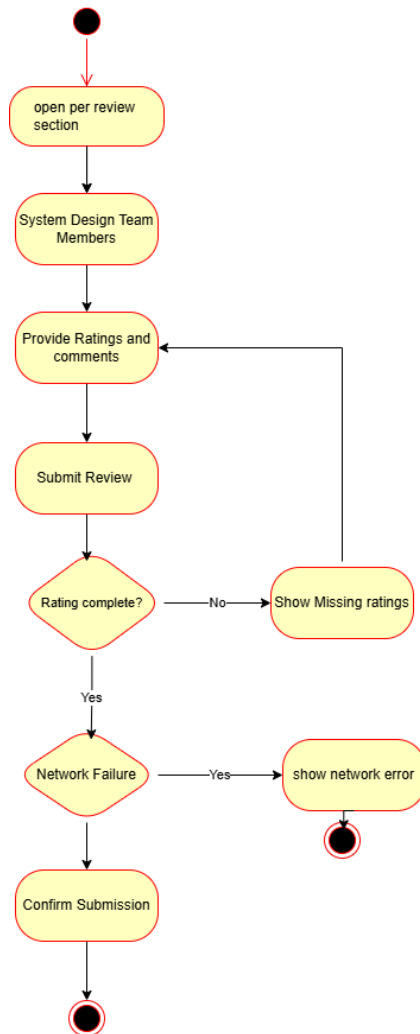
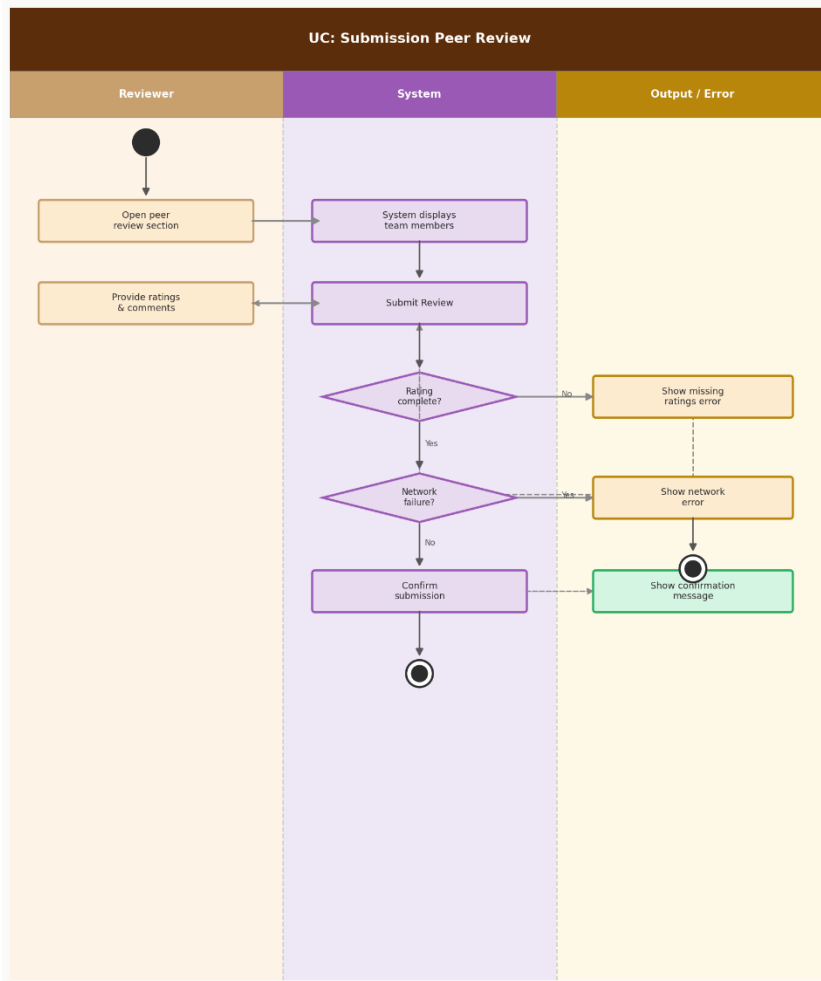


Figure: Submit Peer Review - Activity Diagram

This activity diagram represents the anonymous peer review submission process. The Student opens the Peer Review section during the active review period.

Swimlane Diagram:



Swimlane Diagram:

Figure: Submit Peer Review - Swimlane Diagram

The swimlane diagram highlights secure and anonymous review handling. The Student lane provides ratings and comments.

Activity Diagram:

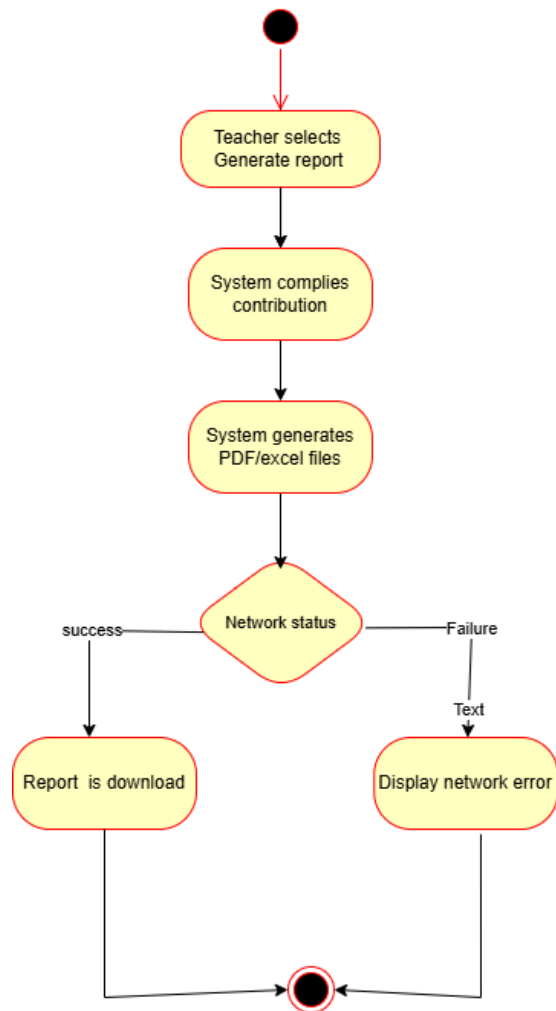


Figure: Generate Report - Activity Diagram

This activity diagram shows the report generation process for Teachers. The Teacher selects a course and project assignment,

Activity Diagram:

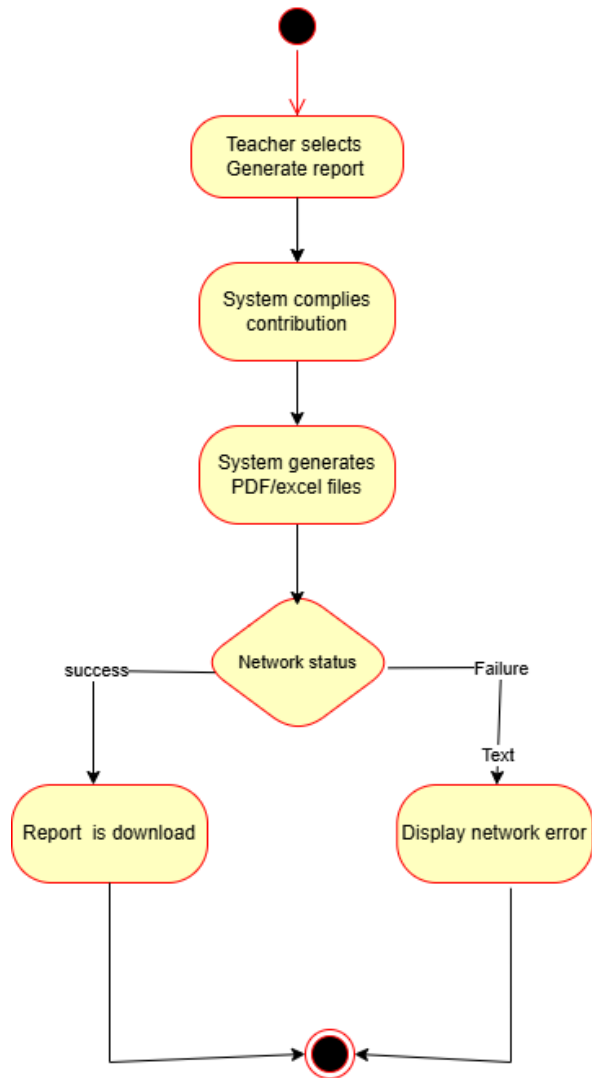


Figure: Generate Report - Activity Diagram

This activity diagram shows the report generation process for Teachers. The Teacher selects a course and project assignment, t

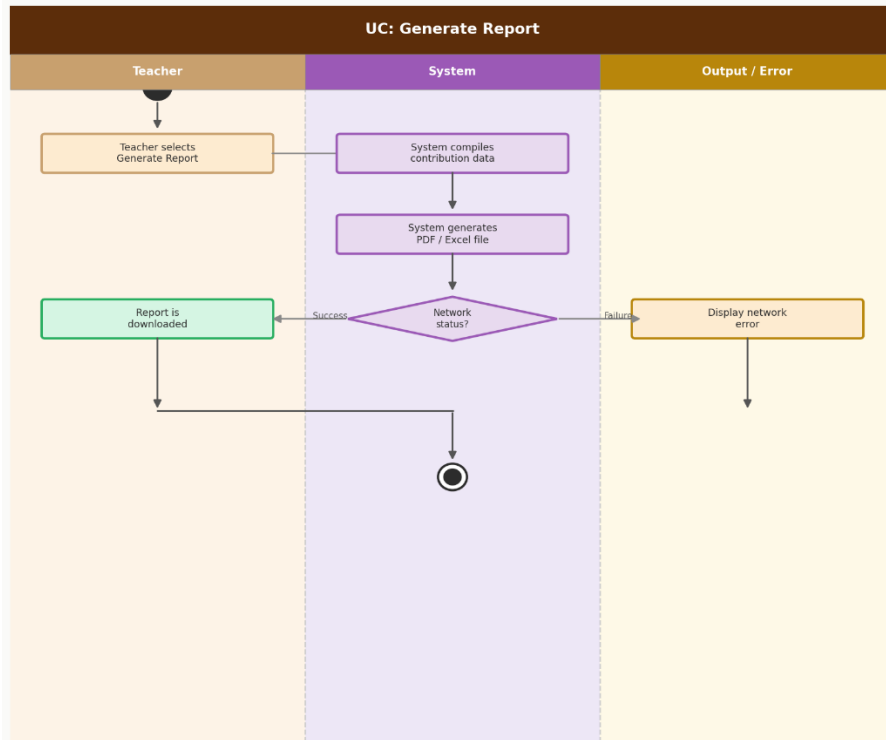


Figure: Generate Report - Swimlane Diagram

The swimlane diagram illustrates Teacher request and system processing. The Teacher lane selects parameters and format.

Activity Diagram:

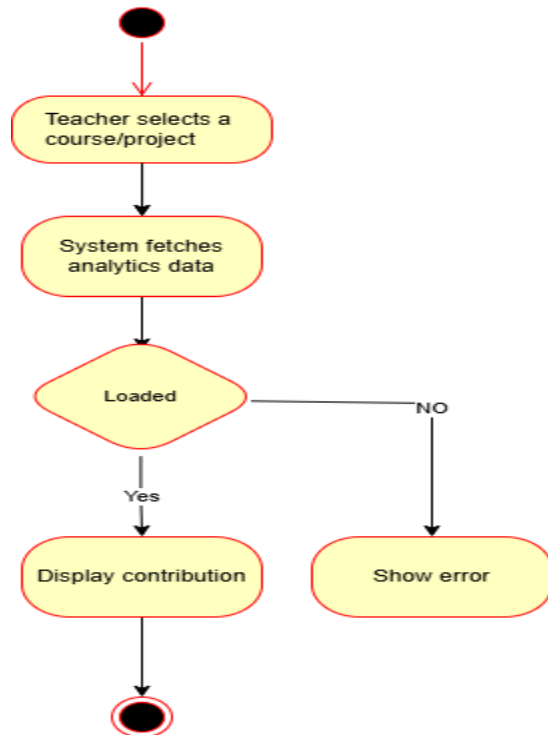
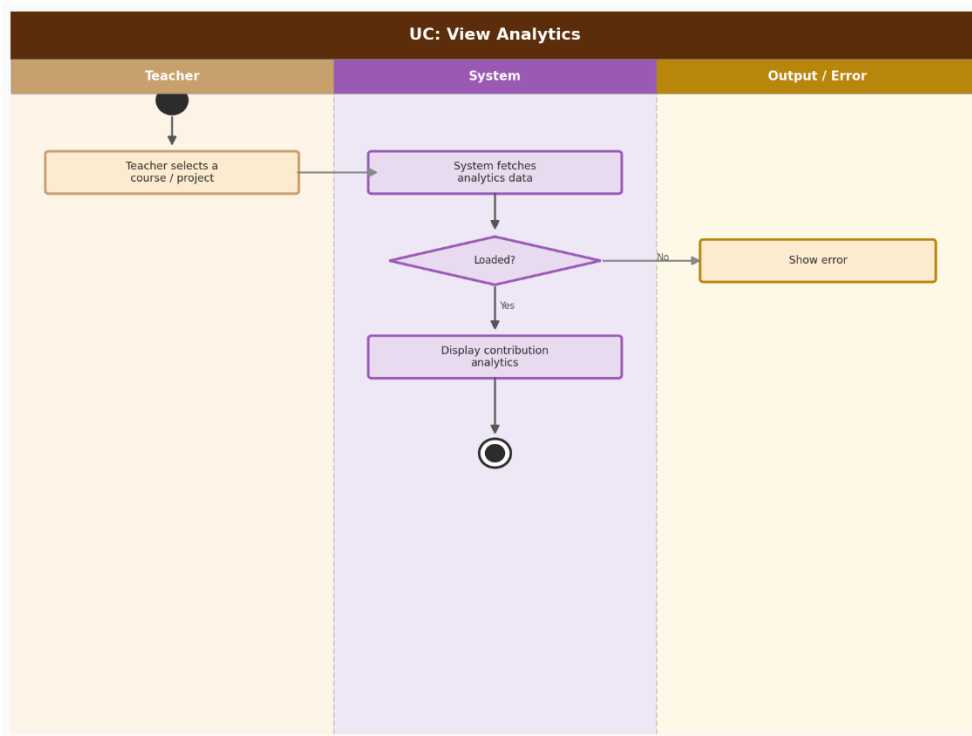


Figure: View Analytics Dashboard - Activity Diagram

This activity diagram represents the analytics dashboard for Teachers. The Teacher selects a course and project assignment.

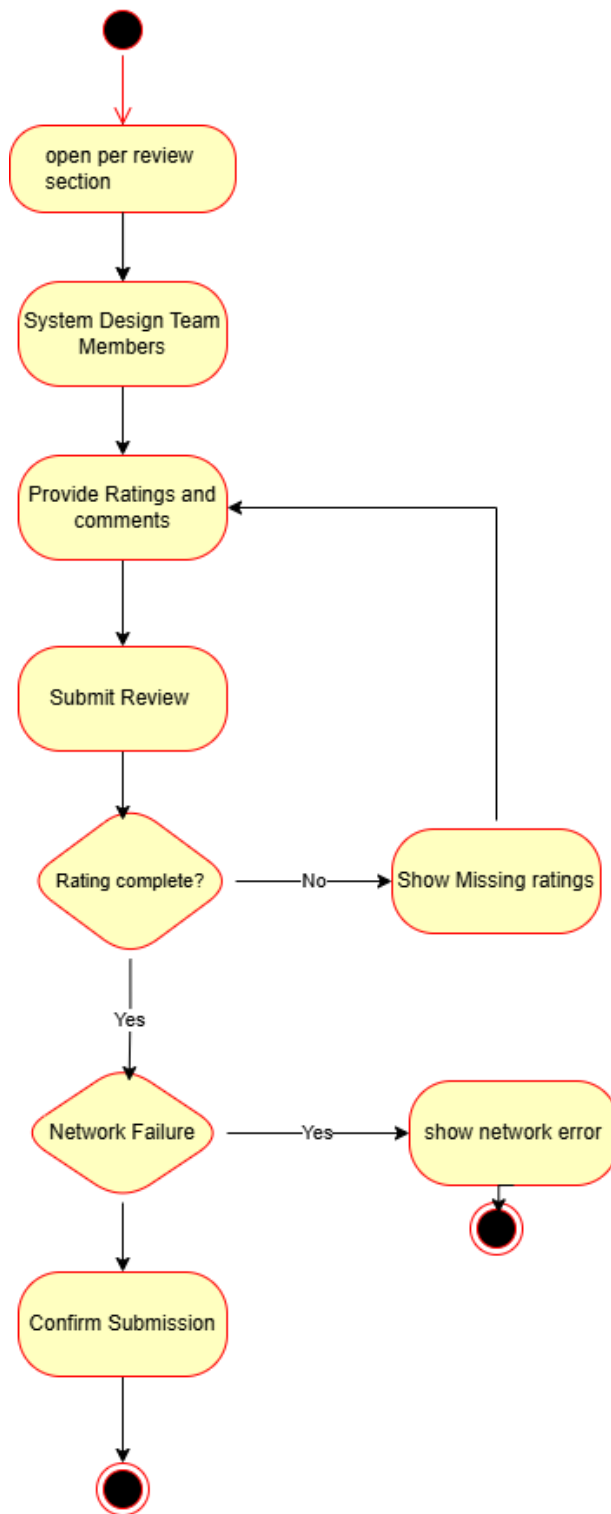


Swimlane Diagram:

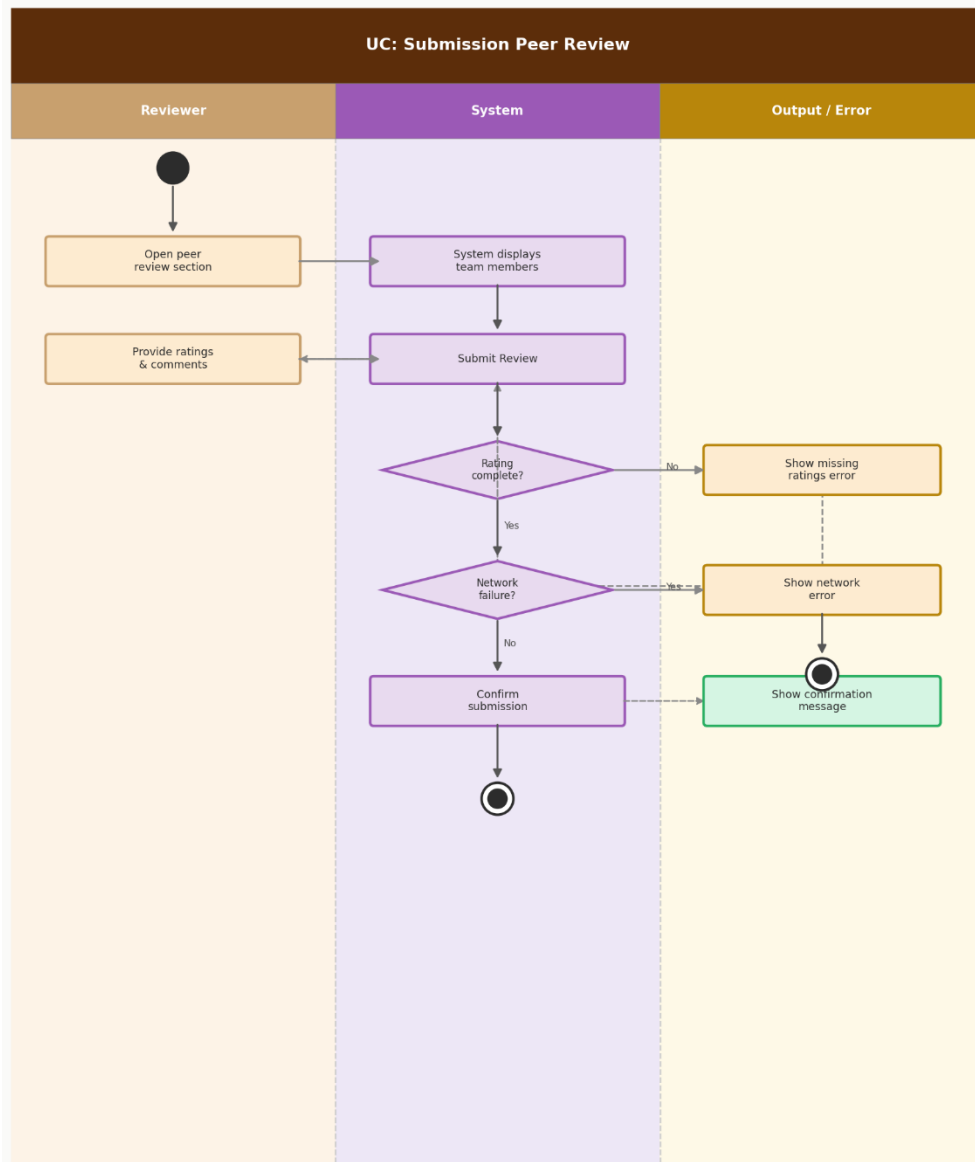
Figure: View Analytics Dashboard - Swimlane Diagram

The swimlane diagram shows data retrieval and visualization. The Teacher lane selects filters. The System lane queries the database, processes analytics, and renders charts and graphs.

Activity Diagram:



Swimlane diagram:



5 Data Modeling

Data Modeling defines the structural foundation of syncsGroup by representation how data is organized , store and related.This section includes the entities relationship and Diagram to visualize entities and relationships It identify keys data object , their attributes ,and interactions among them .

5.1 Entity Relations Diagram (ERD)

5.1.1 Data Objects (Entity):

Sl No	Noun	P/S	Attributes
1	User	P	2,3,4,5,6,7,8
2	userId	P	
3	Fullname	P	
4	Email	P	
5	Password	P	
6	Role	P	
7	Department	P	
8	profilePicture	P	
9	Admin	s	10,11
10	adminID	P	
11	Email	P	
12	Teacher	s	13,14
13	teacherID	P	
14	Email	P	
15	Student	s	16,17,18
16	studentID	P	
17	classRoll	P	
18	Session	P	
19	Course	s	20,21,22
20	courseID	P	
21	Title	P	
22	Code	P	
23	Group	s	24,25
24	groupID	P	
25	InvitationCode	P	
26	ProjectAssignment	s	27,28,29,30
27	assignmentID	P	
28	Title	P	
29	weightageConfig	P	
30	Deadline	P	
31	GitHubRepo	s	32,33
32	repoID	P	
33	repoURL	P	
34	ContributionScore	s	35,36,37
35	scoreID	P	
36	Value	P	
37	breakdown	P	
38	peerReview	s	39,40,41

39	reviewID	P	
40	Rating	P	
41	Comments	P	
42	AuditLog	s	46,47
43	logID	P	
44	Details	P	
45	Report	P	
46	reportID	P	
47	FilePath	p	

5.1.2 Final Data Objects (Entities)

1. User
2. Admin
3. Teacher
4. Student
5. Course
6. group
7. ProjectAssignment
8. GitHubRepo
9. ContributionScore
10. PeerReview
11. AuditLog
12. Review

5.1.3 Data Attributes

User (userID, fullName, email, password, role, department, profilePicture)

Admin (adminID, email, password)

Teacher (teacherID, email, department)

Student (studentID, classRoll, session, email)

Course (courseID, title, code, semester)

Group (groupID, invitationCode)

ProjectAssignment (assignmentID, title, weightageConfig, deadline)

GitHubRepo (repoID, repoURL, lastFetched)

ContributionScore (scoreID, value, breakdown)
PeerReview (reviewID, rating, comments, submittedAt)
AuditLog (logID, details, timestamp)
Report (reportID, filePath, generatedAt)

5.1.4 Relationships

User has one **Admin** (1:1)
User has one **Teacher** (1:1)
User has one **Student** (1:1)
Teacher manages **Course** (1:M)
Course contains **Group** (1:M)
Student belongs to **Group** (M:1)
Course has **ProjectAssignment** (1:M)
Group submits **PeerReview** (1:M)
Group connects to **GitHubRepo** (1:1)
Student receives **ContributionScore** (1:M)
Teacher generates **Report** (1:M)
System writes **AuditLog** (M:1)

5.1.5 ER Diagram

(A diagram would be inserted here showing entities: User, Admin, Teacher, Student, Course, Group, ProjectAssignment, GitHubRepo, ContributionScore, PeerReview, AuditLog, Report with their relationships and attributes.)

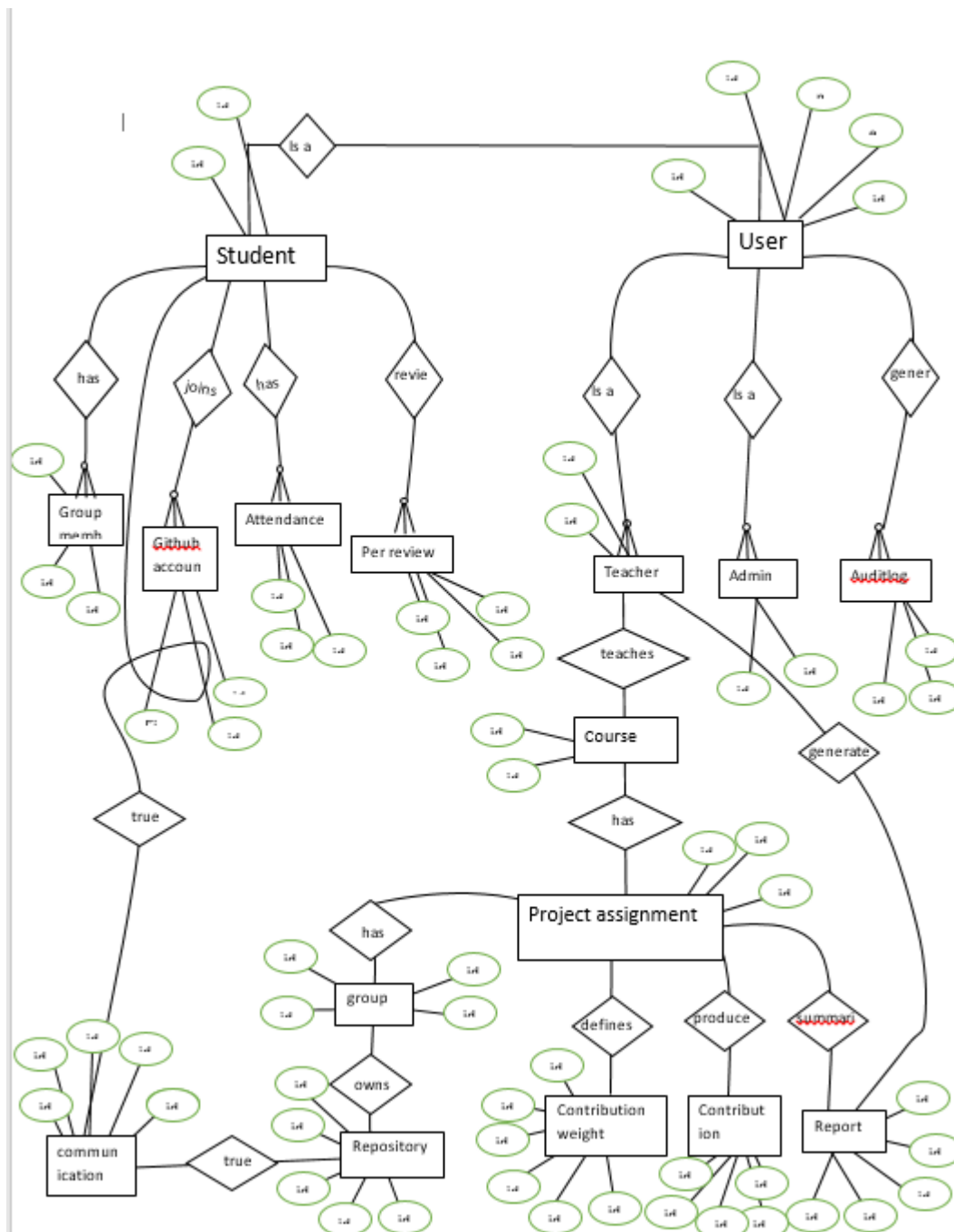


Figure 1 : Entity Relation Diagram of database with attributes

5.2 Schema

(A database schema diagram would be inserted here showing all tables, columns, data types, primary keys, and foreign key relationships.)

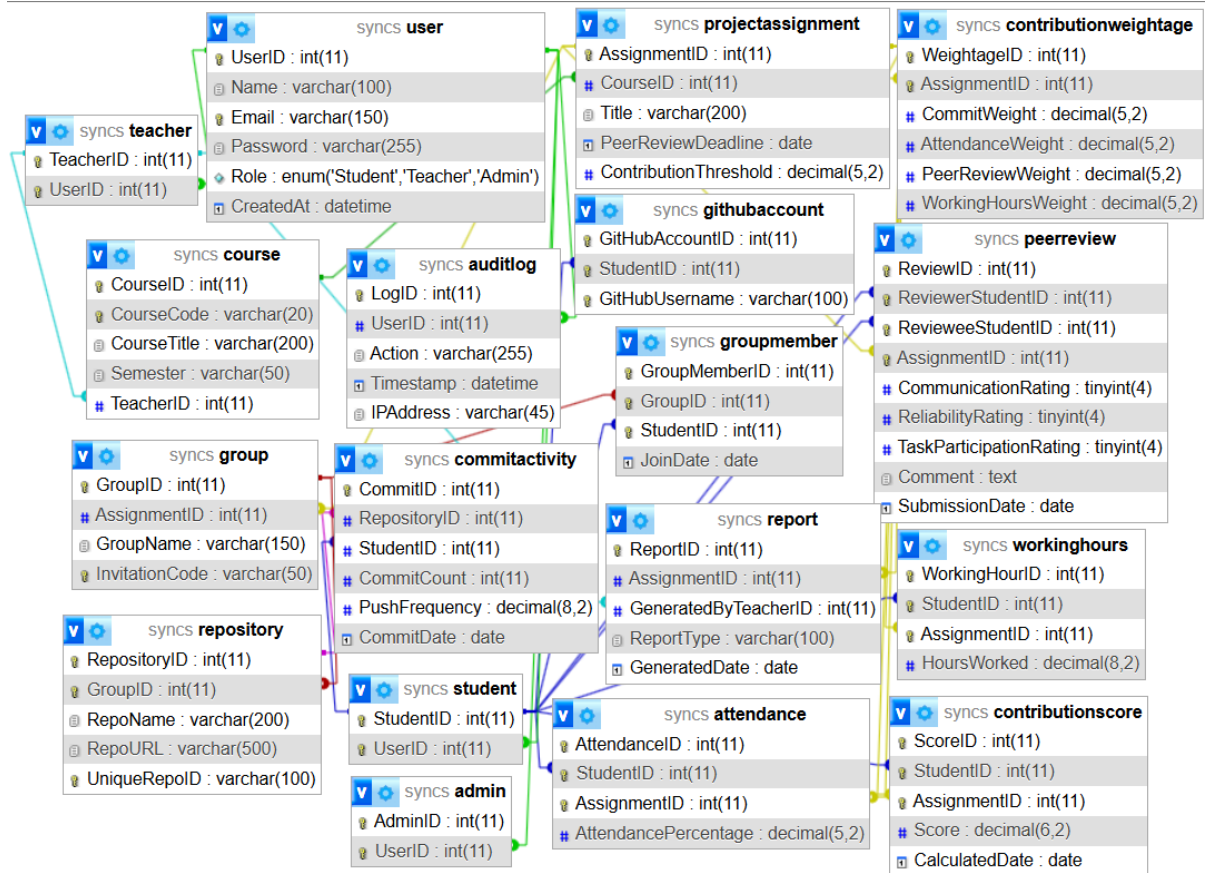


Figure 2: Database Schema Diagram

6. Class-Based Modeling

Class-based modeling defines the structure of the entire system by identifying the static structure of objects in that system. A class model defines attributes and operations for the objects of each class and also the relationship between the objects, and the collaborations that occur between the classes of the systems. The elements of a class-based model include classes and objects, attributes, operations, Class- Responsibility Collaborator (CRC) models, collaboration diagrams, and packages

6.1 Step-1: Identifying and Categorizing All Nouns

Table 1: Categorization of Identified Nouns

Category	Identified Nouns
External Entities	Student, Teacher, Admin, Database, Interface, GitHub API
Things	Course, Group, Project, Contribution Score, Peer Review, Report, Repository, Weightage, Notification, Audit Log
Occurrences / Events	Login, Register, Create Group, Join Group, Submit Review, Fetch Commits, Generate Report, Flag Contributor
Roles	Student, Teacher, Admin
Organizational Units	University, Department, Course Code
Places	Database, Server, GitHub Cloud
Structures	System, Dashboard, Analytics Chart

6.2 Step-2: Selection of Potential Classes

The identified nouns from Step-1 were systematically evaluated to determine whether they qualify as potential classes in the object-oriented design of the GroupSync System. The selection was based on the following standard class selection criteria:

Criterion	Description
1. Retained Information	The noun represents data that must be stored persistently in the system.
2. Needed Services	The noun requires specific operations or behaviors to be performed on or by it.
3. Multiple Attributes	The noun has multiple relevant properties that describe it.
4. Common Attributes	The noun shares attributes with other nouns, suggesting a potential inheritance hierarchy.
5. Common Operations	The noun shares operations with other nouns, suggesting a potential inheritance hierarchy.
6. Essential System Requirement	The noun is explicitly required by stakeholders or system specifications.

6.2.1 Potential Class Selection Table

Table: Potential Class Selection and Evaluation Status for GroupSync System

Sl. No	Potential Class (Noun)	Status
1	User	Accepted
2	Admin	Accepted
3	Teacher	Accepted
4	Student	Accepted
5	Course	Accepted

6	Group	Accepted
7	ProjectAssignment	Accepted
8	ContributionScore	Accepted
9	PeerReview	Accepted
10	GitHubRepo	Accepted
11	Report	Accepted
12	Notification	Accepted
13	AuditLog	Accepted
14	Repository (abstract)	Rejected . Fails criteria 3, 4, and 5.
15	Weightage	Rejected. Fails criteria 1, 2, and 6.
16	Interface	Rejected. Fails criteria 1, 2, and 6.
17	GitHub API	Rejected. Fails criteria 1, 3, and 4.
18	Database	Rejected. Fails criteria 2 and 5.
19	Server	Rejected. Fails criteria 1, 2, and 6.
20	University	Rejected. Fails criteria 2, 3, and 5.
21	Department	Rejected. Fails criteria 2, 3, and 6.
22	Course Code	Rejected. Fails criteria 1, 2, and 5.
23	Dashboard	Rejected. Fails criteria 1, 2, and 6.
24	Analytics Chart	Rejected. Fails criteria 1, 2, and 4.
25	Login	Rejected. Fails criteria 1, 3, and 6.
26	Register	Rejected. Fails criteria 1, 3, and 6.
27	Create Group	Rejected. Fails criteria 1, 3, and 6.
28	Join Group	Rejected. Fails criteria 1, 3, and 6.
29	Submit Review	Rejected. Fails criteria 1, 3, and 6.
30	Fetch Commits	Rejected. Fails criteria 1, 3, and 6.
31	Generate Report	Rejected. Fails criteria 1, 3, and 6.
32	Flag Contributor	Rejected. Fails criteria 1, 3, and 6.

33	System	Rejected. Fails criteria 2, 3, and 5.
34	GitHub Cloud	Rejected. Fails criteria 1, 2, and 3.

6.3 Class Based Modeling Diagram

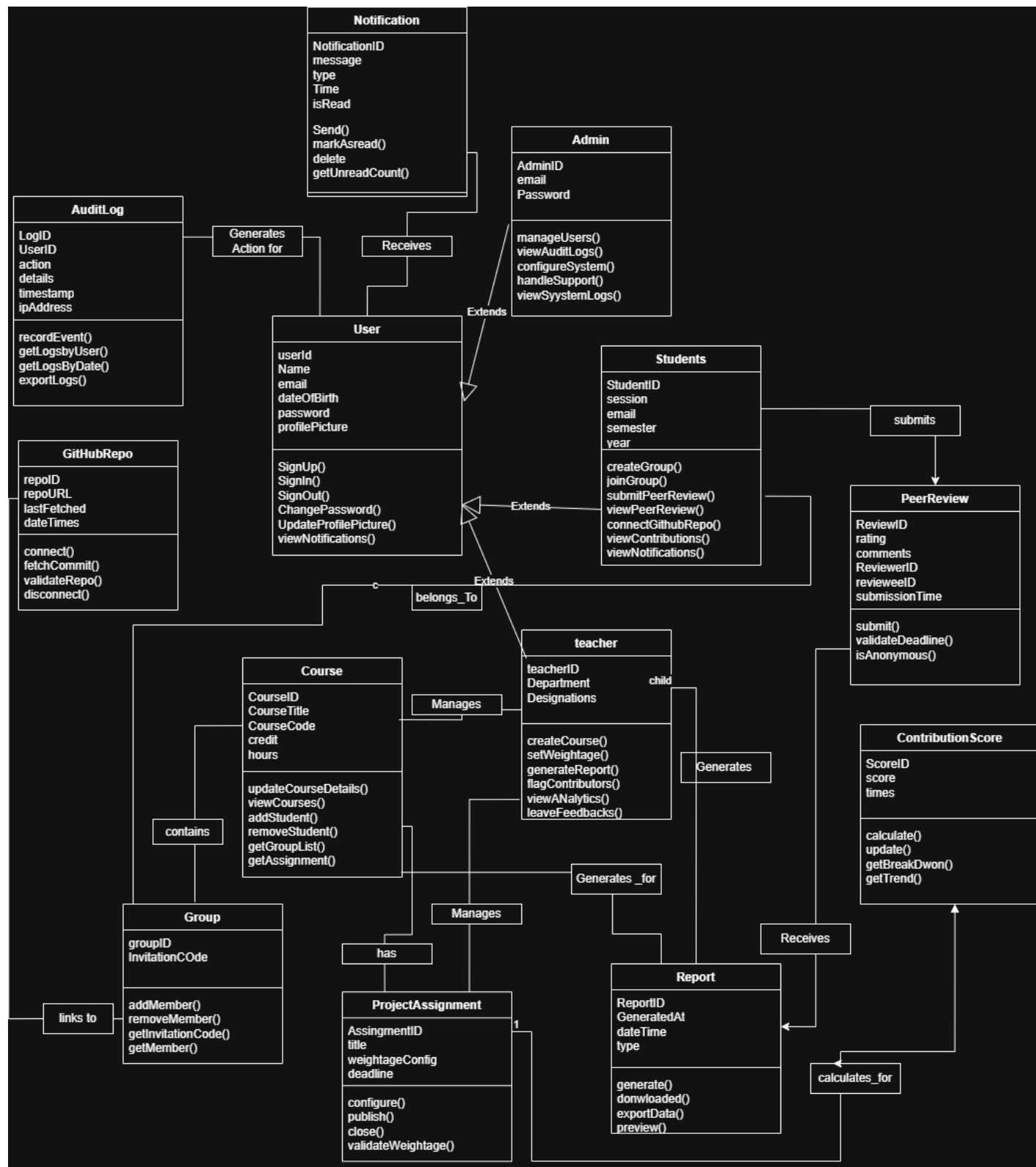


Figure 1: Class Based Modeling Diagram of the GroupSync

7: Behavioral Modeling

Behavioral modeling describes how the GroupSync system behaves at runtime — how its components respond to events, transition between states, and interact with actors over time. This section presents two types of UML behavioral diagrams:

1. State Transition Diagrams — describe how each major entity or subsystem moves between states in response to events.
2. Sequence Diagrams — describe how actors and system components interact in a time-ordered sequence to complete key use cases.

7.1 State Transition Diagrams

A State Transition Diagram captures all possible states of a system entity and the events or conditions that trigger transitions between those states. For GroupSync, eight entities have been modeled — covering user lifecycle, group management, GitHub integration, contribution scoring, peer review workflow, free rider detection, professor reporting, and system session management.

Figure 1: User Account State Diagram

This diagram models the complete lifecycle of a GroupSync user account — from initial registration with a .edu email address, through email verification and GitHub OAuth connection, to active use, logout, suspension, and eventual deletion. The Suspended state is triggered by a professor or admin action such as a policy violation, and can be restored to Active once resolved.

Figure 1: User Account State Diagram

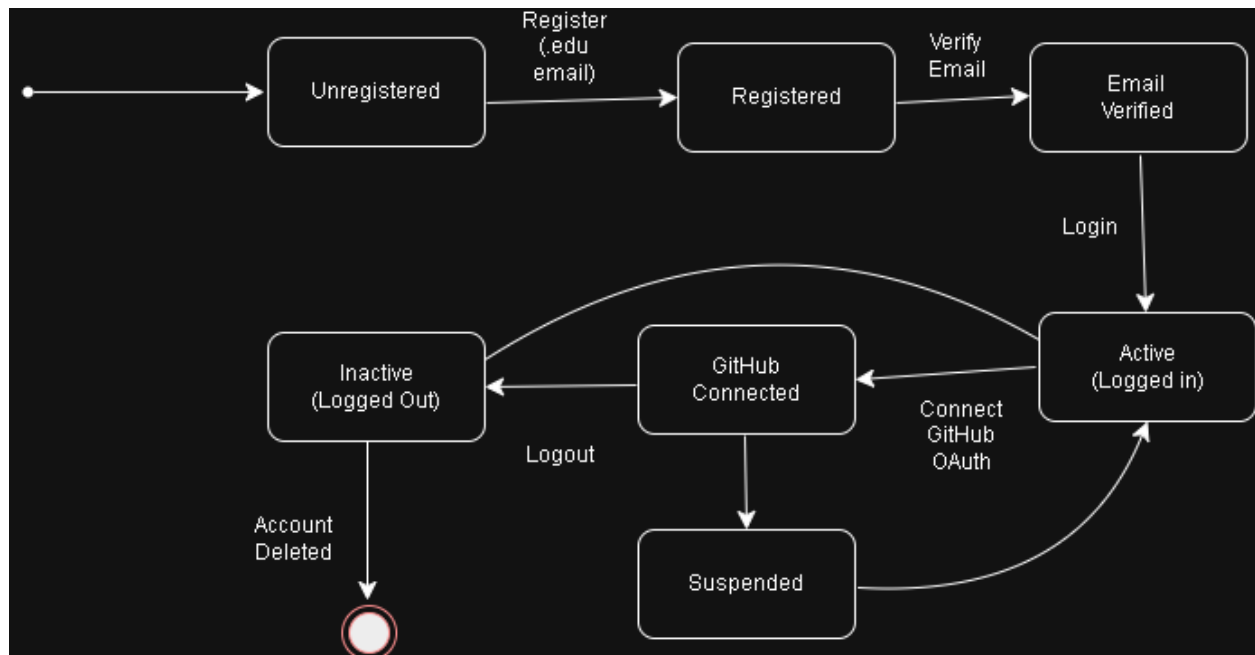


Figure 1: User Account State Diagram

Figure 2: Group State Diagram

This diagram represents the lifecycle of a project group within GroupSync. A group starts in the Created state when a student creates it, progresses through Recruiting (while members join via invite code), GitHub Linked (once a repository is added), and Active (when work begins). If a free rider is detected, the group enters Under Review and may be Flagged. On project submission the group moves to Completed and eventually to Archived after the course ends.

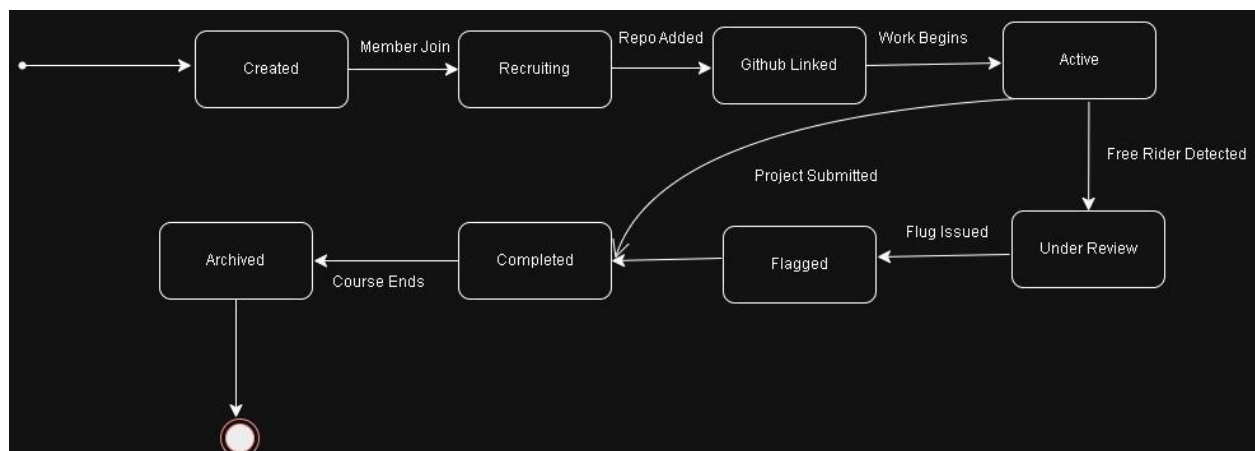


Figure 2: Group State Diagram

Figure 3: GitHub Integration State Diagram

This diagram models the states involved in connecting a student's GitHub account and keeping it synchronized. After OAuth authorization and repository registration, the system enters Webhook Active state and listens for push events. On each event it transitions to Syncing and then Sync Complete. Error states — Rate Limited and Token Expired — are handled with automatic retry and re-authentication flows respectively.

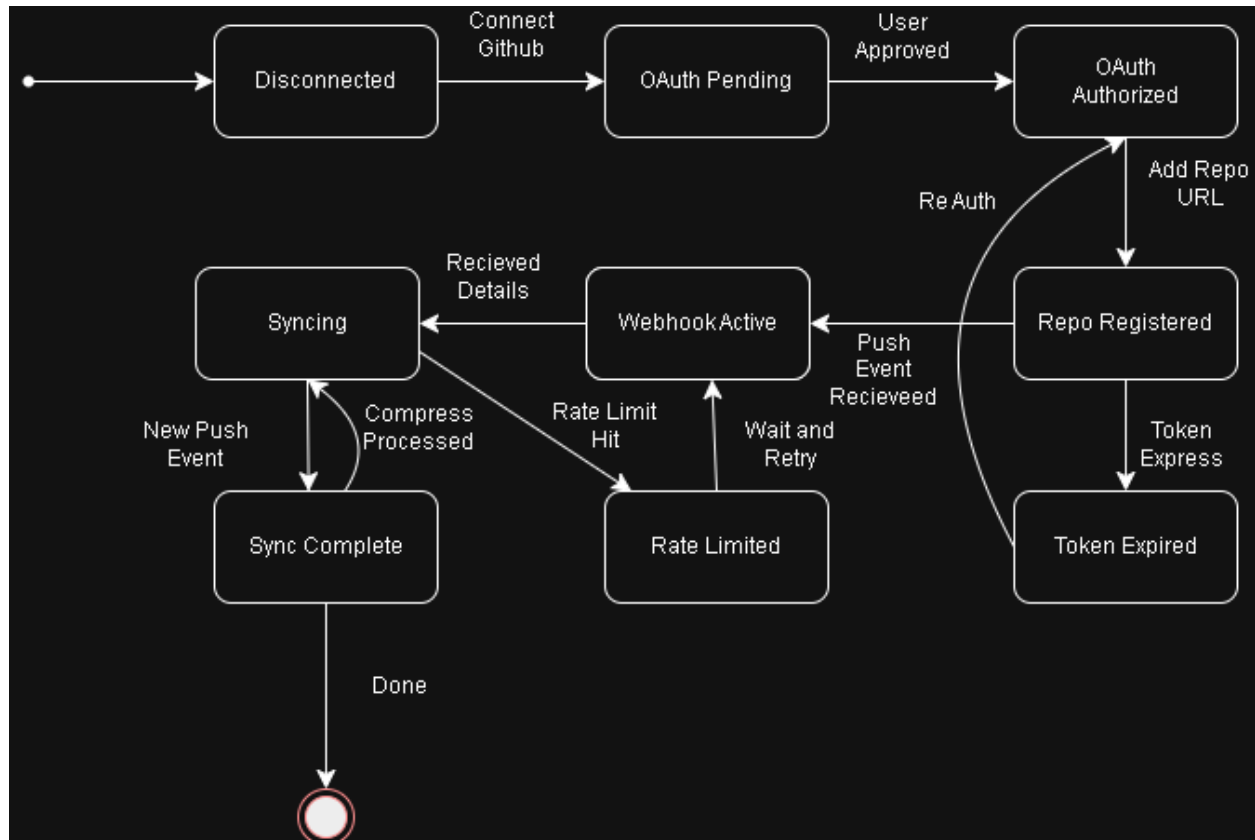


Figure 3: GitHub Integration State Diagram

Figure 4: Contribution Score State Diagram

This diagram tracks the weekly scoring lifecycle for each student. After metrics are aggregated (commits, hours, meetings, peer ratings), the scoring algorithm runs and produces a score. Scores of 70 or above result in LOW RISK with no action. Scores between 30 and 69 trigger MEDIUM RISK monitoring. Scores below 50 trigger an AI analysis call (one API call per student), and scores below 30 elevate to HIGH RISK, resulting in a professor alert. At the start of each new week the score resets for fresh calculation.

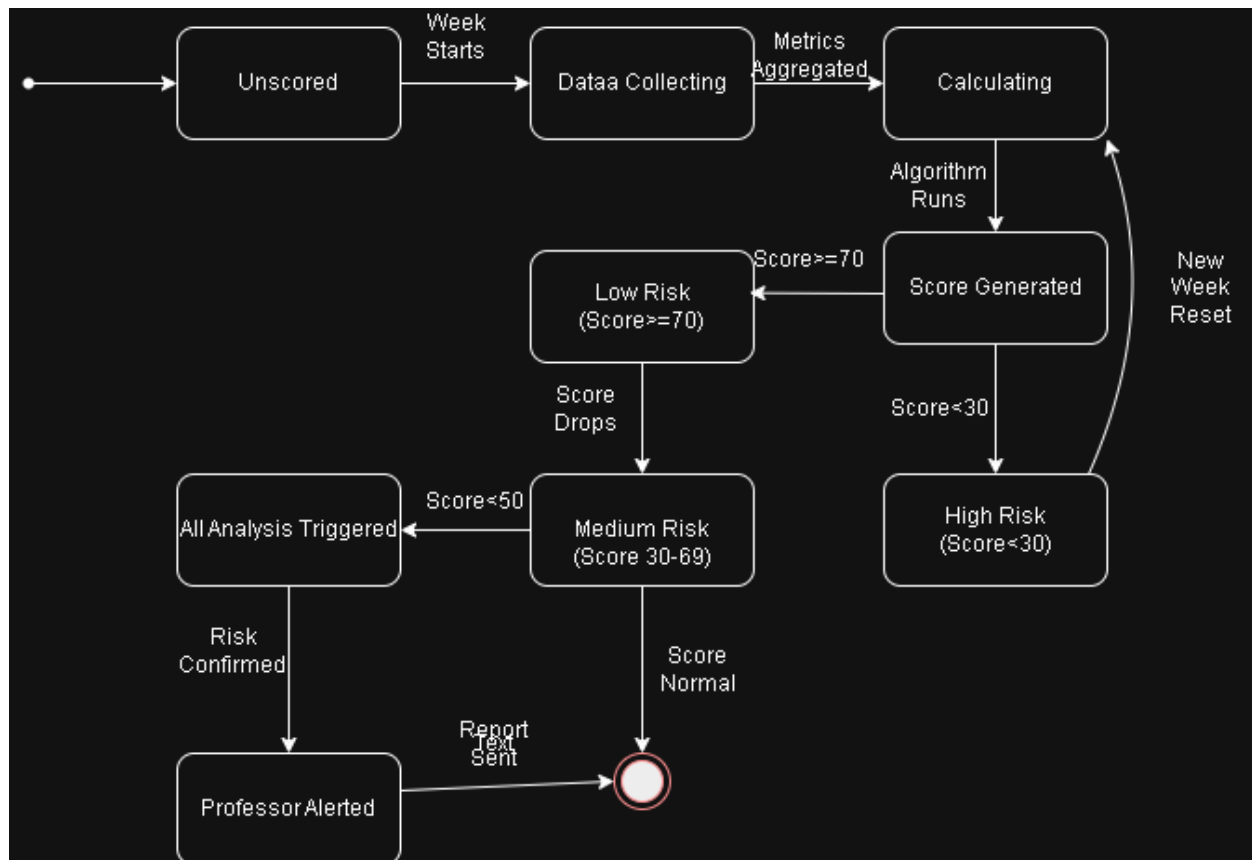


Figure 4: Contribution Score State Diagram

Figure 5: Peer Review State Diagram

This diagram models the anonymous peer review workflow. Reviews open at a set point in the week (Pending), students fill in 5-category ratings and text feedback (In Progress), and submit them. The system validates each submission — reviews that deviate by more than 2 standard deviations from the group average are Flagged as outliers and sent for professor review before being either cleared or rejected. Accepted reviews are aggregated into the student's contribution score.

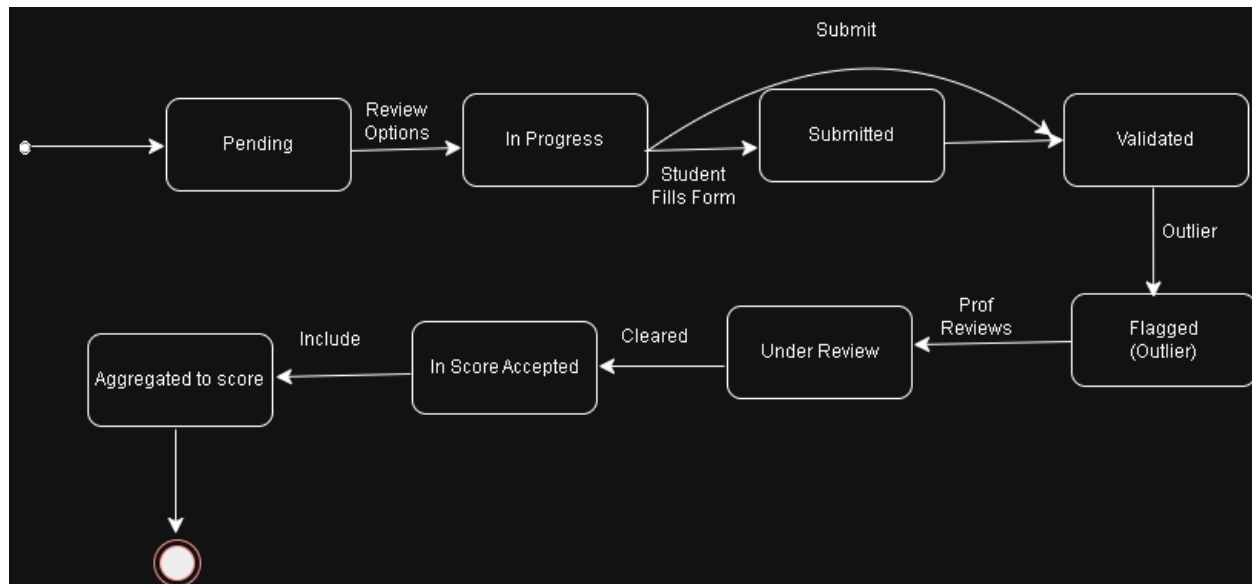


Figure 5: Peer Review State Diagram

Figure 6: Free Rider Detection State Diagram

This diagram models the weekly free rider detection pipeline. Every student enters a Rule Engine Check. Those with normal scores are marked Low Risk (no action). Students with scores between 30–50% of the group average are routed to AI Analysis, which returns a risk level. Students below 30% are immediately flagged as High Risk. High Risk triggers automatic professor notification and a warning email to the student. If the score improves in a subsequent week the student moves to Resolved.

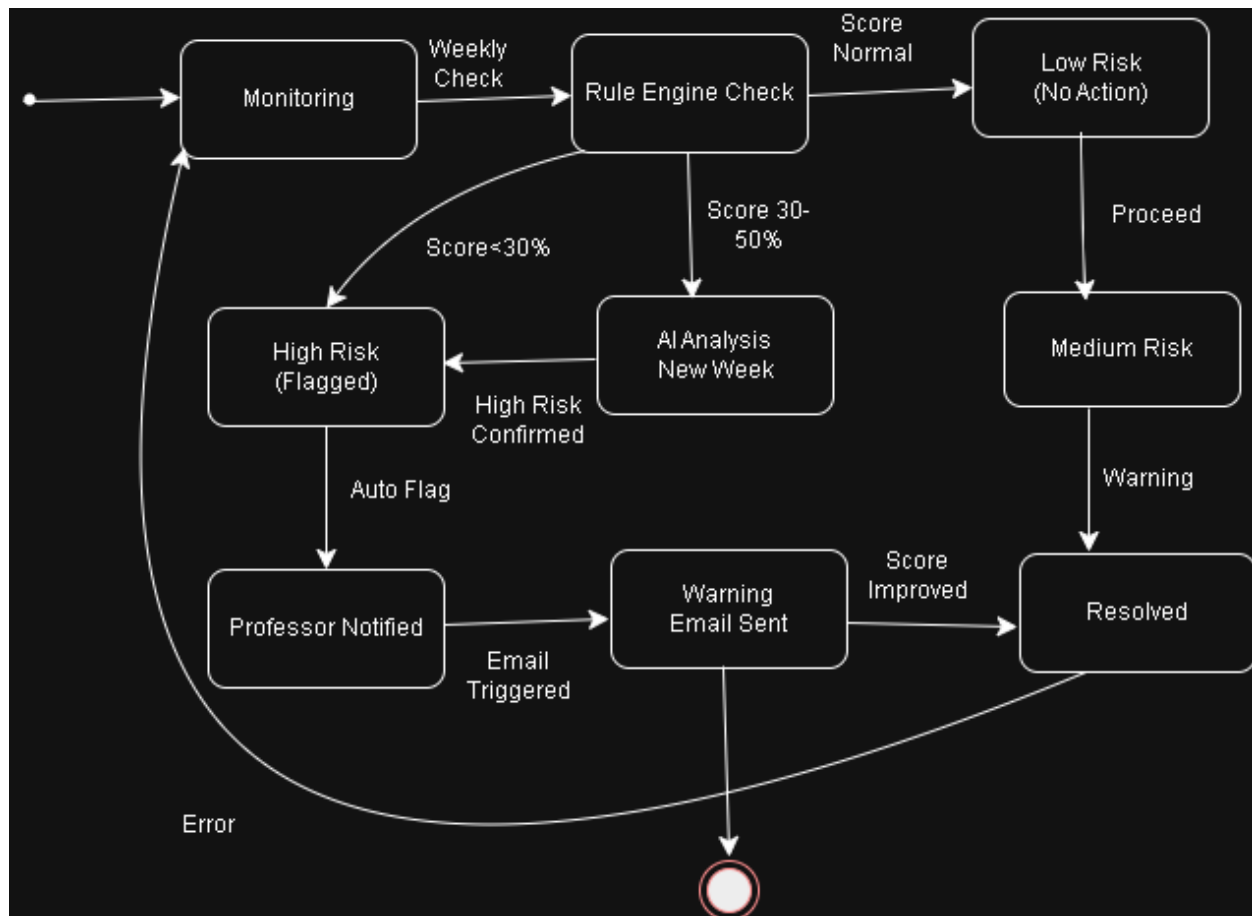


Figure 6: Free Rider Detection State Diagram

Figure 7: Professor Report State Diagram

This diagram models the states a professor passes through when generating analytics reports. Starting from Idle on the dashboard, the professor selects filters (group and week), which triggers data fetching and rendering of analytics charts. From there, the professor can generate a PDF report. Once ready, the report can be downloaded or exported as CSV for LMS import. The professor can return to generate additional reports without leaving the dashboard.

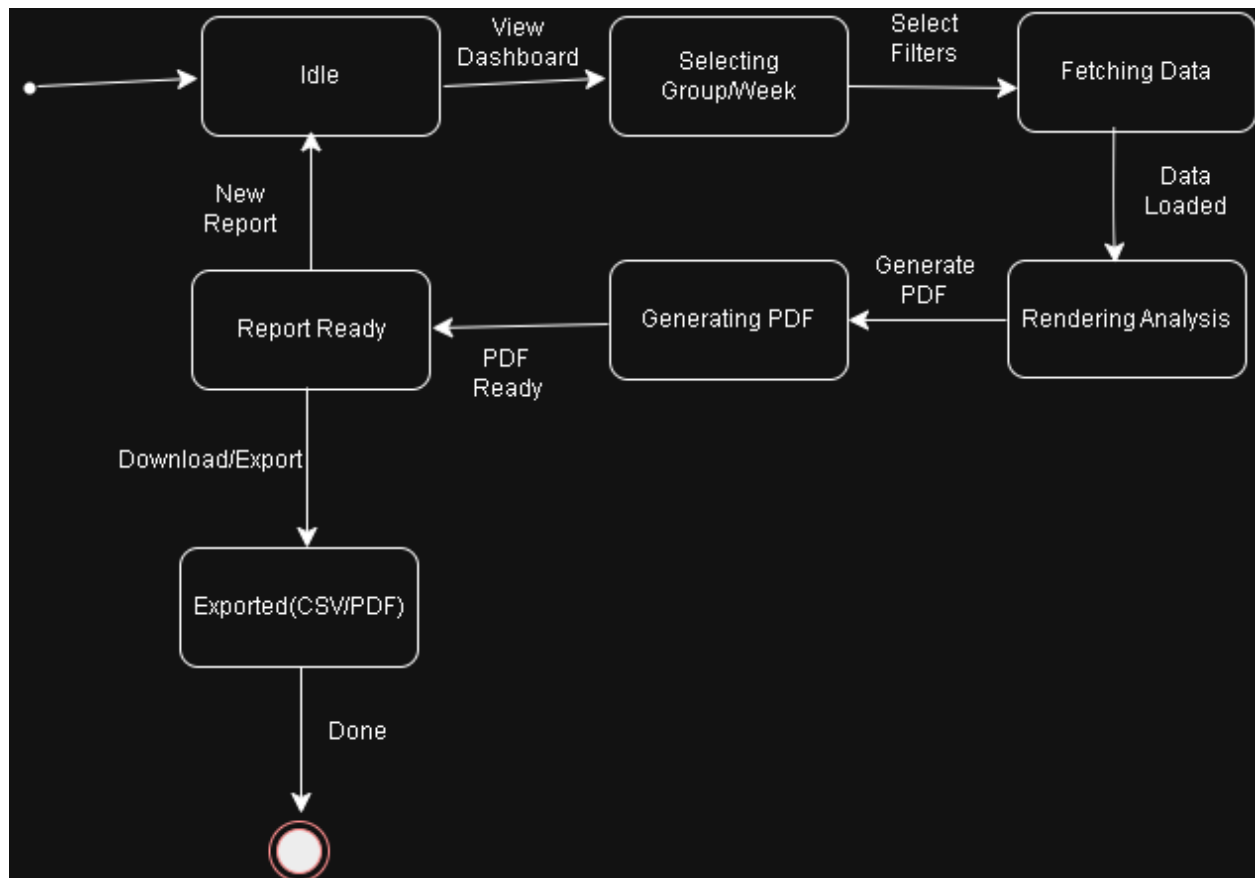


Figure 7: Professor Report State Diagram

Figure 8: System Session State Diagram

This diagram models the JWT-based session lifecycle for all users. A login request triggers authentication with bcrypt credential comparison. On success, a JWT token is issued and a WebSocket session opens. As the token nears expiry it is automatically refreshed without logging the user out. Excessive failed login attempts result in a Rate Limited / Blocked state. Logout or session expiry ends the session and returns the system to Idle.

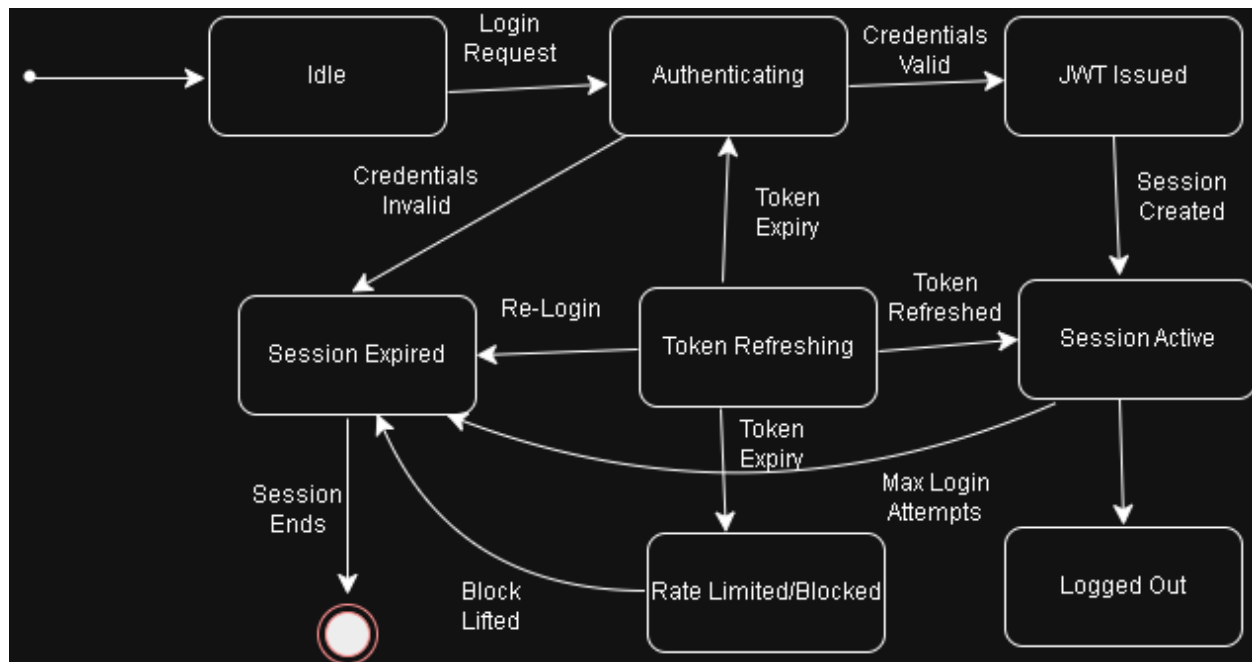


Figure 8: System Session State Diagram

7.2 Sequence Diagrams

A Sequence Diagram shows how actors and system objects interact in a time-ordered sequence to accomplish a specific use case. Messages are shown as horizontal arrows between vertical lifelines. Solid arrows represent synchronous calls; dashed arrows represent responses or asynchronous returns. Activation boxes indicate when an object is actively processing.

Figure 9: Student Registration & GitHub OAuth Sequence Diagram

This diagram illustrates the full onboarding flow for a new student. The student registers using a .edu email address, which the backend validates before inserting the user record into MongoDB and sending a verification email. After clicking the verification link, the student connects their GitHub account via OAuth 2.0 — the backend stores the access token and GitHub username. The account is then fully active and ready for group participation.

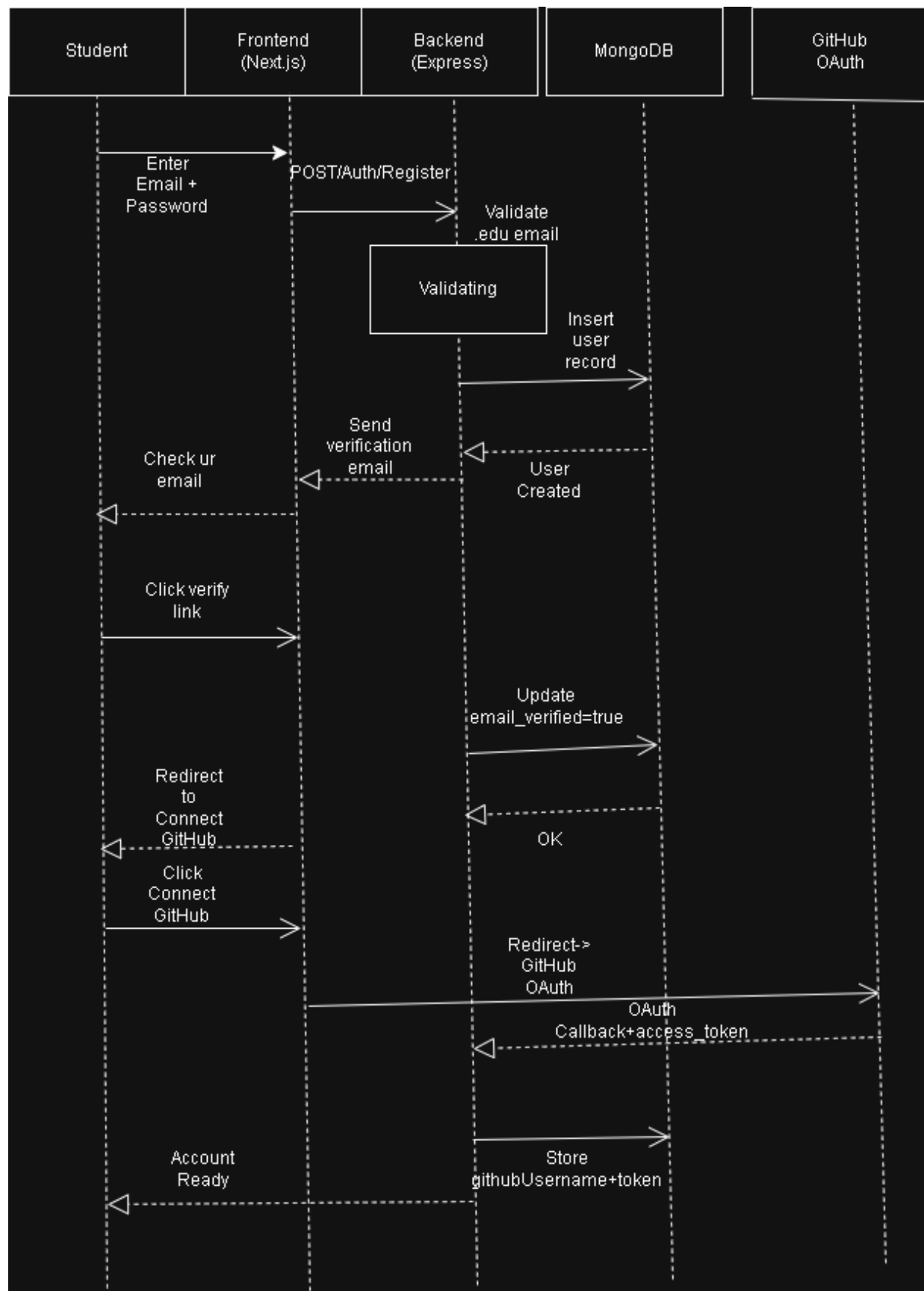


Figure 9: Student Registration & GitHub OAuth Sequence Diagram

Figure 10: Login & JWT Authentication Sequence Diagram

This diagram covers the authentication flow for both students and professors. The backend looks up the user by email and compares the password hash using bcrypt. On success, a signed JWT token is returned and the frontend establishes a persistent WebSocket connection via Socket.io for the live activity feed. Failed attempts show an error and increment the attempt counter. After exceeding the maximum attempts the account is blocked and an email notification is sent.

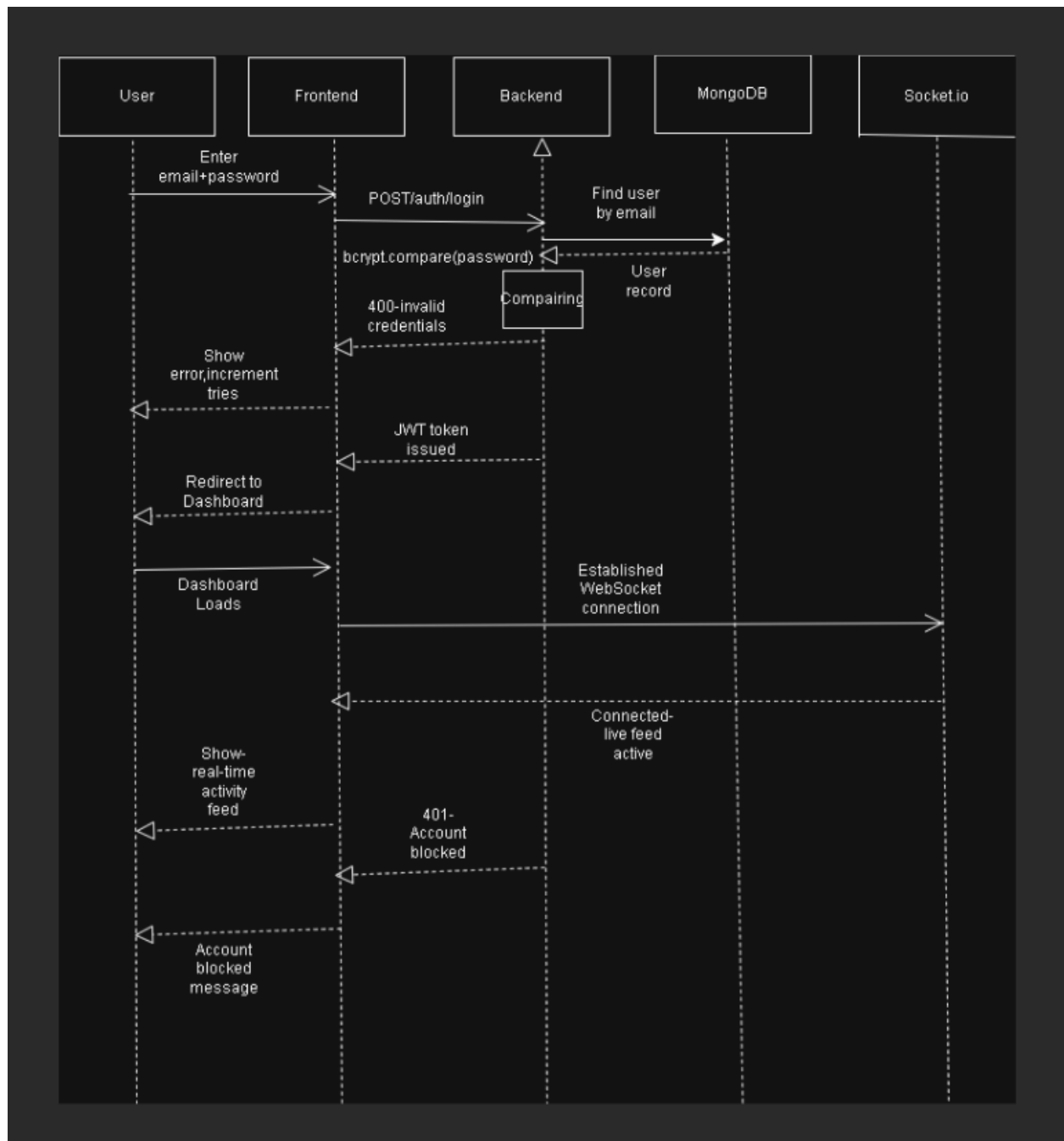


Figure 10: Login & JWT Authentication Sequence Diagram

Figure 11: GitHub Webhook & Contribution Processing Sequence Diagram

This diagram shows how GitHub push events are automatically processed. When a student pushes code, GitHub sends a signed webhook to the GroupSync backend. The signature is verified, the group is identified by repository URL, and commit data (count, lines added, commit messages) is extracted and stored. The Score Engine then fetches all weekly metrics, runs the 7-

metric scoring algorithm, updates the contribution record in MongoDB, checks free rider thresholds, and broadcasts the updated score to connected clients via Socket.io.

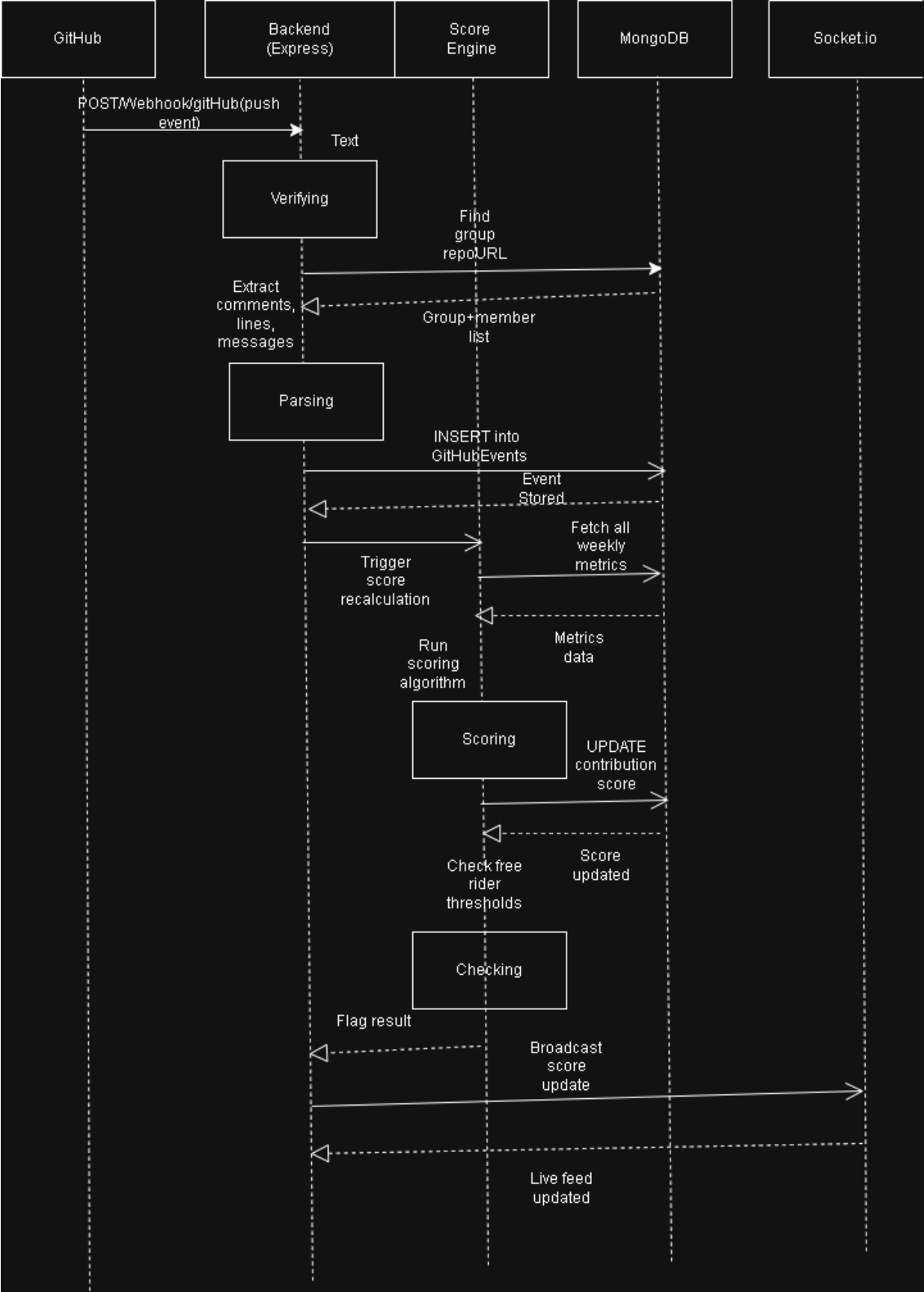


Figure 11: GitHub Webhook & Contribution Processing Sequence Diagram

Figure 12: Free Rider AI Detection & Alert Sequence Diagram

This diagram illustrates the AI-enhanced free rider detection workflow, triggered only when a student's weekly score falls below 50. The backend fetches the student's full metrics and sends a single structured prompt to the OpenAI API (GPT-3.5-turbo). The API returns a JSON object with fields: isFreeRider, confidence, reason, and action. Based on the action field — warn, review, or ignore — the system either sends a warning email to the student (via SendGrid), flags the student as HIGH RISK and alerts the professor, or takes no action.

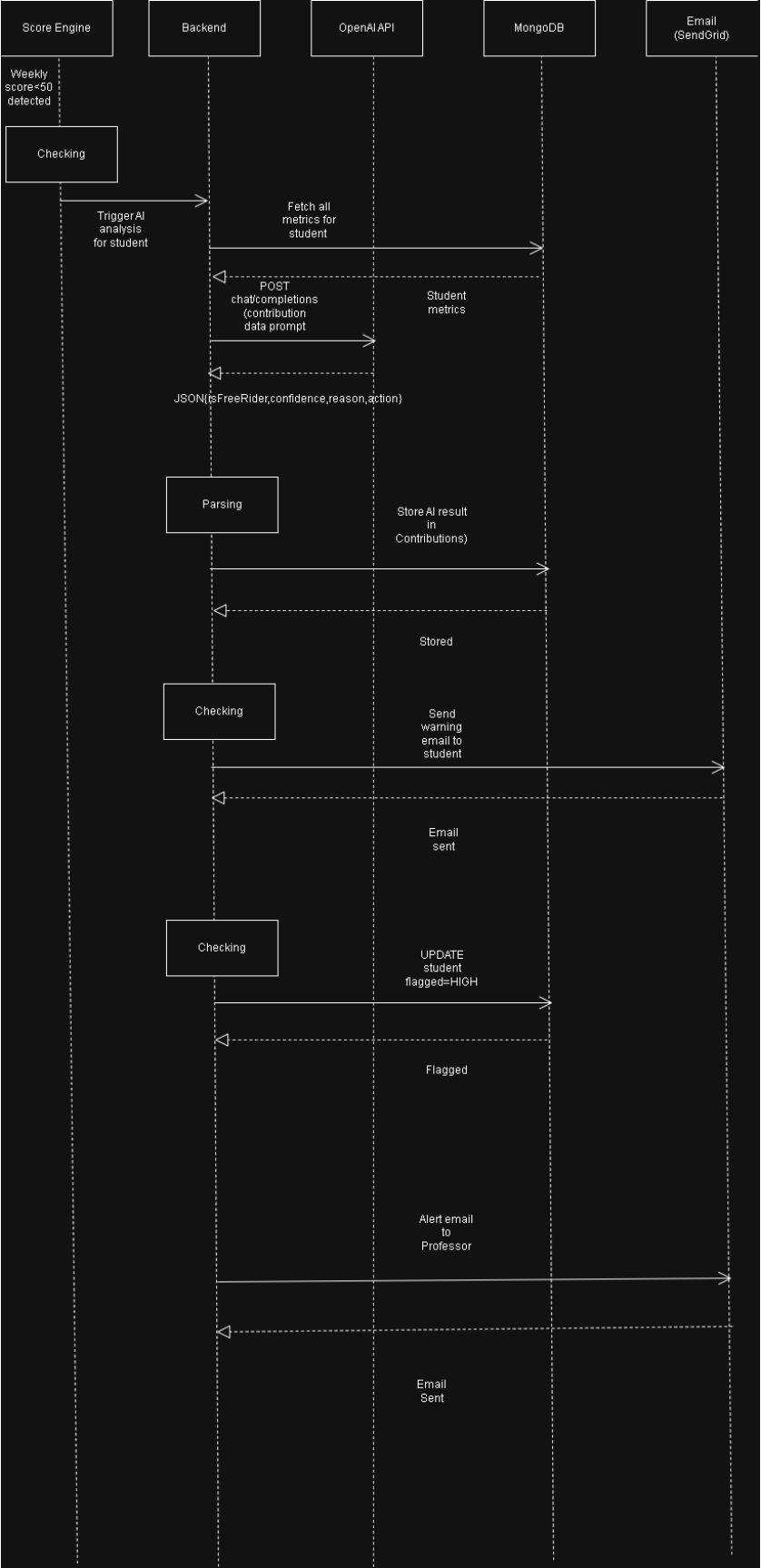


Figure 12: Free Rider AI Detection & Alert Sequence Diagram

Figure 13: Professor Report Generation Sequence Diagram

This diagram covers the professor's analytics and report generation workflow. After login, the professor views the course dashboard showing all groups and flagged students. Selecting a group and week filter triggers a data fetch that populates analytics charts and a contribution table. When the professor requests a PDF report, the backend fetches the complete group data and passes it to the PDF generation service, which returns a formatted buffer. The PDF is then streamed to the professor's browser for download.

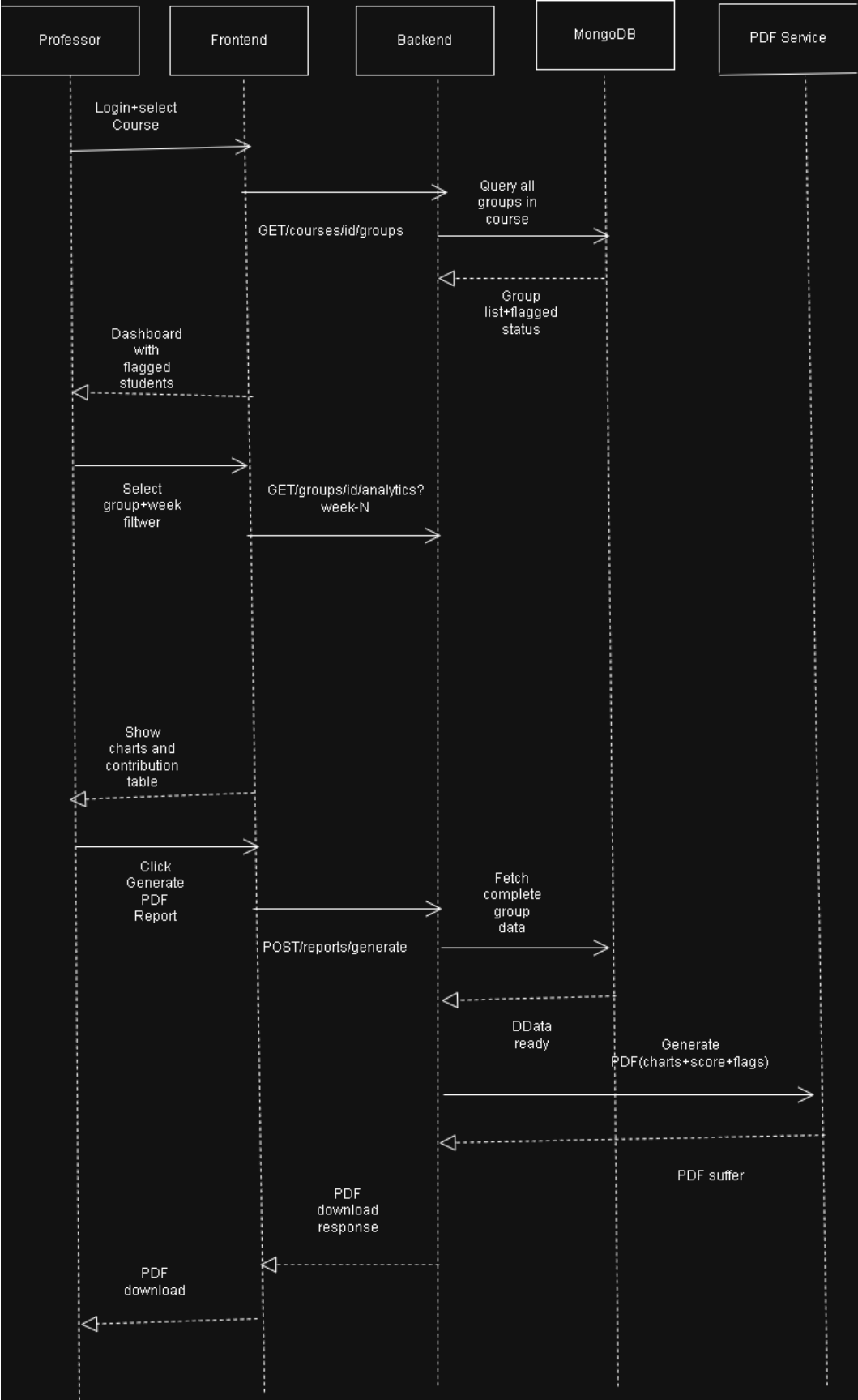


Figure 13: Professor Report Generation Sequence Diagram

8. Flow-Oriented Modeling

Data Flow Modeling provides a structural and process-oriented view of the GroupSync platform. Unlike object-oriented models that emphasize system states and class hierarchies, Data Flow Diagrams (DFDs) trace the lifecycle of data—illustrating how information is gathered from external entities, transformed through computational processes, mapped into internal persistent data stores, and generated as actionable outputs.

8.1 Context Diagram (Level 0 DFD)

The Context Diagram establishes the absolute operational boundary of the GroupSync system. It treats the entire platform as a single centralized processing hub (0.0 GroupSync System) and maps the high-level data streams connecting the system with four primary external entities: Students, Teachers, Administrators, and the GitHub REST API.

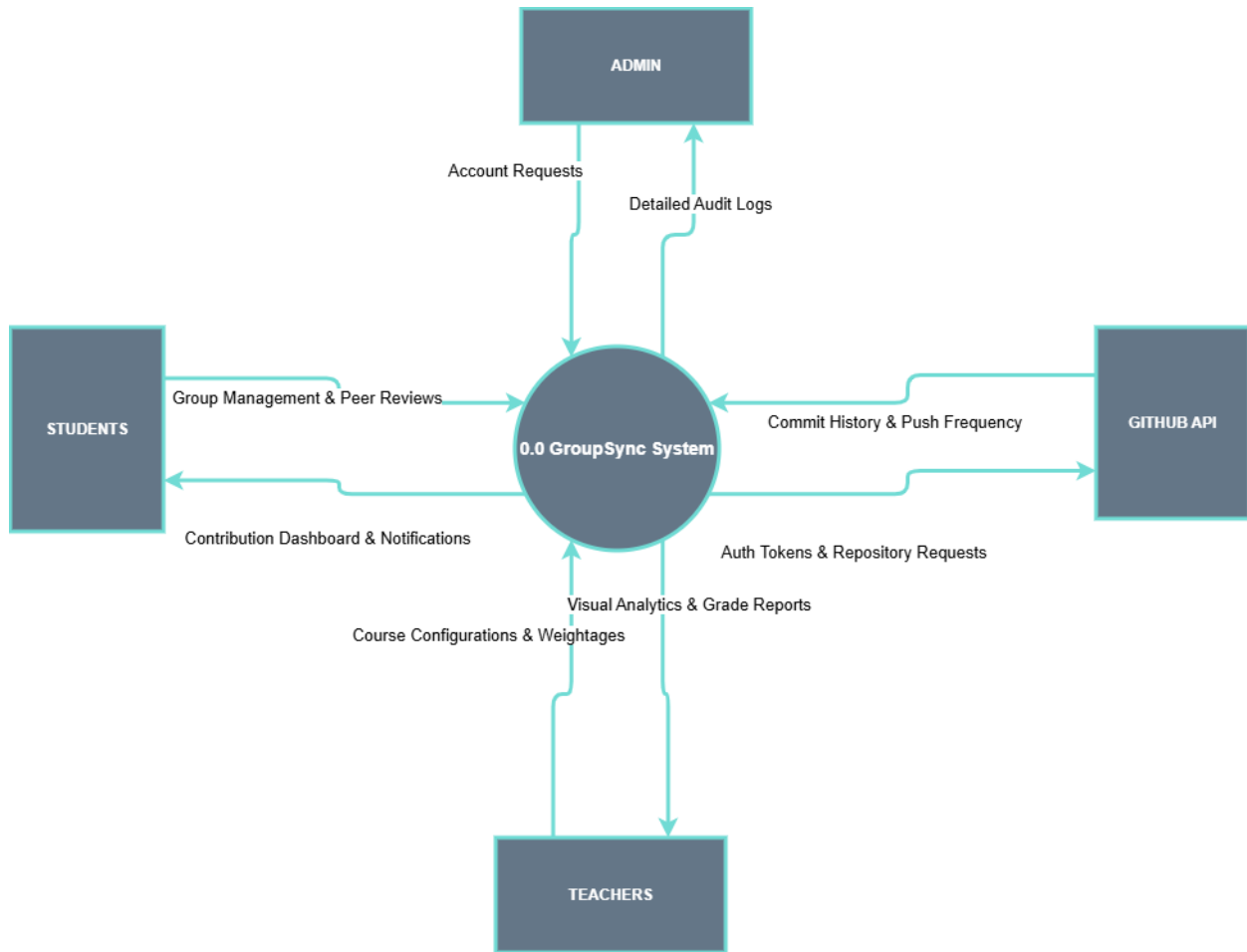


Fig01:Level 0 Data Flow Diagram for GrouSync System

Architectural Data Streams:

- **Students:** Input critical project metadata, including Group Management parameters and Peer Reviews. In return, the system feeds back calculated personal contribution metrics via the Contribution Dashboard and issues system notifications.
- **Teachers:** Input structural boundaries by defining Course Configurations and numerical Weightages for different project evaluation metrics. The system returns processed visual analytics and high-fidelity, data-driven Grade Reports.
- **Admin:** Transmits Administrative/Account Requests to govern user profiles and securely receives Detailed Audit Logs for monitoring platform health and compliance.

- **GitHub API:** Acts as an automated external agent. The system dispatches Auth Tokens and Repository Requests, while the API streams raw commit streams, branch histories, and push frequencies back into the system.

8.2 Level 1 Data Flow Diagram (DFD)

The Level 1 DFD deconstructs the unified system into five core sub-systems or processes. It presents a granular view of how data flows across specialized modules and details the exact read/write pathways into five dedicated data stores (D1 to D5).

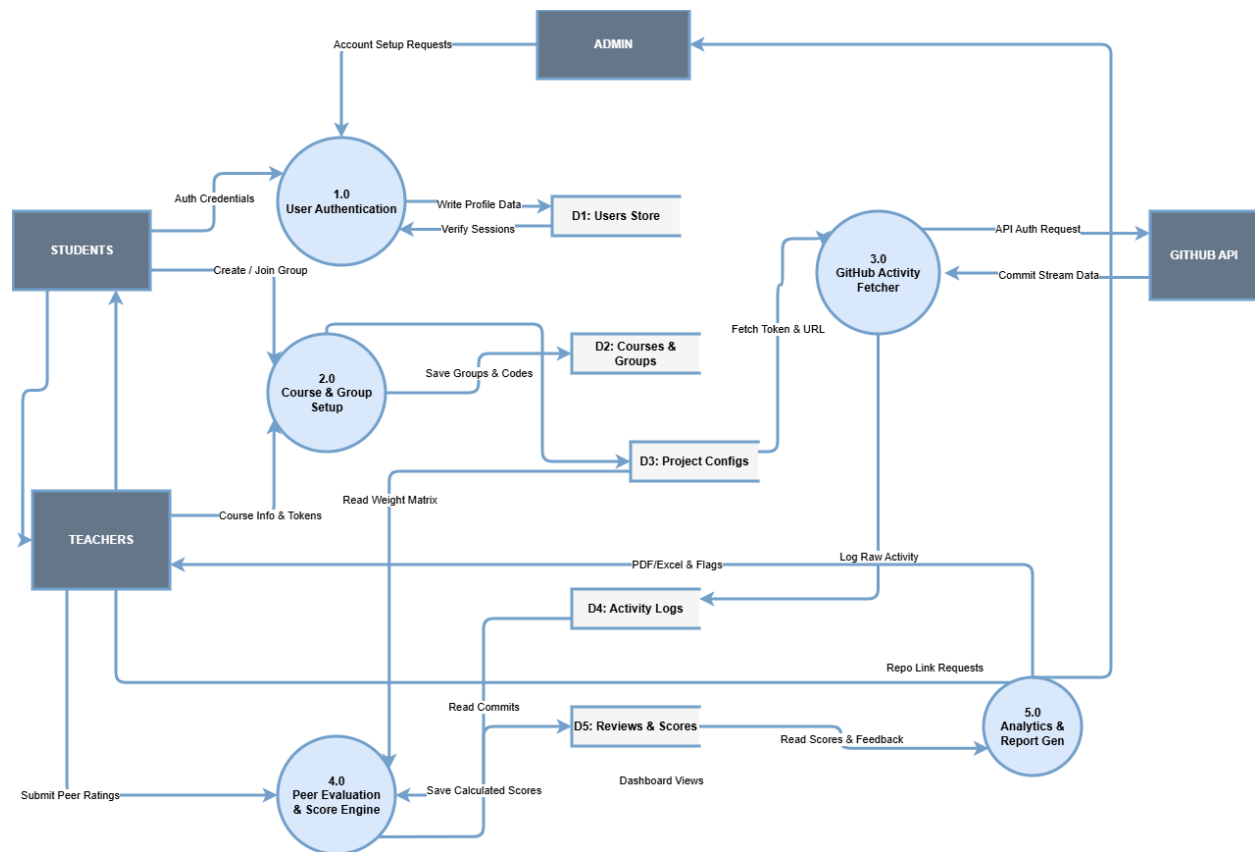


Fig01:Level 1 Data Flow Diagram for GrouSync System

Detailed Process Descriptions:

1. 1.0 User Authentication: Manages credential verification. It ingests login tokens from users, validates them against D1: Users Store, generates secure session states, and writes security footprints into D4: Activity Logs.
2. 2.0 Course & Group Setup: Handles academic indexing. Teachers post course rosters, and Students form teams via invitation codes. This process continuously populates and updates structural metadata inside D2: Courses & Groups.
3. 3.0 GitHub Activity Fetcher: Manages external API sync. Using repo links stored in D2 and configurations from D3, this automated daemon communicates with the GitHub API to fetch commit streams, which are written as aggregated logs into D4: Activity Logs.
4. 4.0 Peer Evaluation & Score Engine: The core algorithmic engine. It collects qualitative student reviews, saves them into D5: Reviews & Scores, reads repository activity from D4 and grading schemas from D3, and computes final individual contribution metrics.
5. 5.0 Analytics & Report Gen: The visualization and reporting pipeline. It reads computed ratings from D5 and raw activity streams from D4 to generate interactive charts for Teachers and compile downloadable PDF/Excel reports.

8.3 Level 2 DFD (Process 4.0 - Peer Evaluation & Score Engine)

To deeply evaluate the algorithmic integrity of the contribution scoring mechanism, Process 4.0 is exploded into its functional components. This Level 2 DFD displays the isolated sub-processes responsible for translating raw qualitative and quantitative inputs into a normalized score out of 100.

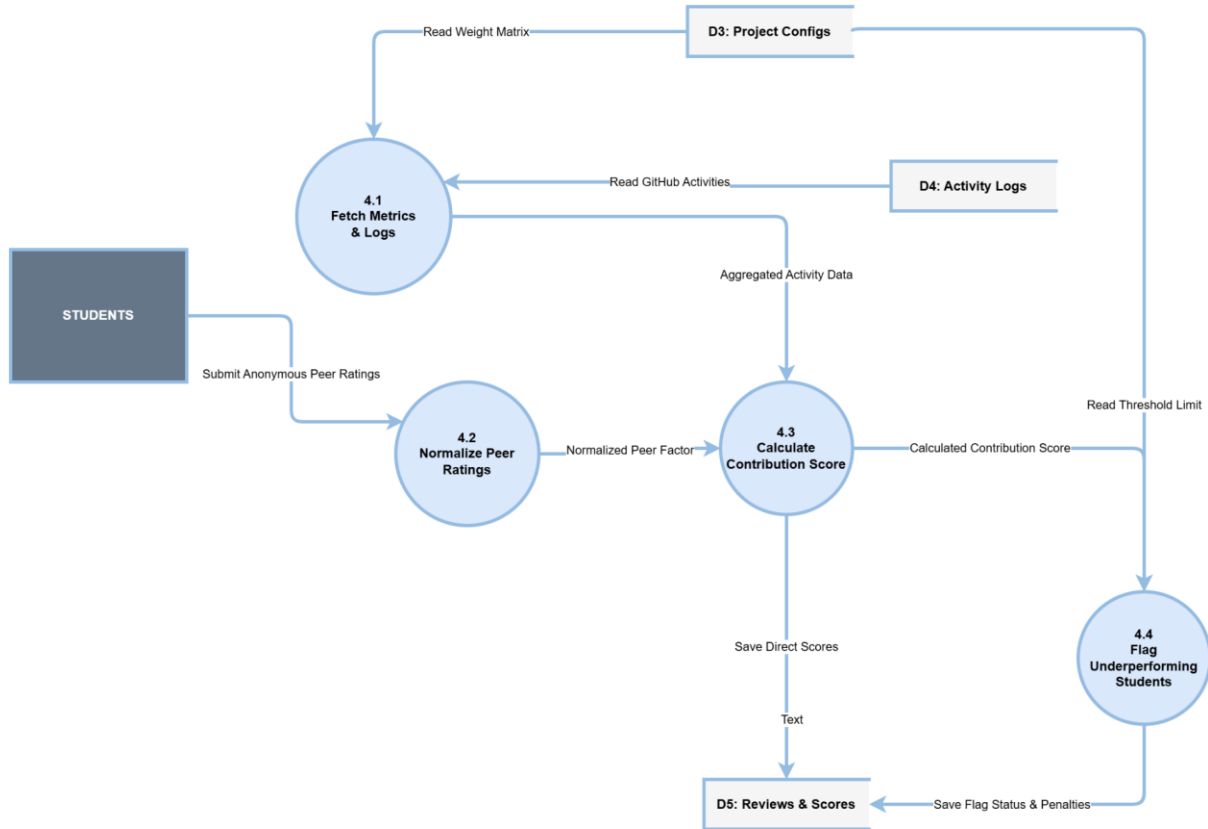


Fig01:Level 2 Data Flow Diagram for GrouSync System

Algorithmic Data Pipeline:

- **4.1 Fetch Metrics & Logs:** This sub-process triggers a data aggregation event. It concurrently pulls the specific project criteria from **D3: Project Configs** and extracts version control telemetry from **D4: Activity Logs** to establish a baseline of physical work done.
- **4.2 Normalize Peer Ratings:** This module ingests anonymous raw ratings from Students and executes a normalization process. It filters out extreme biases or outlier peer reviews, storing the balanced peer factor directly into **D5: Reviews & Scores**.
- **4.3 Calculate Contribution Score:** The mathematical core of the platform. It pulls the normalized peer factors from **D5** and the aggregated activity data from **4.1**. It applies the teacher-defined weightage matrix to compute a final, transparent 0-100 individual contribution score, saving the result back to **D5**.
- **4.4 Flag Underperforming Students:** A proactive monitoring agent. It streams the newly calculated scores from **4.3** and cross-references them against the strict alert threshold limits defined in **D3**. If a student's total involvement drops below the target percentage, it writes a flag status into **D5** and triggers an immediate warning notification.

9. Requirements Analysis Model

9.1 Functional Requirements Summary

The functional requirements define the services and operations that the GroupSync system must provide to its users.

Student Functions

- Register and authenticate using university credentials.
- Create or join project groups.
- Connect GitHub repositories.
- Submit anonymous peer reviews.
- View contribution scores and analytics.
- Receive notifications and warnings.

Teacher Functions

- Create and manage courses.
- Configure project assignments.
- Define evaluation weightages.
- Monitor contribution analytics.
- Generate PDF and Excel reports.
- Identify low-performing students.

Admin Functions

- Manage user accounts and roles.
 - Configure system settings.
 - Monitor audit logs.
 - Perform backup and recovery operations.
-

9.2 Non-Functional Requirements

Performance Requirements

- System shall support at least 100 concurrent users.
- Dashboard pages shall load within 3 seconds.
- Contribution scores shall be updated within 30 seconds after new GitHub activity is fetched.

Security Requirements

- Passwords shall be encrypted using bcrypt.
- JWT-based authentication shall be used.
- Role-based access control shall be enforced.
- All API communications shall use HTTPS.

Reliability Requirements

- System availability shall be at least 99%.
- Automated backups shall execute daily.
- Recovery time shall not exceed 30 minutes.

Usability Requirements

- User interface shall be responsive.
- Navigation shall remain consistent across devices.
- Error messages shall be understandable and actionable.

Maintainability Requirements

- System shall follow modular architecture.
- APIs shall be documented.
- Source code shall follow coding standards.

10. Requirements Validation

Requirements validation ensures that the documented requirements accurately represent stakeholder needs and are feasible for implementation.

Validation Techniques Used

Requirement Reviews

Stakeholders reviewed the documented requirements to verify correctness and completeness.

Scenario Validation

Usage scenarios were analyzed to confirm that all major workflows and exception cases were covered.

Consistency Checking

Requirements were checked for contradictions, ambiguity, and duplication.

Feasibility Analysis

Each requirement was evaluated based on technical feasibility, available resources, and project timeline constraints.

Validation Results

- Requirements are complete and traceable.
 - Functional and non-functional requirements are consistent.
 - Stakeholder conflicts have been resolved.
 - System scope is clearly defined.
 - Requirements are feasible within project constraints.
-

11. Risk Analysis

Technical Risks

Operational Risks

Risk	Impact	Mitigation
Biased Peer Reviews	Unfair evaluation	Rating normalization and outlier detection
Low User Adoption	Reduced effectiveness	User training and intuitive UI
Incorrect Weightage Configuration	Invalid scores	Validation rules enforcing 100% total

12. Requirements Traceability Matrix (RTM)

Requirement ID	Requirement	Use Case	Class	DFD Process
FR-01	User Login	Sign In	User	1.0 Authentication
FR-02	Create Group	Create Group	Group	2.0 Course & Group Setup
FR-03	Connect GitHub	Connect Repository	GitHubRepo	3.0 GitHub Activity Fetcher
FR-04	Submit Peer Review	Submit Review	PeerReview	4.0 Score Engine
FR-05	Generate Report	Generate Report	Report	5.0 Analytics & Report Generation

13. Conclusion

The GroupSync system has been designed to address the longstanding challenge of fair and transparent assessment in university group projects. Through stakeholder-driven requirements engineering, structured system modeling, GitHub activity integration, peer evaluation mechanisms, and automated contribution analytics, the proposed solution provides a reliable framework for measuring individual participation within collaborative academic environments.

The combination of use case modeling, data modeling, class-based design, behavioral analysis, and data flow modeling ensures that the system is both technically feasible and aligned with

stakeholder expectations. Furthermore, the proposed architecture promotes fairness, accountability, transparency, and scalability, making GroupSync a valuable platform for modern project-based learning environments.